

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology, Bangladesh



A Thesis report on

**A Deep Reinforcement Learning Approach for Manipulator Control
with Model Compression for Reduced Computation and Memory
Requirements**

**PREPARED
BY**

**Fahim Faisal (ID: 2008001)
Hodayra Mobesshira Hemu (ID: 2008005)**

Department of Mechatronics Engineering

JULY 2026

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology, Bangladesh



Department of Mechatronics Engineering

A Thesis report on

**A Deep Reinforcement Learning Approach for Manipulator Control with
Model Compression for Reduced Computation and Memory Requirements**

This thesis report is submitted in partial fulfilment of the requirements for the degree of Bachelor of Science in Mechatronics Engineering at Rajshahi University of Engineering & Technology, Rajshahi, Bangladesh.

Supervised by

Dr. Dip Kumar Saha

Assistant Professor

Mechatronics Engineering

Rajshahi University of Engineering
& Technology.

Submitted by

Fahim Faisal

ID: 2008001

Homayra Mobesshira Hemu

ID: 2008005

Heaven's Light Is Our Guide

Rajshahi University of Engineering & Technology, Bangladesh

Department of Mechatronics Engineering

Certificate

This is to certify that the thesis entitle “ A Deep Reinforcement Learning Approach for Manipulator Control with Model Compression for Reduced Computation and Memory Requirements” has been carried out by Fahim Faisal , Roll: 2008001 and Homayra Mobesshira Hemu, Roll: 2008005 under my supervision in the Department of Mechatronics Engineering at Rajshahi University of Engineering & Technology, Bangladesh.

Supervised by-

.....

Dr. Dip Kumar Saha

Assistant Professor

Dept. of Mechatronics Engineering

Rajshahi University of Engineering
& Technology.

Countersigned

.....

Head

Dept. of Mechatronics Engineering

Rajshahi University of Engineering
& Technology

Signature of the students -

.....

Fahim Faisal

Roll No.: 2008001

.....

Homayra Mobesshira Hemu

Roll No.: 2008005

External

.....

Sarafat Hussain Abhi

Asistant Professor

Dept. of Mechatronics Engineering
Rajshahi University of Engineering
& Technology

Declaration

We, Fahim Faisal and Hodayra Mobesshira Hemu, hereby declare that we are the sole author of the thesis entitled “A Deep Reinforcement Learning Approach For manipulator Control With Model Compression For Reduced Computation & Memory Requirement”. This is written and submitted by us to the Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology, in partial fulfilment of the requirements for acquiring the degree of Bachelor of Science in Mechatronics Engineering. This work was done under the supervision of Dr. Dip Kumar Saha, Assistant Professor, Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology.

We certify that, to the best of our knowledge, our thesis does not infringe upon anyone’s copyright nor violate proprietary rights, and any ideas, technique, quotations or any other material from the work of other people included in my thesis, published or otherwise, are fully acknowledged in accordance with the standard reference practices.

We further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this university or any other institutions or universities.

Authors

Acknowledgement

We want to express our gratitude to all the persons who directly or indirectly contributed in the completion of this thesis. First and foremost, we would like to express our outmost gratitude to the Almighty Allah for blessing us with the ability to complete this thesis.

We would like to express our sincere gratitude to our supervisor Dr. Dip Kumar Saha, Assistant Professor, Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology, for his invaluable guidance and support throughout the journey of this thesis. His knowledge and interest in the field of Robotics really helped us to overcome multiple hurdles. It makes no doubt that the results of this thesis would have not been the same without his assistance.

The authors would also like to express their sincere gratitude to Sarafat Hussain Abhi, Assistant Professor, Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology, for his guidance, support and cooperation in completing the thesis.

Our special gratitude goes to Md. Firoj Ali, Associate Professor and Head, Department of Mechatronics Engineering, Rajshahi University of Engineering & Technology, for providing us with insightful suggestions and best facilities available in the department. We also would like to thank all the faculty members of Department of Mechatronics Engineering for their invaluable suggestions and support. We would also like to thank all our classmates for their timely supports and helps. Most importantly we like to thank our parents for their never ending support and encouragement in difficult times of our life.

RUET, Rajshahi
July, 2026

Authors
Fahim Faisal
ID: 2008001
Hodayra Mobesshira Hemu
ID: 2008005

Abstract

Embedded deployment of DRL robotic manipulation policies is constrained for deployment on cheap MCUs like esp32 devkit by the limited memory, compute, and power budgets, where large CNN based pose perception models are specifically the biggest hurdle. Solving these resource requirements makes robotics more accessible, efficient and thus, cheaper. This thesis introduces a framework for deploying reinforcement-learning controller for a 7 degree of freedom robotic manipulator on ultra resource constrained MCUs using task decomposition to avoid large monolithic models, fiducial marker for on device perception avoiding large CNN networks. The Proposed framework is demonstrated through a pick-and-place task with a learned grasp policy, a learned transport policy, and a deterministic finite-state-machine sequencing layer. The policies were trained using Twin Delayed Deep Deterministic Policy Gradient in a robosuite/MuJoCo simulation and deployed by quantizing the model to INT8 TensorFlow Lite Micro on a plain ESP32 microcontroller with no PSRAM. The work demonstrated is demonstrated with a fully randomized pickable object spawn position and orientation, with an on-device perception pipeline with a memory-reduced AprilTag detector and a compact FP32 residual pose corrector which run on the same microcontroller as the two control policies. The INT8 fidelity gap that was closed by quantization campaigns, which included weight-range clipping, in-loop quantization-aware training, replay-buffer scaling and matched critic capacity, and finally, the deployed INT8 grasp actor is found to be at parity with its FP32 original on fixed spawn. Training a randomly spawn aware controller, it achieves 95.33% success in 100 trials (91.5–99.4% 95% CI), while 99.83% of the fixed-spawned trials are successful (99.3–99.8% 95% CI), and a HIL run with a real microcontroller verifies that 97 of 100 episodes are successful (91.5–99.4% 95% CI). The on-device perception pipeline achieves 89.67% task success with an 84% of the ground truth task success, with a median raw pose error of 0.06 mm compared to 20.48 mm for the detector, and operating alongside both of the control policies in the microcontroller's heap of approximately 210 kB with about 36 kB of space to spare.

List of Contents

Declaration	i
Acknowledgement.....	ii
Abstract	iii
List of Figures	viii
List of Tables.....	ix
List of Abbreviations	x
Chapter 1	1
Introduction.....	1
1.1 Overview.....	1
1.2 Background and Motivation	2
1.3 Research Question	4
1.4 Objectives	5
1.6 Organization of the Report	6
1.7 Summary.....	7
Chapter 2	9
Literature Review	9
2.2 Existing Research	10
2.2.1 Options Framework and Sub-MDP Decomposition:	10
2.5 Comparative Analysis.....	13
2.4 Summary.....	20
Chapter 3	21
Engineering Design and Methodology	21
3.1 Introduction.....	21
3.2 Methods and Tools	21
3.3 System Requirements and Constraints	25
3.4 Design Requirements.....	26
3.3.1 Model Size Requirements.....	26
3.3.2 Inference Performance Requirements.....	27
3.3.3 Inference Frequency Requirements for Continuous Control	27
3.4.4 On-Device Perception Requirements	28

3.5 Model Design and Performance Criteria	29
3.6 Design Steps	30
3.6.1 Conceptual System Design Steps	30
3.6.2 Training Algorithm Design Steps	31
3.7 Proposed System Model	33
3.7.1 Preliminary Monolithic Model	33
3.7.2 Our Model	35
3.7.3 Perception-Augmented Object Pose	37
3.8 Summary	38
Chapter 4	39
System Implementation	39
4.1 Introduction	39
4.2 Methodology of Formulating The RL Problem	40
4.3 Methodology of Training	41
4.4 Methodology of Compression	43
4.5 Selection of Components	44
4.6 Optimization	45
4.7 Principle of Execution	51
4.8 On-Device Perception Implementation	52
4.8.1 Detection Pipeline	52
4.8.2 Memory-Reduced AprilTag Port	53
4.8.3 Residual Pose Corrector	53
4.8.4 Latching Through the Grasp	54
4.9 Summary	55
Chapter 5	56
Experimental Set up	56
5.1 Introduction	56
5.2 Methodology of Validation	56
5.2.1 First stage: offline check on the computer	56
5.2.2 Second stage: hardware-in-the-loop check	57
5.3 Experimental Setup	58
5.3.1 Training computer	59
5.4 Test Bench	60
5.4.1 Serial protocol	60

5.4.2 Software parts	60
5.4.3 Perception protocol.....	61
5.5 Instrumentation	62
5.5.1 Training-side instruments	62
5.5.2 Bench-side instruments.....	62
5.6 Calibration	63
5.6.1 Quantizer calibration	63
5.6.2 Link calibration	63
5.6.3 Metric calibration	63
5.6.4 Perception / corrector calibration.....	63
5.7 Experimental Procedure.....	64
5.8 Data Collection	65
5.9 Design Assessment and Performance Indices	66
5.9.1 Training indices	66
5.9.2 Fixed evaluation and the rule-layer ablation.....	66
5.9.3 Hardware-in-the-loop indices	67
5.10 Summary.....	68
Chapter 6	69
Results And Discussion.....	69
6.1 Introduction.....	69
6.2 Investigation and Analysis of Data.....	69
6.2.1 Grasp-Stage Training	70
6.2.2 Place-Stage Training Dynamics.....	70
6.2.3 Failure-Mode Analysis	71
6.2.4 Controlled Comparisons and Ablations	72
6.2.5 Random-Spawn Grasp Training and Quantization Dynamics	75
6.2.6 On-Device Perception Detection Dynamics	77
6.3 Results.....	77
6.3.1 End-to-End Task Success and the Rule-Layer Ablation	77
6.3.2 Failure Anatomy of the Final System	78
6.3.3 INT8 Compression: Cost and Fidelity	79
6.3.4 Hardware-in-the-Loop Results	80
6.3.5 On-Device Perception Results.....	82
6.4 Discussion.....	84
6.4.1 Meeting the Design Requirements.....	85
6.4.2 What the ablation means.....	86
6.4.3 Limitations.....	86

6.5 Summary.....	87
Chapter 7	88
Socio-Economic Impact and Sustainability	88
7.1 Introduction.....	88
7.2 Impact of the Thesis on Societal, Health, Safety, Legal and Cultural Issues	88
7.2.1 Societal Impact	88
7.2.2 Health and Safety Impact	89
7.2.3 Legal Impact.....	90
7.2.4 Cultural Impact.....	90
7.3 Impact of the Thesis on the Environment and Sustainability	91
7.3.1 Energy Efficiency of the Deployed System	91
7.3.2 Energy Saving in Training	91
7.3.3 Right-Sized Hardware and Responsible Use of Resources	92
7.3.4 Relation to the Sustainable Development Goals (SDGs)	92
7.4 Summary.....	93
Chapter 8	94
Thesis Management & Finance	94
8.1 Introduction.....	94
8.2 Management of the Thesis.....	94
8.2.1 Time Frame.....	95
8.3 Overall Budget.....	96
8.3.1 Component/Software Budget.....	96
Chapter 9	100
Conclusions, Limitations and Future Works	100
9.1 Conclusion	100
9.2 Limitations.....	101
9.3 Future Works	102
E . Addressing Complex Engineering Activities	114

List of Figures

FIGURE 3.1 TWO STAGE MODEL FOR THE PICKPLACE TASK, REPRESENTING GRAP AND PLACMODEL (TRANSPORTER) WITH HANDOFF HANDLER IN BETWEEN. AND THEN FINALLY THE P CONTROLLER TAKES OVER TO LOWER AND RELEASE THE OBJECT.	35
FIGURE 4.1: ROBOSUITE PICK-AND-PLACE ENVIRONMENT WITH 7-DOF FRANKA PANDA ROBOTIC MANIPULATOR WITH PICKED OBJECT (BREAD) BESIDE A TABLE VISUALIZED.	40
FIGURE 4.2: REWARD DESIGN FOR TRAINING INVOLVING VARIOUS SUBTASKS.	42
FIGURE 4.3: METHODOLOGY OF COMPRESSION WITH QAT DETAILING FINETUNING LOOP FOR OPTIMIZING WEIGHTS.	43
FIGURE 4.4: CUMULATIVE EFFECT OF THE RANDOM-SPAWN INT8 QUANTIZATION LEVERS.	47
FIGURE 4.5: FSM DIAGRAM FOR THE WHOLE TASK.	52
FIGURE 4.6: ON-DEVICE APRILTAG DETECTION AND CORRECTION PIPELINE.	54
FIGURE 4.7: ESP32 ON-CHIP MEMORY BUDGET WITH BOTH ACTORS AND THE PERCEPTION PIPELINE RESIDENT.	54
FIGURE 5.1: OFFLINE TESTING ARCHITECTURE OF THE TRAINED MODEL ON THE COMPUTER. ONLY A MODEL WITH A PASS ON THE THRESHOLDS GOES TO THE SECOND STAGE.	57
FIGURE 5.2: FLOW OF THE HARDWARE-IN-THE-LOOP CHECK. THE ESP32 IS THE ONLY SOURCE OF CONTROL ACTIONS. THE PHYSICS STAYS ON THE COMPUTER.	58
FIGURE 5.3: SERIAL FRAME LAYOUTS FOR THE STATE, PERCEPTION-CORRECTION AND RAW-IMAGE PROTOCOL VARIANTS.	64
FIGURE 6.1: GRASP-STAGE (STAGE 1) TRAINING CONVERGENCE. ROLLING SUCCESS RATE (WINDOW = 100 EPISODES) REACHES AND MAINTAINS $\approx 95\%$, TRIGGERING EARLY STOPPING AT EPISODE $\approx 1,020$	70
FIGURE 6.2: REVERSE CURRICULUM & TRAINING PROGRESSION, ROLLING SUCCESS RATE (WINDOW = 100 EPISODES) ACROSS REVERSE CURRICULUM PHASES. THE TARGET BIN DISTANCE D DECREASES AS TRAINING PROGRESSES, INCREASING TASK DIFFICULTY FROM EASY $\rightarrow$ MEDIUM $\rightarrow$ HARD $\rightarrow$ MAX	71
FIGURE 6.3: EACH BAR DESCRIBES EVALUATIONS OF 50 EPISODES. PERCENTAGES MAY NOT SUM TO 100 DUE TO ROUNDING. REVERSE CURRICULUM: DISTANCE D (INITIAL OBJECT-BIN CENTER DISTANCE) DECREASES FROM 0.20 M $\rightarrow$ 0.14 M $\rightarrow$ 0.10 M $\rightarrow$ 0.05 M.	72
FIGURE 6.4: DETAILED COMPARISONS OF DIFFERENT CONTROLLED ENVIRONMENT SETTINGS OF INVOLVING DIFFERENT TRAINING RUN.	74
FIGURE 6.5: RANDOM-SPAWN GRASP ACTOR OF INT8 SUCCESS AND SEED VARIANCE ACROSS THE QUANTIZATION CAMPAIGN. SEED-TO-SEED SPREAD COLLAPSE CAN BE NOTICED.	76
FIGURE 6.6: CUMULATIVE EFFECT OF EACH QUANTIZATION LEVER.	76
FIGURE 6.7: MODEL SIZE COMPARISON OF OUR MODEL VS VARIOUS CONFIGURATIONS OF ROBOMIC [47].	79
FIGURE 6.8: VISUAL INFOGRAPHIC OF THE HIL PERFORMANCE.	82
FIGURE 6.9: ON-DEVICE PERCEPTION ACCURACY AGAINST GROUND TRUTH CEILING.	83
FIGURE 6.10: RESIDUAL-CORRECTOR EFFECT ON POSE ERROR.	84
FIGURE 6.11: ESP32 SERIAL LOG AT THE MOMENT OF ON-DEVICE APRILTAG DETECTION.	84
FIGURE 6.12: TASK COMPLETION RATE OF WITH AND WITHOUT IMPLEMENTATION OF THE FSM INVOLVING THE P-CONTROLLER.	85

List of Tables

TABLE 2.1: CHRONOLOGICAL COMPARISON OF RELATED WORKS AND THEIR RELEVANCE TO THIS THESIS.....	13
TABLE 3.1: DESIGN REQUIREMENTS AND THE CORRESPONDING DESIGN RESPONSES.	28
TABLE 3.2: PERCEPTION REQUIREMENT AND THEIR CONSTRAINTS.....	28
TABLE 3.3: THE FINITE STATE MACHINE META-CONTROLLER: LEARNED SUB-POLICIES (STATES 1, 3) INTERLEAVED WITH SCRIPTED MICRO-SKILLS (STATES 2, 4).....	31
TABLE 4.1: MAIN SOFTWARE COMPONENTS AND WHAT EACH ONE IS USED FOR.	44
TABLE 4.2: RANDOM-SPAWN INT8 QUANTIZATION LEVERS, APPLIED CUMULATIVELY.	46
TABLE 5.1: DETAILS OF THE SIMULATION SET-UP AND OF THE TRAINING COMPUTER.....	59
TABLE 5.2: ELEMENTS OF THE HARDWARE-IN-THE-LOOP TEST BENCH.	60
TABLE 5.3: FINAL ON-DEVICE PERCEPTION DEPLOYMENT CONFIGURATION.	61
TABLE 5.4: INSTRUMENTS USED FOR DIAGNOSIS AND FOR MEASUREMENT.	62
TABLE 5.5: RULES OF THE EXPERIMENTAL PROCEDURE AND THEIR EFFECT ON THE DATA.	65
TABLE 5.6: DATA COLLECTED AT THE TWO LEVELS.	66
TABLE 5.7: RULE-LAYER ABLATION. THE GAIN FROM 78% TO 92% COMES FULLY FROM THE RULE-LAYER VALUES. THE TRAINED NETWORK HAS NO CHANGE.	67
TABLE 5.8: DESIGN RULES MAPPED TO PERFORMANCE INDICES AND TO LEVELS OF JUDGEMENT.	67
TABLE 6.1: PENALTY-SHAPE SWEEP. UNIFORM PRESSURE, NOT SHAPED PRESSURE, IS THE WORKING MECHANISM.	73
TABLE 6.2: SUMMARY OF CONTROLLED ABLATION STUDY.....	74
TABLE 6.3: DETERMINISTIC END-TO-END EVALUATION. THE 78% $\rightarrow$ 92% GAIN COMES ENTIRELY FROM RULE-LAYER PARAMETERS; THE LEARNED NETWORK IS UNCHANGED....	78
TABLE 6.4: DEPLOYED SYSTEM (GRASP ACTOR c2m512_s1 + ORIGINAL TRANSPORT ACTOR), HELD OUT ON 12 SEEDS $\times$ 100 EPISODES, ANCHOR OFF.	78
TABLE 6.5: INT8 COMPRESSION — SIZE, LATENCY AND HELD-OUT FIDELITY AGAINST FP32.	80
TABLE 6.6: HARDWARE-IN-THE-LOOP TIMING AND LINK STATISTICS.	81
TABLE 6.7: END-TO-END TASK SUCCESS BY PERCEPTION ROUTE (12 SEEDS $\times$ 50 EPISODES, RANDOM SPAWN).....	82
TABLE 7.1: POWER CONSUMPTION COMPARISON BETWEEN ESP32 AND NVIDIA JETSON NANO.....	91
TABLE 7.2: TRAINING COMPARISON BETWEEN MONOLITHIC AND STAGED APPROACHES.	92
TABLE 8.1: TIME FRAME OF THE THESIS WORK.	95
TABLE 8.2: GANNT CHART OF THE THESIS PLANNING.	95
TABLE 8.3: OVERALL BUDGET OF THE THESIS PROJECT.....	96
TABLE 8.4: DETAILED COMPONENT AND SOFTWARE BUDGET.....	96

List of Symbols & Abbreviations

DRL	Deep Reinforcement Learning
SRAM	Static Random Access Memory
ESP32	Espressif 32-bit Microcontroller
FSM	Finite State Machine
TD3	Twin Delayed Deep Deterministic Policy Gradient
INT8	8-bit Integer Number
FP32	32-bit Floating Point Number
MCU	Microcontroller Unit
QAT	Quantization-Aware Training
HIL	Hardware-in-the-Loop
MLP	Multi-Layer Perceptron
HRL	Hierarchical Reinforcement Learning
RL	Reinforcement learning
NN	Neural Network
MDP	Markov Decision Process
PTQ	Post-Training Quantization
TFLite	TensorFlow Lite
DOF	Degrees of Freedom
OSC_POSE	Operational Space Pose Controller
r	Shaping reward
γ	Discount factor
$\Phi(s)$	Potential function
$\Phi(s')$	Potential of the next state s'
CNN	Convolutional Neural Network
RNN	Recurrent Neural Network
PER	Prioritized Experience Replay PER
SDGs	Sustainable Development Goals
AI	Artificial Intelligence
DevKit	Development Kit
IDE	Integrated Development Environment
GPU	Graphics Processing Unit
CPU	Central Processing Unit
UART	Universal Asynchronous Receiver/Transmitter
USB	Universal Serial Bus
XNNPACK	eXtreme Neural Network PACKage
PT2E	PyTorch 2 Export
WSL2	Windows Subsystem for Linux 2

Chapter 1

Introduction

1.1 Overview

Currently, there are many applications of robotic manipulators in modern industry. The typical jobs are pick and place, assembly and material handling. The control of these robots is done by classical methods. A conversion of a target pose into joint angles is given by inverse kinematics, and a generation of a collision-free path is given by a motion planner. Under known conditions, the performance of these methods is very good, and a change of the object position is handled by them without difficulty.

A dynamic model of the robot is a requirement of these methods. The determination of the link inertias, of the friction coefficients and of the contact parameters is a slow work. Another requirement on the engineer is to derive a control law from the equations of motion. The construction of such a model becomes difficult and the accuracy of the model becomes low when the contact is rich, the object is deformable, or the friction is unknown.

Deep Reinforcement Learning (DRL) provides an alternative. A control policy is learned by trial and error within a simulator [1,2]. This is similar to the learning of a new skill by a human through practice. No derivation of a control law from the equations of motion is made by the engineer. Instead, a specification of a reward function is made.

Here, the engineering work does not decrease, but the nature of the work changes. Where earlier, more effort was required to create a model of the robot's dynamics, now that effort goes towards designing the task structure and reward function. A major advantage of this is that the same reward function can be used for different robots. But the dynamics model or inertia information created for a particular robot cannot be easily used for another robot.

Then comes the question of practical or hands-on application. Is the execution of such a learned policy possible on cheap hardware? The microcontroller ESP32 is low cost, low power

but comes with limited SRAM and limited Flash memory [3] [4]. One thing that is commonly stated is that a DRL model can't fit in such a chip. If the policy is a camera image-taking policy, the statement is correct, since the number of parameters for a vision encoder is very large. The statement is not correct for a policy having a low dimensional state vector.

The real problems are of a different kind. When a task is very large and the reward for success is received very late, it becomes difficult for the agent to learn properly or explore new paths. It is not possible to do the entire task well with a single large policy. Training such a large policy requires a lot of data and computing power, which is not possible on a typical laptop. In addition, a hard real-time system must make decisions very quickly and within a certain time. But an iterative numerical solver cannot produce results in the same fast and precise time.

This thesis performs a treatment of these obstacles. A staged deep reinforcement learning approach is proposed for the manipulator control. A division of the pick and place task into stage-specific policies is made, and a very small neural network is employed by each policy. Next, trained policies are compressed using INT8 quantization-aware training. Then the trained policies are compressed using INT8 quantization-aware training. A Finite State Machine (FSM) is used as a supervisor for the whole system and the switch between the stages occurs when certain transition conditions are met. Not all stages are learning stages. A test-lift check is scripted and the deployment cost of a release routine is close to zero. A complete path is shown, from the training of a policy in simulation up to its execution on resource-constrained embedded hardware.

1.2 Background and Motivation

Robotic manipulator control is currently an important application of Deep Reinforcement Learning (DRL). In conventional control methods, an engineer has to analyze the dynamics equations of the robot and create control formulas. But in reinforcement learning, the robot learns the appropriate control policy itself through simulation in a trial-and-error manner.

One thing needs to be clearly mentioned. Even using reinforcement learning, the need for physics is not completely eliminated. Because the simulator used for training is itself based on a dynamics model. In addition, the output of the policy learned in this study is finally sent to a conventional Operational Space Controller. That is, RL reduces the need to create control formulas by hand, but does not replace the physics model.

Significant success has been achieved in robotic manipulation through this method. Using algorithms such as Twin Delayed Deep Deterministic Policy Gradient (TD3) [5], [6], it has been possible to train a robot arm to reach, grasp, and place an object in a simulation. However, most studies have used a monolithic policy [2], [7]. That is, a single neural network is trained to perform the entire pick-and-place task. As a result, the same model has to learn multiple behaviors such as approaching, grasping, lifting, carrying, placing, and manipulating objects at different positions simultaneously.

The main problem with such an approach is not the size of the model, but the complexity of the training. Since pick-and-place is a long-term task, rewards are only given for successful completion of the task. As a result, rewards are sparse and it becomes difficult for the agent to understand which tasks were important for success [14], [15]. As a result, training is slow, unstable, and often fails. This problem is not solved by using large neural networks alone. In fact, large models can often learn incorrect behavior by exploiting poorly designed rewards. Therefore, the solution to the problem is not to increase the size of the model, but to change the structure of the task.

This observation is the main motivation for this study [8], [9]. If a single policy cannot effectively learn the entire task, then the task should be divided into smaller steps. In this study, the entire pick-and-place task is divided into multiple sub-steps. One policy is trained for the Grasp step and another policy for the Transport step. On the other hand, simple tasks such as releasing the object are implemented as scripted routines instead of being taught. As a result, each policy only needs to learn a specific behavior, which can be effectively taught with small neural networks.

A Deterministic Finite State Machine (FSM) then executes these steps in the correct order. In addition, the entire pick-and-place task is completed using scripted micro-skills such as Test-Lift Check and Release Routine.

A point needs to be clarified here. This method uses two different actors, so the total number of parameters may be higher than for a single actor. The aim of this research is not to reduce the total parameters. Rather, it is shown that by dividing the task into stages, a small 64–32 MLP is

sufficient for each stage and can be trained effectively.

This stage-based method also facilitates embedded deployment. Since each policy is small, they are converted to INT8 models using Quantization-Aware Training (QAT) and deployed on the ESP32 microcontroller. The system is then verified through Hardware-in-the-Loop (HIL) testing. Here, the physics simulation runs at 20 Hz on the host computer, while each control action is calculated by the real ESP32.

ESP32 was selected as the target hardware in this study due to its low cost, low power consumption, built-in communication facilities, and ease of use. Its limited SRAM and Flash memory make embedded deployment more challenging. Therefore, successfully running the model on ESP32 indicates potential for other low-cost embedded platforms as well. Although the Tensilica Xtensa processor of ESP32 is different from ARM Cortex-M, the main goal of this study is to present an effective method for implementing reinforcement learning on low-power hardware. This can further expand the scope of low-cost intelligent robots in education, small industries, and developing countries.

1.3 Research Question

This study seeks to answer the following three questions:

Research Question 1:

Does dividing the pick-and-place task into several smaller steps using a finite state machine (FSM) and using a different reinforcement learning (RL) policy for each step reduce training complexity and increase learning accuracy and reliability compared to a monolithic RL model?

Research Question 2:

Can trained stage-specific policies be compressed using int8 Quantization-Aware Training (QAT) without significant performance loss, so that they can be deployed on ESP32 microcontrollers?

Research Question 3:

Can quantized policies generate control actions fast enough for real-time control on real ESP32 hardware, and is it possible to verify its effectiveness through Hardware-in-the-Loop (HIL) testing before real implementation?

1.4 Objectives

The main objectives of this thesis are as follows:

1. To decompose the task into stage-specific policies into a compact network and compress the trained policies to INT8 via quantization-aware training for fitting inside limited SRAM and Flash memory.
2. To recover the object's pose via a memory efficient AprilTag detector and a learned residual corrector, replacing the simulator's ground truth.
3. To validate the deployed system on real ESP32 hardware using HIL simulation.

1.5 Key Contribution

- This study develops a stage-wise reinforcement learning (RL) framework for the pick-and-place task. Here, two small MLP (64-32) policies for the Grasp and Transport stages are trained using the TD3 algorithm. These policies are controlled by a deterministic finite state machine (FSM) coupled with scripted routines, so that each policy learns only one specific task.
- An effective training method is developed and validated to solve various problems encountered during training. It prevents reward exploitation using Potential-Based Reward Shaping, solves the sparse reward problem using Reverse Curriculum, and provides a fast termination of non-progressing episodes. The entire process is log-based and tested by changing one variable at a time, so that the results are reliable and reproducible.

- This study builds a complete pipeline that converts trained PyTorch policies into int8 models using Quantization-Aware Training (QAT) and deploys them to the ESP32 microcontroller. This reduces the size of each model by about four times and takes about 15 milliseconds to compute a control action.
- This study builds a Hardware-in-the-Loop (HIL) validation system, where a real ESP32 microcontroller computes each control action for a physics simulation over a reliable framed serial communication. This ensures that the compressed models are fast and efficient enough for real-time control on the low-cost ESP32 before use in a real-world environment.

1.6 Organization of the Report

This report is organized into nine chapters as follows:

- **Chapter 1** gives the introduction of the thesis, including the overview, background & motivation, research question, objectives , key contribution, and the organization of the report.
- **Chapter 2** presents the existing research. It reviews the literature on previous studies on reinforcement learning, model compression techniques, and embedded deployment in robotic manipulation. In addition, a comparative analysis is conducted to highlight the strengths and limitations of existing methods and the research gap that this study attempts to fill. In addition, the rationale for the methods chosen for this thesis is also explained.
- **Chapter 3** introduces the engineering design and methodology of the proposed system. It explains the methods and tools used, the design requirements, the overall system architecture, and the design process.
- **Chapter 4** focuses on the implementation of the proposed framework. It describes the reinforcement learning formulation, training procedure, model compression, component selection, optimization techniques, and the overall execution process.
- **Chapter 5** explains the experimental setup used to evaluate the proposed system. It includes the validation methodology, simulation environment, and the performance metrics used for assessment.

- **Chapter 6** presents the investigation and analysis of the experimental data, followed by the results and a discussion of the system's performance.
- **Chapter 7** examines the broader impact of the proposed work. It discusses its contributions to society, health, safety, legal and cultural issues, as well as its environmental impact and sustainability.
- **Chapter 8** outlines the management and financial aspects of the project, including the project schedule, budget, resource planning, and the contributions of both team members.
- **Chapter 9** concludes the thesis by summarizing the main findings, discussing the limitations of the proposed work, and suggesting possible directions for future research.

The references cited throughout this thesis are provided at the end of the report.

1.7 Summary

This chapter outlines the main content of the thesis and the basis of the research. At the beginning, the main idea of the research is explained, which aims to implement Deep Reinforcement Learning (DRL) based robotic manipulator control on a low-cost ESP32 microcontroller.

The background and motivation of the study show that a monolithic RL model has two main limitations. First, such a model is difficult to train and does not generalize well. Second, its size and computational requirements are not suitable for running on small microcontrollers.

Research questions have been formulated to address these issues. It explores whether stage-based task decomposition can make training easier and more reliable, whether the model can be adapted for embedded deployment using int8 Quantization-Aware Training (QAT), and whether its performance can be verified on real hardware through Hardware-in-the-Loop (HIL) testing. This chapter then briefly presents the research objectives, main contributions, and the overall structure of the thesis.

However, this study is not a completely new initiative; rather, it builds on the concepts, methods, and results of previous studies. Therefore, before presenting the proposed method,

it is necessary to review the existing research on reinforcement learning, hierarchical or stage-based task decomposition, model compression, and embedded deployment in robotic manipulation. For this reason, the relevant literature review is presented in the next chapter. In addition, the limitations of the existing research are analyzed and the research gap that this thesis aims to fill is highlighted.

Chapter 2

Literature Review

2.1 Introduction

There are two main challenges in using reinforcement learning (RL) in robot manipulation. The first challenge is training. If a long and complex task such as pick-and-place is taught with a single RL policy, the learning process becomes difficult. As a result, training is slow, often does not converge correctly, or may fail completely [14], [15].

The second challenge is embedded deployment. Many studies have shown that the trained RL model is so large that it is difficult to store it in the memory of a microcontroller [3], [4]. However, this problem is mainly seen in the case of image-based policies. On the other hand, simple RL policies using low-dimensional states are relatively small and can be easily stored in the flash memory of the ESP32. In practice, the major challenges in embedded deployment are control loop latency, deterministic timing, and memory management (tensor arena fragmentation) when running multiple models in limited SRAM.

Staged Reinforcement Learning is an effective approach to address these issues. It divides a large and complex task into several smaller subtasks and trains a separate policy for each subtask. This simplifies training and allows each policy to focus on learning a specific skill.

This chapter compares different studies from four perspectives: problem complexity, computational cost of the algorithm, hardware resource requirements, and applicability to embedded deployment. In addition, special emphasis is placed on two important issues: selecting the right handover state between sub-policies and maintaining model performance after quantization.

2.2 Existing Research

2.2.1 Options Framework and Sub-MDP Decomposition:

The theoretical underpinning for the staged reinforcement learning is the Options Framework [9]. In this model, a long task is broken into "options" each of which is a temporally extended action that has its own policy and own termination condition. The architecture employed in this thesis is an example of this framework, with hand defined options: the intra-option policies are trained using RL and the termination conditions are hard coded. Moreover, a deterministic finite state machine (FSM) [16, [17] is used instead of the "policy-over-options" (high-level decision maker). This replacement is a conscious simplification for MCU deployment and there are two reasons for that. First, meta-level learning is not needed in this case as the sequence of the sub tasks of a pick and place operation is known. Secondly, a learned high-level controller is not able to guarantee determinism, verifiability and very low memory cost, which a deterministic FSM can provide, which is undesirable on an MCU.

2.2.2 Skill Chain Construction and Meta-Controller Design:

The literature on skill chaining demonstrates the interconnection of different skills in a sequence, where the end state of one skill is the beginning state of the next skill [18] and [19]. There are two design options for the meta-controller controlling this chain: a learned controller or a rule-based controller. In the case of MCU deployment scenarios, rule-based FSM meta-controllers perform better than learned meta-controllers [20, 21]. Their deterministic behavior eliminates the extra inference overhead of a learned module, and studies show that they do not significantly reduce the end-to-end success rate, as long as the handover state sampling between the skills is sufficient.

2.2.3 Handover State Reward Distribution:

Shift The present work directly takes the mitigation from the literature. The reset of the transport environment will not be based on an idealized grasp state. Instead, the frozen grasp policy is performed at reset, and the resulting terminal state is made the initial state of the transport policy. There is a further filter: A scripted test-lift is called and the handover is accepted only when the lift is made and the grip is maintained, otherwise the test lift is repeated, up to a maximum of eight attempts. The median training logs indicated about 2.5-3

trials per episode. Based on this, it was concluded that a good set of target grasps had already been generated by the grasp policy and only a subset of this was needed to be selected, so no retraining of the grasp model was performed.

2.2.4 Training Techniques and Reward Design for Staged Policies :

The literature reports three complementary elements for training stage-specific sub-policies: stage-specific sparse reward functions [23], pruned state spaces [8] and demonstration-based warm-start initialization [14, 19, 23]. An improvement of sample efficiency by an order of magnitude is claimed for their combination [24]. None of these three is used in the present work and the reasons should be mentioned. Since no demonstrations were available, a warm-start was not possible. The observation layout wasn't pruned per stage, because a single 46 dimensional observation layout makes the firmware simple, and the memory cost of the extra inputs is quite low. No sparse stage rewards were used for grasp stage. In this last regard, there is a comment that is contrary to the literature. The reward was dense at the grasp stage and with no special stabilization, about 95 percent was reached without any curriculum. The reward, however, is rather trivial in the transport stage and a potential based shaping term [Ng et al., 1999] and a reverse curriculum were needed. The lesson to be learned here is not that dense rewards are for free space and sparse rewards are for contact. The lesson is that a dense well-supported reward trains directly, and a rare terminal reward requires shaping, curriculum, and termination design.

2.2.5 Model Compression and INT8 Quantization for MCU Deployment:

Although the task decomposition results in small sub-policies, the models have to be compressed and fit and run on an MCU. The accuracy of the neural network used in the RL policy is the most important factor to get a super-light model for both free space motion phases (reach, transport) and contact-rich phases (grasp, place). The models are already small, so the quantization needs to be very high quality to ensure the policy performs well; there is not much redundancy left to absorb quantization error. This is why quantization-aware training (QAT) is crucial for this purpose rather than just post-training quantization [12] and [13]. The literature on INT8 quantization also reveals that the architecture of the network has a significant impact on the effectiveness of quantization. Compared to convolutional networks,

the fully connected MLP architecture used in the RL policy networks is more robust to post-training quantization (PTQ) due to its more uniform weight distribution and its narrower range of activation values [13]. On the output side, however, there is a reverse effect: The continuous action output of a control policy is less tolerant of quantization error than the output of a classification task, since a small numerical error translates into a physical control error [12]. This is because it is also a combination of high weights and high sensitivity outputs, which is why even for the simplest MLP policies, careful quantization with QAT fine-tuning is still required.

2.2.6 TFLite Micro Inference Constraints and Optimization:

TensorFlow Lite Micro (TFLite Micro) is the default inference engine for deploying quantized models to microcontrollers [3, 26]. It is reported that INT8 inference on TFLite Micro for the ESP32 can achieve the RL policy inference latency of less than 15ms, which is well within the 50ms control cycle requirement. But the most important engineering challenge is not speed, but memory: TFLite Micro manages memory for tensors in a fragmented way, and this fragmentation is the engineering challenge that impacts the reliability of multi-model switching deployment, where multiple sub-policy models share the same limited SRAM.

2.2.7 Hardware-in-the-Loop Verification and Embedded System Integration:

The common approach to the testing of embedded controllers prior to real deployment is to use the technique of hardware-in-the-loop (HIL) validation [27, 28]. For staged RL, HIL validation is a closed loop, in which states are sent to the MCU by the simulation, actions are computed by the MCU with the deployed policies, and then those actions are sent back to the simulation. The methodology of HIL is in the measurement. It measures the impact of the combined effect of MCU inference delay, communication jitter and quantization error on the manipulation success rate before deployment, rather than using idealized inference assumptions within the simulation. The communication path between the simulation computer and the MCU should be sufficiently quick to not affect the measurement of timing. A serial communication protocol which uses a synchronous frame mode has a communication latency of less than 5ms per step at 921600 baud. The total step time for HIL testing is largely dependent on the MCU inference time, and not on the communication overhead, so the HIL

test results closely approximate the on-device results. If multiple stage specific models need to run on one MCU, then the architecture of the firmware is also crucial. The ESP32 has a dynamic loading strategy from Flash to RAM: The models are stored in the bigger flash memory and only the active model is loaded into the RAM [3], [4]. This technique allows switching and storing 5 stages of models without any additional SRAM requirement. The switching delay of the FSM is under 10 ms, which is considerably less than the control cycle of 50 ms, so that the switching of the stage policies does not impact the real-time control.

2.5 Comparative Analysis

Table 2.1 presents a comparative analysis of the key studies included in the literature review. The studies are arranged in chronological order of publication and the relationship of each study to this thesis is highlighted.

Table 2.1: Chronological comparison of related works and their relevance to this thesis.

Authors	Principle & Process	Year	Journal	Core Relevance to Thesis
Levine S., Finn C., Darrell T., & Abbeel P. [2]	End-to-End Visuomotor Learning: Uses Guided Policy Search (GPS) to map raw pixel inputs directly to low-level motor torques.	2016	JMLR	Foundation for end-to-end perception and control. Demonstrates joint optimization of vision and control pipelines.
Jacob B., Kligys S. et al. [12]	QAT: Simulates INT8 quantization during training to preserve model accuracy.	2018	CVPR	Reference for QAT. Supports accurate INT8 deployment of compact models.

Fujimoto S., van Hoof H., Meger D. [5]	TD3: Introduces twin critics, delayed policy updates, and target policy smoothing.	2018	ICML	Foundation of the thesis. Provides the TD3 algorithm used for all stage policies.
Kurrek P., Zoghلامي F. et al. [45]	Q-Model: RL development methodology using subtask rewards and HRL execution.	2020	J. Comput. Inf. Sci. Eng.	Development methodology. Supports structured HRL-based robotic system design.
Capotondi A., Rusci M. et al. [42]	CMix-NN: Mixed-precision neural network library supporting 8-, 4-, and 2-bit inference.	2020	IEEE TCAS-II	Extends quantization. Suggests future lower-bit deployment for smaller models.
Krishnan S., Boroujerdian B. et al. [27]	Air Learning: Hardware-in-the-loop framework with embedded inference and simulated environment.	2021	Machine Learning	Supports HIL validation. Matches the thesis ESP32-Robosuite testing setup.
Novac P.-E., Boukli Hacene G. et al. [40]	MCU Quantization: End-to-end INT8 deployment pipeline for Cortex-M devices.	2021	Sensors	MCU deployment baseline. Supports the thesis ESP32 quantization workflow.
Agarwala, A., Das, A., Juba, B., Panigrahy,	Proves that avoiding cross-task interference inside a monolithic network	2021	ICLR	Justifies the use of task decomposition for embedded deployment to

R., Sharan, V., Wang, X., & Zhang, Q. [48]	requires hidden-layer widths to scale significantly faster than the sum of separate sub-networks.			facilitate small combined network footprint.
Marzari, L., Pore, A., Dall'Alba, D., Aragon-Camarasa, G., Farinelli, A., & Fiorini, P. [50]	Demonstrate that end-to-end monolithic models require massive network capacity to balance opposing dynamics, while decomposing tasks into smaller sized networks solve them efficiently.	2021	ICAR	Directly supports the approach of task decomposition in favor of efficiency and compactness that is required for deployment on ultra cheap MCUs.
Muratore F., Ramos F. et al. [29]	Domain Randomization: Reviews randomization techniques for robust policy transfer.	2022	Frontiers in Robotics and AI	Supports randomized training. Explains robustness gained from domain randomization.
Ju H., Juan R. et al. [39]	Sim-to-Real Survey: Reviews domain randomization, adaptation, and transfer methods.	2022	Nature Machine Intelligence	Transfer foundation. Supports the thesis sim-to-real testing methodology.

Heo Y., Lee S. [44]	GAN Distillation: Uses lightweight GAN generators for teacher-student knowledge transfer.	2022	Electronics	Supports lightweight models. Useful for ESP32 memory-constrained deployment.
Chu Y., Li P. et al. [38]	Pruning + Distillation: Combines channel pruning with attention-based knowledge distillation.	2022	Applied Intelligence	Alternative compression. Demonstrates combined pruning and distillation for compact models.
Kang M., Oh S. [18]	Latent Skill Chaining: Links independently trained skills using bridge policies and transition classifiers.	2022	Intelligent Service Robotics	Supports bridge policies. Shows independently trained skills can be successfully connected.
Wang J., Zhang W. et al. [43]	Knowledge Distillation: Transfers key features from teacher to compact student networks.	2023	Computer Vision and Image Understanding	Alternative compression. Supports small student models for edge deployment.
Han D., Mulyana B. et al. [7]	DRL Survey: Reviews actor-critic methods and HRL for robotic manipulation.	2023	Sensors	Background reference. Justifies using TD3 and hierarchical RL.

Liu X., Wang G. et al. [8]	HRL + Human Knowledge: Uses human priors to divide manipulation into subtasks with rule-based transitions and PGPE optimization.	2023	IEEE T-ASE	Direct precedent for the thesis. Validates staged task decomposition, FSM transitions, and sim-to- real transfer.
Pfeiffer, J., Ruder, S., Vulić, I., & Ponti, E. M. [49]	Demonstrates that end-to-end task networks require massive over- parameterization to prevent gradient interference, whereas modular routing avoids it.	2023	PMLR	Supports the notion that single network for end- to-end tasks require a much higher parameter count than modular ones.
Badalian K., Koch L. et al. [28]	LExCI: Framework for training and deploying RL agents on embedded systems.	2024	Applied Intelligence	Closest deployment work. Supports RL deployment on embedded hardware.
Hoeller D., Rudin N. et al. [21]	Hierarchical Skills: Trains reusable low-level skills selected by a high- level controller with sim-to-real transfer.	2024	Science Robotics	Strong HRL precedent. Demonstrates hierarchical skill selection similar to the thesis FSM.
Zhang N., Zhao Y. et al. [23]	HRL + Demonstration: Generates object-	2025	Arabian Journal for Science and Engineering	Supports staged task decomposition. Demonstrates effective

	centered sub-goals from demonstrations for hierarchical policy learning.			sub-goal learning for long-horizon manipulation.
Alves J.B., Lau N., Silva F. [19]	Curriculum Skill Chaining: Learns subtasks from a single demonstration and curriculum-guided transition policies.	2025	Robotics and Autonomous Systems	Supports handoff learning. Shows one demonstration can effectively segment robotic skills.
Xin L., Tian H., Sheng B. [22]	HAFER: Combines adaptive rewards with Failure Experience Replay (FER) to preserve rare transition failures.	2026	Neurocomputing	Validates handoff-state sampling. FER supports learning from rare boundary failures.
Mandlekar A., Xu D., Wong J., Nasiriany S., Wang C., Kulkarni R., Fei-Fei L., Savarese S., Zhu Y., & Martín-Martín R. [47]	Empirical study (robomimic): benchmarks offline-learning algorithms (BC, BC-RNN, CQL, IRIS) to isolate what actually drives manipulation success.	2022	CoRL(PMLR)	Uses robosuite, the same framework this thesis is done on. It allows comparison to Its BC-RNN model size is the deployment-size baseline in Table 6.4.

A few important points are clear from the comparative analysis in Table 2.1. Basic technologies such as TD3 [5] and QAT [12] are already well established. Similarly, the concepts of hierarchical or stage-based task decomposition have also been successfully presented in various studies [8], [21], [23]. In addition, the quantization pipeline required to run INT8 models on microcontrollers has also been studied [40], [41]. However, none of the reviewed studies have presented stage-based training, INT8 model compression, and closed-loop embedded validation on a typical microcontroller in a complete system. Three major research gaps can be identified from this analysis, and the present thesis attempts to fill these gaps.

First, there is a validation gap. Previous studies have proposed embedded RL frameworks [27], [28] and have used RL on microcontrollers for applications such as flight control or mobile robots. However, there is no example of implementing the INT8 quantized stage-based manipulation policy on a general microcontroller and evaluating it in a closed-loop physics simulation. As a result, there is insufficient information in the existing studies on how inference latency, communication jitter, and quantization effects together affect system performance.

Second, there is a reporting gap. Many studies do not disclose in detail the practical problems that arise during training. For example, issues such as poorly designed reward functions, sparse reward problems, replay buffer limitations, or reward exploitation are usually not mentioned in the final research paper. As a result, new researchers do not get enough guidance on the causes, solutions, and practical experiences of these problems in the training process.

Third, there is an attribution gap. In studies that have used learned policies and scripted routines together, the separate contributions of the two parts have not been clearly analyzed. In particular, there are very few examples of ablation studies that keep the same neural network and change only the rule-based part.

In addition, several topics were not included in this study and are left for future research. These include stage-based adaptive domain randomization [29],[31], joint optimization of

policy gradient and quantization-aware training (QAT), and the development of a standardized benchmark for comparing stage-based RL systems on microcontrollers. These are discussed as future work in Section 9.

2.4 Summary

This chapter reviewed the previous studies that provide the foundation for this research. It covered the key topics related to hierarchical reinforcement learning, reward design, model quantization, TensorFlow Lite Micro, and hardware-in-the-loop validation. A review of the published research between 2018 and 2026 revealed that while these techniques have been explored separately, there was no existing research that merged staged reinforcement learning, INT8 model compression, and embedded hardware validation to a single framework. This study seeks to fill that void.

The following chapter, Engineering Design and Methodology, discusses the design and implementation of the proposed system, which involves reinforcement learning, system architecture, quantization process, and hardware-in-the-loop testing.

Chapter 3

Engineering Design and Methodology

3.1 Introduction

The previous chapter surveyed the literature, analyzed the related literature and established the gap in the literature that this thesis will fill. From these results, the engineering design and methodology of the proposed system is presented in this chapter.

In section 3.2 the methods and tools to be used throughout this research are introduced. The system requirements and constraints are discussed in Section 3.3 and the design requirements that guided the development of proposed framework are discussed. The model design and performance criteria are given in Section 3.4. The overall design process is described in Section 3.5, which covers conceptual design and training algorithm. The proposed system model and its complete architecture is discussed in section 3.6.

In summary, this chapter provides the engineering base of the proposed system, covering the system design, requirements, methodology, and architecture.

3.2 Methods and Tools

This section describes the main methods, software, and tools used in this thesis. For each item, a short explanation is given first, followed by its role in this work.

Reinforcement Learning (RL)

Reinforcement learning is a field of machine learning. In reinforcement learning, an agent learns to maximize a reward in a given situation by taking the right actions. It is different from supervised learning: in supervised learning, the training data comes with the correct answer, and the model is trained with that answer. In reinforcement learning, there is no answer key. The agent must decide by itself what to do to perform the given task, and it learns only from its own experience of trial and error. In this thesis, reinforcement learning is used to make a robotic arm learn the pick-and-place task by itself, without any hand-written control rules.

Deep Reinforcement Learning (DRL)

Deep reinforcement learning combines reinforcement learning with deep neural networks. Instead of a table of values, a neural network is used to represent the policy (the mapping from what the robot observes to what the robot should do). This allows the agent to work with continuous states and continuous actions, which is necessary for robot control. In this thesis, small Multi-Layer Perceptron (MLP) networks with 64 and 32 hidden units are used as the policies, which keeps the models small enough for microcontroller deployment.

Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 is a modern deep reinforcement learning algorithm designed for continuous control problems such as robot arm movement [5], [6]. It uses an actor network, which decides the action, and two critic networks, which estimate how good the action is. TD3 improves training stability with three tricks: it uses twin critics and takes the smaller of their two estimates to avoid over-optimistic values, it updates the actor less frequently than the critics (delayed updates), and it adds small noise to the target actions (target policy smoothing). In this thesis, TD3 is used to train both the grasp policy and the transport policy. Only the actor networks are deployed on the microcontroller; the critic networks are used during training only.

Prioritized Experience Replay (PER)

During training, the agent stores its past experiences (state, action, reward, next state) in a memory called a replay buffer, and learns by sampling from this memory. Normal replay samples the experiences uniformly. Prioritized experience replay instead samples the experiences from which the agent can learn the most (those with the largest learning error) more often [34]. This makes training more sample-efficient. In this thesis, PER with a SumTree data structure is used together with TD3, with a replay buffer of 200,000 transitions.

Finite State Machine (FSM)

A finite state machine is a simple and classical control structure. The system can be in exactly one state at a time, and it moves from one state to another when a defined condition is satisfied. An FSM is fully deterministic: the same situation always produces the same behavior, which

makes it easy to verify and very cheap in memory. In this thesis, an FSM works as the supervisor of the whole pick-and-place task. It switches between the stages (grasp, test-lift, transport, release) according to predefined transition conditions, so each small learned policy only has to do its own job.

Python

Python is a high-level, easy-to-read programming language and the standard language of machine learning research, because almost all major machine learning libraries are built for it. In this thesis, Python is used for the entire training side of the work: the training scripts, the environment wrappers, the reward functions, the logging and diagnosis tools, and the PC side of the hardware-in-the-loop test system are all written in Python.

PyTorch

PyTorch is an open-source deep learning framework. It provides automatic differentiation, GPU/CPU tensor computation, and building blocks for neural networks, and it is one of the most widely used frameworks in machine learning research. In this thesis, PyTorch is used to build and train the actor and critic networks of the TD3 algorithm, and also for the quantization-aware fine-tuning of the trained policies.

robosuite and MuJoCo

MuJoCo (Multi-Joint dynamics with Contact) is a fast and accurate physics engine that is widely used for robot simulation. robosuite is an open-source robot learning framework built on top of MuJoCo [10], [11]. It provides ready-made robot models and manipulation tasks with realistic physics. In this thesis, the robosuite PickPlace environment with a Franka Emika Panda 7-DOF robot arm is used as the training and testing environment. The robot is controlled with the OSC_POSE controller at 20 Hz, and the observation is a 46-dimensional state vector, so no camera or image processing is needed.

Model Quantization and Quantization-Aware Training (QAT)

A trained neural network normally stores its numbers as 32-bit floating-point values. Quantization converts these numbers into 8-bit integers (INT8), which makes the model about four times smaller and much faster on small processors [12]. However, quantization introduces

small numerical errors. Quantization-aware training solves this: the model is fine-tuned while simulating the quantization effect (fake quantization), so the weights adjust themselves to survive the conversion with minimal accuracy loss. In this thesis, both actor networks are QAT fine-tuned and converted to full-integer INT8, which is essential because the models are already very small and cannot afford large quantization errors.

LiteRT / ai_edge_torch Conversion Toolchain

LiteRT (formerly TensorFlow Lite) is a format and runtime for running machine learning models on small devices, and ai_edge_torch is the toolchain that converts PyTorch models into this format. In this thesis, the QAT fine-tuned PyTorch actors are converted with this toolchain into full-integer INT8 .tflite model files, and each model file is then embedded as a C array header so it can be compiled directly into the microcontroller firmware.

TensorFlow Lite Micro (TFLite Micro)

TensorFlow Lite Micro is an inference engine designed to run machine learning models on microcontrollers [3]. It does not need an operating system or dynamic memory allocation; it runs the model inside a fixed memory area called the tensor arena. In this thesis, TFLite Micro (through the ArduTFLite library) runs the INT8 actor models on the ESP32, computing one control action in about 15 milliseconds, which fits comfortably inside the 50-millisecond control cycle.

Arduino IDE and C++

The Arduino IDE is a free and beginner-friendly development environment for programming microcontrollers, and C++ is the programming language used in it. In this thesis, the complete ESP32 firmware is written in C++ using the Arduino IDE. The firmware receives the state frames over the serial port, runs the INT8 model with TFLite Micro, and sends the computed action back to the PC.

ESP32 Microcontroller

The ESP32 is a low-cost, low-power microcontroller with a dual-core processor, limited SRAM, and limited Flash memory. It costs only a few dollars and is widely available, which makes it a popular choice for embedded and IoT projects. In this thesis, the ESP32 DevKit

board is the deployment target of the whole work: the goal of the thesis is to prove that reinforcement learning policies can run in real time on this class of hardware.

Serial Communication (UART)

Serial communication (UART) is a simple and reliable way to exchange data between a computer and a microcontroller over a USB cable, one byte at a time, at a chosen speed (baud rate). In this thesis, a custom framed serial protocol was designed on top of UART at 921600 baud. Each message carries a sync pattern, a sequence number, the data payload, and a checksum, so the link can detect errors and re-synchronize itself. The states are sent to the ESP32 in quantized INT8 form, which also reduces the message size by about 3.5 times compared to float values.

Hardware-in-the-Loop (HIL) Testing

Hardware-in-the-loop testing is a validation method where the real target hardware is placed inside the control loop of a simulation. The physics simulation is performed on the PC, and all the control actions are calculated by the real device. In this thesis the 46-dimensional state is sent from the PC to the ESP32, the ESP32 performs the INT8 policy and sends the 7-dimensional action back to the PC, and the simulation performs the action. This is a proof that the quantized models are correct for real integer calculations, real memory size, and real communication delays, before deployment on a physical robot.

3.3 System Requirements and Constraints

In general, modern learning based manipulation controllers depend on a high capacity network, often including convolutional or recurrent layers, which are trained and deployed on memory-rich hardware with high floating-point performance. This is not suitable for the type of devices that dominates embedded and edge robotics where cost, power and physical size dictates that the controller can be no larger than a microcontroller with a few kilobytes of usable RAM. The problem of the design in this thesis is thus:

to structure the learning, the network architecture, and the deployment pipeline such that a reliable closed-loop pick-and-place controller can run entirely on a resource-constrained MCU, in real time, without sacrificing task success.

There are three sub-problems here that are coupled to the umbrella problem. The first one is a representation problem: the observation and the network must be small enough to be on-device, which means we cannot use image inputs and their associated large feature extractors. Second, a learning problem: a single small network trained end-to-end on the whole task will have to deal with the sparseness of the success signals and internally conflicting goals, which will make reliable convergence difficult as will be demonstrated later. Third, a deployment problem: the trained policy needs to be quantized, embedded, and deployed with a fixed realtime control budget, and the resultant behaviour must be validated on the actual silicon and not assumed based on simulation.

The task itself is instantiated in the robosuite PickPlace environment (MuJoCo physics) [10], [11], with an arm of Franka Panda with 7 degrees of freedom (7DOF) (`franka_panda`), a single bread object (`single_object_mode=2`), and an operational space pose controller (`OSC_POSE`) at 20 Hz. The action is a 7-dimensional vector `[dx, dy, dz, droll, dpitch, dyaw, gripper]` and the observation is a 46-dimensional Full-state vector (robot proprioception and object pose). The full state is observable and hence the control problem is in principle a Markov decision process (MDP), which the design is engineered to make use of plain feed-forward MLPs.

3.4 Design Requirements

The requirements below are hard constraint that a candidate design must meet to be deployable on the target MCU under real-time continuous control. Together, they determine the design space explored in the remainder of the chapter.

3.3.1 Model Size Requirements

The deployed policy needs to be compact enough to fit in the ESP32's on-chip RAM, available along with the TensorFlow Lite for Microcontrollers (TFLite-Micro) runtime and the tensor arena. This constraint results in three design commitments:

- **Compact fully-connected topology.** These learned actors are just a few kilobytes in size and consist of a $46 \times 64 \times 32 \times 7$ MLP with ReLU hidden activations and tanh output. No CNN, RNN or attention is applied.
- **Vector-only observation.** The 46-D full-state observation eliminates the need for cameras and CNN feature extractors, the biggest factor in making the model small, due to the MDP property of the state representation.
- **INT8 weights and activations.** The final actors are converted to full-integer INT8, which results in about 4x smaller model size than FP32 and enables the model and a smaller tensor arena to fit into on-chip RAM.

These limits only apply to the actor networks. The twin critics used in the training of TD3 are not artifacts that are used for training only and do not need to be large.

3.3.2 Inference Performance Requirements

One forward pass of the actor on the MCU is needed for each closed-loop control step. The inference latency of that pass should be small compared to the control period, otherwise the sensing, communication and actuation should also be budgeted. Therefore, integer inference is not just a memory optimization, it's a latency requirement: INT8 execution of the actor is measured/expected to be ~15ms on the ESP32, while an equivalent FP32 pass is ~30ms. The timing margin that the continuous-control loop requires is created by halving the inference latency.

3.3.3 Inference Frequency Requirements for Continuous Control

Closed-loop: Continuous control - controller must generate a new action from the current observation at each control step, and not on an ad hoc basis. The OSC_POSE controller operates at 20 Hz, thus establishing a 50ms control period. During this time, the system needs to learn the 46-D observation, send it to the MCU, perform INT8 inference and return the 7-D action and step the actuation. Inference takes ~15 ms; the rest of the pipeline (~35 ms) can be used for serial transport and host-side processing, with real-time margin. If the per-step latency of any design is longer than the control period, it would cause a closed loop failure and is not feasible.

Table 3.1: Design requirements and the corresponding design responses.

Requirement	Constraint	Design response
Model size	Fit actor + TFLite-Micro arena in on-chip RAM (few KB budget)	64-32 MLP, 46-D vector observation, INT8 quantization ($\sim 4\times$ smaller)
Inference latency	One forward pass $\ll$ control period	Full-integer INT8 inference (~ 15 ms vs ~ 30 ms FP32)
Inference frequency	Fresh action every step at 20 Hz (50 ms period)	Per-step closed-loop pipeline with ~ 35 ms margin for I/O

3.4.4 On-Device Perception Requirements

The Section 3.3.1–3.3.3 budgets do not become more lax when perception is added to the target of deployment. The detector and its corrector have to be stored in the same on-chip RAM as the two actor tensor arenas, and as the ESP32 in this work reports no PSRAM, all the controller RAM is on those two arenas. It is possible because of two characteristics of the task. First, the pose of an object only needs to be computed once, when the arm is in "home pose" and the tag is located where it is expected to be, after the grasp, the pose is recomposed from the latched grasp offset and the forward kinematics of the arm, so the detector does not have to operate at 20 Hz, the control rate. Second, a plain ESP32 actually limits: its memory footprint, not its accuracy; resolution, decimation and a region of interest can be configured to suit the measured heap space without affecting the ability to detect the tag, which is determined solely by whether the tag is visible from the wrist at the home pose or not, not how many pixels it occupies after detection.

Table 3.2: Perception requirement and their constraints.

Requirement	Constraint	Design response
Model size	Fit actor + TFLite-Micro arena in on-chip RAM (few KB budget)	64-32 MLP, 46-D vector observation, INT8 quantization ($\sim 4\times$ smaller)
Inference latency	One forward pass $\ll$ control period	Full-integer INT8 inference (~ 15 ms vs ~ 30 ms FP32)
Inference frequency	Fresh action every step at 20 Hz (50 ms period)	Per-step closed-loop pipeline with ~ 35 ms margin for I/O

Requirement	Constraint	Design response
Perception footprint	Detector + corrector must coexist with both actor tensor arenas inside the ESP32's usable heap (~210 KB, <code>MALLOC_CAP_8BIT</code>), no PSRAM	Memory-reduced AprilTag C port (8→4 B/px union-find, cropped ROI, tuned cluster-map knobs) + a 6.1 KB FP32 residual corrector, run once per episode rather than every control step

3.5 Model Design and Performance Criteria

A candidate design is assessed in terms of the following criteria in the light of the requirements of Section 3.4 Design Requirements. These criteria are used to inform all design decisions in Sections 3.5 and 3.6, and are realised as performance indices in Section **Error! Reference source not found.**

- 1. Task success rate.** Fraction of episodes in which the object is placed in the target bin (deterministic [noise-free] evaluation). This is the first and foremost criterion: if a small and fast model cannot do the job reliably, then it is worthless.
- 2. Robustness.** Lack of sensitivity to the physical disturbances that occur during manipulation, such as grip slippage under inertial load, marginal grasps and terminal placement error. Robustness is sought in the learning (reward and curriculum design) and in the deterministic rule layer.
- 3. Determinism and verifiability.** The sequencing logic must be reproducible and inspectable since the controller is running unattended on an MCU. This is to the benefit of a deterministic FSM meta-controller and scripted micro-skills over learned meta-level policies where the task order is known beforehand.
- 4. Memory footprint.** The on-device model + runtime must be within the RAM budget, which is measured by the size of the model in kilobytes (after INT8 conversion).
- 5. Latency margin.** The per-step inference time must be within the 50ms control period, measured on-device and end-to-end cycle time.
- 6. Deployability and fidelity.** Behavior must be preserved during quantization and verified on real hardware, not just in simulation, and measured by HIL end-to-end success and cycle-time statistics.

A common theme is that these criteria are not mutually independent and that some criteria are mutually exclusive. The design contribution of this thesis is an architecture that resolves the most harmful of these tensions, not by a scalar compromise, but by structural decomposition, as developed next.

3.6 Design Steps

This section presents the reasoning that takes the requirements and criteria above to the proposed system model. It is split into the design of an integrated system (3.6.1 Conceptual System Design Steps) and the design of the training algorithm (3.6.2 Training Algorithm Design Steps).

3.6.1 Conceptual System Design Steps

Conceptual Design. The system consists of four components, as follows: (1) Manipulator which includes Panda arm and gripper; (2) Environment which includes an object and the bin; (3) DRL control model that includes learned sub-policies; and (4) ESP32 MCU of the real-time inference. This collectively closes a perception–action loop, where duties are split between learned and scripted control.

Step 1 : Control Loop. The environment sends a 46-D observation to the MCU and using the DRL model, it computes a 7-D action. The OSC_POSE controller carries out the action, updates the environment and returns to us the next observation. Hence MCU is now the Real time controller.

Step 2 : Task Decomposition. In the pick-and-place task, we split it into two stages: grasp and place. By using a separate learned policy with its own reward for each stage, the complexity of learning is reduced as conflicting objectives between different stages are avoided.

Step 3 : FSM Coordination. A FSM determines the sequence of learned policies and fires scripted transitions. As the task order is fixed, an FSM serves as a simple yet effective route to guarantee without inference penalties.

Table 3.3: The finite state machine meta-controller: learned sub-policies (states 1, 3) interleaved with scripted micro-skills (states 2, 4).

No.	FSM state	Controller	Exit condition
1	GRASP	Learned grasp policy (64→32)	8 consecutive grasp checks pass
2	TEST-LIFT	Scripted +z probe	Object rises ≥ 3 cm and stays gripped; else retry GRASP (≤ 8 attempts)
3	TRANSPORT	Learned transport policy (64→32)	Object held within 0.14 m of bin center for 3 consecutive steps
4	RELEASE	Scripted P-controller (recenter → lower → open → retract)	Placement success check → DONE

Step 4 : Scripted Micro-skills. P-controlled routines execute the lift test and release procedure. These deterministic behaviours are computationally inexpensive and were more reliable than learned approaches, particularly for releasing without tactile sensing.

Theoretical Framing. Through accommodating the options framework, our architecture formulates learned sub-policies using fixed termination conditions and a deterministic FSM with no learned meta-controller. Hierarchical reinforcement learning and sequential controller composition also falls into this regime, with a focus on simplicity, robustness and low memory consumption for MCU deployment.

3.6.2 Training Algorithm Design Steps

Learning Methodology. In this section, we outline the training process of two deployed policies as well as the INT8 quantization procedure for MCU deployment. It also notes the original monolithic strategy that inspired the staged design.

Step 1 - Base RL Algorithm: Both sub-policies are trained based on TD3 [6], [5], [33], [34] with Prioritized Experience Replay (PER) [35]. We show that twin critics, delayed policy updates and target policy smoothing stabilize training. Here actor and critic networks use 64-32, a discount $\gamma = 0.99$, a replay buffer of 200,000 transitions, actor and critic learning rates of

3×10^{-4} , batch size 512, and target-network smoothing $\tau = 0.005$ hidden layers but we only deploy the actor networks on the MCU.

Step - 2 Stage-wise Training: The problem divides into two independently trained policies:

- **Grasp Policy:** Trained with a dense reward positively biasing reachability, grapsibility and stability during lifting, achieving ~95% grasp success.
- **Transport policy :** it trains on states well understood and learns to place object given sparse rewards, potential-based reward shape scheme, reverse curriculum. Potential-based shaping mitigates the risk of exploitation while still making progress towards the goal.

A key shaping decision at this step is the use of **potential-based reward shaping** [25] of the form

$$r = \gamma\Phi(s') - \Phi(s),$$

which is policy-invariant and therefore cannot be “farmed” by lingering. The potential Φ is a bounded cumulative-milestone function over the object pose (lift $\rightarrow$ approach $\rightarrow$ place) with a per-episode ratchet on lift credit so that lowering the object into the bin cannot erase progress. This design directly removes a hovering exploit that per-step state rewards had induced.

Step 3 - Monolithic Baseline: Sometimes we first trained a single policy for the entire task, but it was unsuccessful because grasp, transport and release are conflicting goals. Conflicts with these rewards could not be resolved by extensive reward tuning, motivating a staged architecture.

Step 4 - Benefits of Staged RL:

1. It eliminates the objective conflict by decoupling transport and release. The transport policy holds the gripper closed and releases it in a scripted routine, this makes holding things still the success condition.
2. It maintains a small and task-specific network for each instance. Two 64 $\rightarrow$ 32 policies are more stable to train, easier to fit into memory constraints of microcontrollers.
3. It exact release to a deterministic P-controller and offers full control without added cost of learning or inference.
4. It offers a way to sequence policies deterministically via an FSM, incurs zero trainable parameters, and fulfills memory/reliability constraints.

Step 5 - INT8 Quantization :

Two actors deployed (46*64*32*7) which are then quantized using QAT (Quantization Aware Training), and quantized to full-integer INT8 TensorFlow Lite models.

(ii) The FP32 actors are carefully tuned with fake quantization calibrating via replay-buffer states.

(ii) QAT models were converted to fully integer INT8. Convert to tflite with very small drop in accuracy!

(iii) The. Embedded as C-array header files for esp32 from tflite models In this design, only the actor networks are quantized and deployed while the critics remain on the host.

3.7 Proposed System Model

Through the conceptual design steps of Section 3.6.1 Conceptual System Design Steps, the system models presented here were obtained. There are two parts : 1. The preliminary monolithic model that was first attempted (Section 3.7.1 Preliminary Monolithic Model), and 2. The detailed final staged model that was adopted (Section 3.7.2 Our Model).

3.7.1 Preliminary Monolithic Model

For the preliminary model, pick-and-place was a single, monolithic learning problem: one TD3 policy covers the full 46-D state and also outputs the full 7-D action ; translation, rotation, and gripper i.e., the entire task, from reaching through grasping, transport, and release. There is no explicit stage structure and no scripted micro-skill; the network is expected to learn the whole behavior. This model seems architecturally easier, one network, one reward, one training loop but it proved unable to meet the criterion of Section

3.4.4 On-Device Perception Requirements

The Section 3.3.1–3.3.3 budgets do not become more lax when perception is added to the target of deployment. The detector and its corrector have to be stored in the same on-chip RAM as the two actor tensor arenas, and as the ESP32 in this work reports no PSRAM, all the controller RAM is on those two arenas. It is possible because of two characteristics of the task. First, the pose of an object only needs to be computed once, when the arm is in "home pose" and the tag is located where it is expected to be, after the grasp, the pose is recomposed

from the latched grasp offset and the forward kinematics of the arm, so the detector does not have to operate at 20 Hz, the control rate. Second, a plain ESP32 actually limits: its memory footprint, not its accuracy; resolution, decimation and a region of interest can be configured to suit the measured heap space without affecting the ability to detect the tag, which is determined solely by whether the tag is visible from the wrist at the home pose or not, not how many pixels it occupies after detection.

Table 3.2: Perception requirement and their constraints.

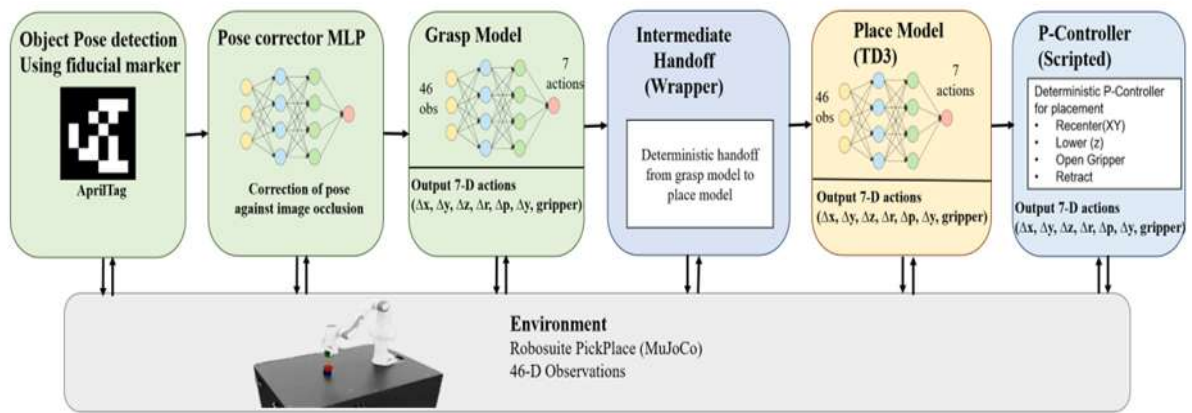
Requirement	Constraint	Design response
Model size	Fit actor + TFLite-Micro arena in on-chip RAM (few KB budget)	64-32 MLP, 46-D vector observation, INT8 quantization ($\sim 4\times$ smaller)
Inference latency	One forward pass $\ll$ control period	Full-integer INT8 inference (~ 15 ms vs ~ 30 ms FP32)
Inference frequency	Fresh action every step at 20 Hz (50 ms period)	Per-step closed-loop pipeline with ~ 35 ms margin for I/O
Perception footprint	Detector + corrector must coexist with both actor tensor arenas inside the ESP32's usable heap (~ 210 KB, <code>MALLOC_CAP_8BIT</code>), no PSRAM	Memory-reduced AprilTag C port ($8 \rightarrow 4$ B/px union-find, cropped ROI, tuned cluster-map knobs) + a 6.1 KB FP32 residual corrector, run once per episode rather than every control step

3.5 Model Design and Performance Criteria. The dominant obstacle was the size of model required to learn the whole task which is became too large for esp32. Then the problem of transport-versus-release conflict described in Section 3.6.2 Training Algorithm Design Steps, Step 3: the same network must both hold the object securely during a long transport and then deliberately let go at a precise location. These sub-goals pull the gripper action in opposite directions, and the policy repeatedly collapsed into one of two degenerate modes dumping the object early, or a safe-hold local optimum in which it lifted the object and held it motionless until timeout.

Attempts to fix this within the monolithic frame is rebalancing drop and hold penalties, and a systematic penalty-shape sweep - required very big model size (Several Megabytes). Moreover, The conflicting rewarding of grasping and then releasing made the training very unstable.

3.7.2 Our Model

The final model realizes the staged architecture: two learned actors sequenced by the



Both the grasp model and place model are **quantized** using QAT for the final deployment to increase inference frequency

Figure 3.1 Two stage model for the pickplace task, representing Grap and Placmodel (transporter) with Handoff handler in between. And then finally the P Controller takes over to lower and release the object.

deterministic FSM of Table 3.3, with scripted micro-skills at the transitions and a set of intermediate handlers that make the composition robust. **Error! Reference source not found.** (conceptual) is captured by the FSM table; the components below detail how the stages are glued together.

Learned sub-policies.

- Grasp actor. A 64→32 policy that reaches for and secures the object, trained under the dense staged reward. It lands around 95% grasp success.
- Transport actor. A 64→32 policy trained transport-only it commands translation while the gripper is scripted closed on every step, so it never actually learns to open the

gripper. Its goal state is "object held stably over the bin," which is what triggers the scripted release. This is what turns the old monolithic safe-hold failure mode into the success condition instead.

Intermediate Handlers

1. Stage wrappers. The grasp environment wrapper defines the dense grasp reward and terminates early on a confirmed grasp. The place environment wrapper begins each transport episode by running the frozen grasp policy to a stable grasp before handing off to the learning policy. Between the two, each stage ends up a clean, self-contained learning problem.

2. Test-lift grasp filter. Between grasp and transport, a scripted probe commands a gentle straight-up lift for ~20 steps and accepts the handoff only if the object rises ≥ 3 cm and stays gripped; otherwise it retries the grasp (≤ 8 attempts). This is what let the grasp model skip retraining altogether the grasps it already produced just needed selecting for ones strong enough to survive motion.

3. Transport action shims (handoff/carry invariants). During transport, three scripted invariants protect the grasp: orientation is frozen (rotation dims set to zero, since rotations shear the object out of the gripper and placement needs no reorientation), the gripper is masked closed (so exploration noise can't pop the grasp open), and translation is gentled (halved) to cap commanded acceleration so real grasps survive inertial load. The masked action that is actually played is stored in the replay buffer and the transitions remain consistent.

4. Scripted release routine. Once the object has been held within the release radius for a hold count, a scripted routine runs four phases: recenter over the bin (P-control in XY), descend until near the bin floor or until height stalls (a touchdown proxy, since there's no force sensing), open the gripper, then retract upward. The retract step isn't optional, robosuite's success check requires the gripper to be clear of the object, so without retracting first, an open-and-check routine would score genuine placements as misses.

Two further design choices support robust training of the transport actor. One is a reverse curriculum, in which a scripted P-controller first carries the object to within a fraction of the full distance from the bin and hands off there, with that fraction ratcheting from 0.2 toward 1.0 as the policy demonstrates sustained mastery (and regressing if it falters). The other is difficulty-weighted best-model governance, where the best policy is selected by the product of rolling mean difficulty and rolling mean success rather than by single-episode score, with

each new best snapshotted into a write-protected directory. Together with the potential-shaping and termination-design decisions from Section 3.5.2, these define the complete final model.

Deployment Form.

Both actors are QAT fine-tuned, converted to full-integer INT8, and embedded as C arrays in the ESP32 firmware, where a TFLite-Micro interpreter executes them from a tensor arena in on-chip RAM. The FSM meta-controller and scripted micro-skills carry no learned parameters, so they add negligible memory and inference cost.

3.7.3 Perception-Augmented Object Pose

The addition of perception does not change the shape of the staged model of section 3.6.2 except in the input to one slice of the model. At the 1st control step of an episode the on-device detector finds the AprilTag in the view of the wrist camera, removes the planar pose ambiguity, and returns the raw pose to the residual corrector that returns an object position and orientation in metres. That corrected pose is stored in the same actor-actor pose slice of the 46-D observation as the simulator's ground truth did before, so there's no need for either actor network to know if the number is from a physics engine or a camera. After the grasp closes the object's pose is no longer read from the detector at all: the object's pose will be latched at the grasp offset upon grasping and carried forward by the forward kinematics of the grasp for the rest of the episode. Computing a world pose after grasp would actually be a bad idea, as we would be holding the wrong pose as the arm moves away with the object. The engineering design and methodology of the proposed embedded reinforcement learning system was presented in this chapter. It described the design requirements, system architecture, development and the evaluation criteria, that were adhered to while developing the proposed solution. All of these form the basis of a controller for a robotic pick and place system that is efficient in use of resources and can be implemented on a low cost microcontroller. The next chapter describes the implementation of the proposed system. It provides a description of the reinforcement learning formulation, training process, model compression, component selection, optimization and deployment process that was used to implement the engineering design presented in this chapter.

3.8 Summary

The engineering design and methodology of the proposed embedded reinforcement learning system was presented in this chapter. It explained the design specifications, system structure, development process and performance criteria that were followed when developing the proposed solution. All of these form the basis of a controller for a robotic pick and place system that is efficient in use of resources and can be implemented on a low cost microcontroller.

The next chapter describes the implementation of the proposed system. It provides a description of the reinforcement learning formulation, training process, model compression, component selection, optimization and deployment process that was used to implement the engineering design presented in this chapter.

Chapter 4

System Implementation

4.1 Introduction

The overall methodology and engineering design of the proposed system were presented in the previous chapter and included the design requirements, system architecture and performance criteria. On this basis, this chapter outlines the implementation of the proposed embedded reinforcement learning framework. It provides an overview of the development, training, optimization, and deployment of the designed system on a resource constrained microcontroller.

In Section 4.2, the reinforcement learning (RL) problem formulation is introduced, which consists of defining the state space, action space, reward function, and learning objective. The training methodology is outlined in Section 4.3, which also introduces the staged learning strategy for training the individual control policies. In Section 4.4, the compression methodology of the trained models is explained, which is quantizing models and converting them to an embedded-friendly format for deployment. In Section 4.5, the hardware and software components chosen for system implementation are introduced. The optimization methods used to enhance the computational efficiency without compromising the performance of the tasks are described in Section 4.6. The execution principle is summarized in Section 4.7, which describes the interaction of the trained models, finite state machine (FSM) and embedded controller during task execution.

In total, this chapter gives the complete implementation procedure of the proposed framework, ranging from reinforcement learning formulation to embedded deployment.

4.2 Methodology of Formulating The RL Problem

1. Environment Platform: For a number of practical reasons, the task was implemented in robosuite, a manipulation framework that is built on top of the MuJoCo physics engine. It ships with Franka Emika Panda 7-DOF arm already modelled, it offers an operational-space pose controller (OSC_POSE) to enable actions to be commanded as a small Cartesian pose change rather than as a small vector of joint torques, and GymWrapper flattens the observation into a plain vector that a small network can take directly. Building an equivalent contact-rich simulation from scratch, with a correctly tuned gripper and stable grasping physics, would have consumed most of the project on its own.



Figure 4.1: Robosuite pick-and-place environment with 7-DOF franka panda robotic manipulator with picked object (bread) beside a table visualized.

2. Task Environment: The specific task is robosuite's PickPlace environment restricted to a single object with `single_object_mode = 2`, which fixes the target to the bread object and its own bin. Two properties of the setup matter for the learning problem. First, the object spawns at a random position on the table at the start of every episode, so the policy cannot memorise a fixed reach; it has to learn a grasp that generalises across the spawn region. Second, the observation is the 46-dimensional full state, robot proprioception together with the object

pose, and there are no cameras anywhere in the pipeline. Keeping the state fully observable means the control problem stays a Markov decision process, and a plain feed-forward MLP is enough to solve it. That single choice is what later makes it realistic to fit the policy on a microcontroller, so it was made deliberately at the formulation stage rather than as an afterthought.

3. Action Space: Actions are the seven-dimensional vector $[dx, dy, dz, droll, dpitch, dyaw, gripper]$, with the gripper command closing at +1 and opening at -1, and the controller runs at 20 Hz.

4.3 Methodology of Training

1. Preliminary Approach: The first working version of the controller was a single network that tried to do everything. One TD3 policy took the 46-dimensional observation and produced the full seven-dimensional action for the whole episode: reach, close, lift, carry, and release. This was the “clean” version on paper and, in a way, it seemed to be working for a while. The arm was able to reach for the object and close the gripper on the object reasonably early in the training process.

2. Reward Engineering: It was at placing where it broke down. After the object was captured in the gripper, the policy was prone to either of two undesirable actions. Occasionally it would throw the object out on the way to the wastebasket. It lifted it up a bit and then stood there still until the episode expired. If the reward was changed, the failure was moved around, but it was not removed: if the reward was to throw away the failure, then the policy began throwing away the failure early, if the reward was to drop the failure, then it was put back to holding. A number of runs were terminated in the same location.

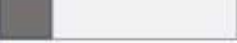
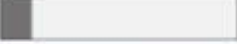
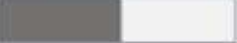
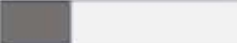
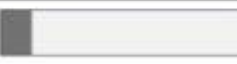
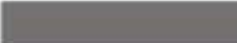
Stage	Behavior(Objectives)	Reward(Example scale)
 Reach	Move gripper close to object	 1.0
 Grip	Close gripper when near	 0.5
 Grasp	Maintain stable grasp	 10.0
 Lift	Lift object above table	 3.0
 Hover/Approach	Move object to target bin	 0.5
 Success	Place in bin (one-time)	 50.0

Figure 4.2: Reward Design for training involving various subtasks.

3. Task Decomposition: After a few of these runs it was no longer a tuning problem. A long-distance carry and a deliberate drop are opposite actions at the level of the gripper command and one small network was expected to do both. It was this that broke. That is the way it's read, and that is what led to the splitting of the task in two, as described in Chapter 3. In practice, the split involved training grasping alone until the arm could successfully grasp the object, and then training transport-and-place on top of a good grasp.

4. Grasp Policy Training: The grasp policy was trained first, using a dense staged reward that shapes the whole behavior: a distance term that rewards getting the gripper to the object, a bonus for closing the gripper once it is close, a per-step reward for keeping the object held, a one-off bonus for a confirmed stable grasp, and small penalties for idling, dropping, and backing away. Because the signal is dense there is a gradient almost everywhere, and this stage trained without any of the extra machinery the place stage later needed. It reached roughly 95% grasp success, with training stopped once success held above 95% over the last hundred episodes.

5. Transport-and-place Policy Training: The transport-and-place policy was then trained with the grasp policy frozen. Each place-training episode starts by running the frozen grasp policy until the object is held, and only then hands control to the policy being trained. The wrapper that does this hand-off, and the filter that decides whether a grasp is good enough to hand off at all, turned out to be as important as the reward, and they are described in Section 7.4 along with the rest of the problems that came up on the place side.

4.4 Methodology of Compression

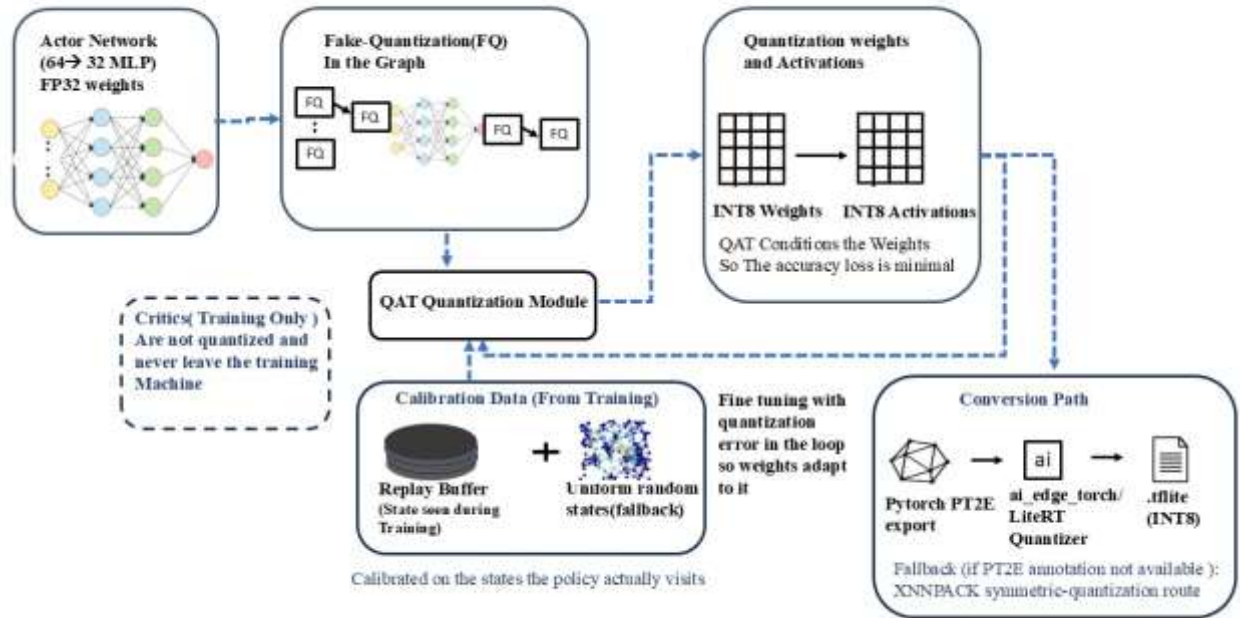


Figure 4.3: Methodology of Compression with QAT detailing finetuning loop for optimizing weights.

A trained actor is a floating-point PyTorch network, and an ESP32 cannot run that at any useful speed. Both deployed actors, the grasp policy and the transport policy, were therefore fine-tuned with quantization-aware training (QAT) and converted to full-integer INT8. The critics are never quantized because they never leave the training machine; only the actors are deployed.

The compression was not a single conversion call but a short pipeline. The trained FP32 actor is first fine-tuned with fake-quantization inserted into the graph, so the network sees quantization error during training and adapts its weights to it. The data used for this fine-tuning is drawn from the states already sitting in the training replay buffer, with uniform random states as a fallback, so the quantizer is calibrated on the states the policy actually visits rather than on an arbitrary distribution. The fine-tuned model is then converted to a full-integer INT8 .tflite flatbuffer, quantizing both weights and activations. Because QAT has already conditioned the weights, this conversion step loses very little accuracy. The conversion path used PyTorch's PT2E export flow together with the ai_edge_torch / LiteRT quantizer, with an

XNNPACK symmetric-quantization route as a fallback where PT2E annotation was not available.

The last step turns the .tflite file into something the microcontroller firmware can compile. A small script walks the flatbuffer byte by byte and writes it out as a C array in a header file, which is then `#included` into the ESP32 sketch and ends up in flash as a constant. On the device, TFLite-Micro reads the model straight from that array. The expected payoff, later confirmed on hardware, is a model about a quarter of the FP32 size and inference roughly twice as fast, around 15 ms instead of 30 ms.

4.5 Selection of Components

The hardware target was fixed early: an ESP32. The whole point of the thesis is to show that this skill fits on a cheap, ordinary microcontroller, so a more capable board would have undermined the claim. The ESP32 has enough on-chip RAM to hold a small INT8 model together with the TFLite-Micro tensor arena, it runs the integer network fast enough to stay inside the 20 Hz control period with room to spare, it has a USB serial interface that the HIL loop needs, and it is supported by the Arduino toolchain and by ArduTFLite/TFLite-Micro, which is what made embedding the model as a C header straightforward. It also costs a few dollars, which is the entire argument in one number.

The Python libraries, listed in Table 7.1 were used. Nothing here is exotic; the choices were made for stability and for having a clean path from a trained PyTorch model down to an integer model on the device.

Table 4.1: Main software components and what each one is used for.

Library / tool	Role in this work
robosuite + MuJoCo	Simulation of the Panda arm, the object, the bin, and the contact physics; provides the PickPlace task and the OSC_POSE controller.
PyTorch	Defining and training the TD3 actors and critics; QAT fine-tuning via the PT2E export flow.
NumPy	Observation and action handling, buffers, and the numerical glue throughout the training and HIL code.

Library / tool	Role in this work
ai_edge_torch / LiteRT + TensorFlow Lite	Converting the QAT-fine-tuned actor to a full-integer INT8 .tflite model (XNNPACK fallback where needed).
pySerial	The PC side of the HIL link: owning the serial port and moving framed messages to and from the ESP32.
Arduino / ArduTFLite (TFLite-Micro)	On-device firmware that parses frames and runs INT8 inference on the ESP32.

4.6 Optimization

This section is the part of the work that took the longest and that shaped almost every design decision in Chapter 3. The grasp model needed very little of it, so it is dealt with quickly. Everything after that is about the place model, presented roughly in the order the problems were hit, because each fix tended to expose the next problem.

The grasp model. Because the grasp reward is dense, training it was mostly a matter of leaving it to run. It reached about 95% success without a curriculum, without special replay handling, and without any of the stabilisation tricks the place model later required. The one design point worth carrying forward is that its reward terminates an episode early on a confirmed grasp, which keeps grasp episodes short and clean. The lesson of the project is that contrast, an easy dense-reward stage next to a very hard sparse-reward stage, is one of them.

For the random object spawn environment, the wide variation in the spawns used for this stage of the grasp, from an anchor to a fully randomized position and yaw rotation, reopened the model-size and fidelity questions that were closed in Section 4.4, which were observed when using the fixed spawn actor; here, the first evaluation of the fixed-spawn seeds was converted to INT8, which was significantly lossier (an early FP32-to-INT8 gap of approximately 12 percentage points on held-out random-spawn seeds). That gap was closed by a series of levers that were added to the previous best, and were measured against it before being retained.

1. **Weight-range clipping.** The main cause of the initial spread between seeds was that all of the weights were bounded at most 8 standard deviations before quantization was applied, which caused a few catastrophically-quantized seeds.
2. **In-loop quantization-aware training.** The idea was to fold fake-quantization directly into the training loop of the actor, as opposed to fine-tuning a pre-trained FP32 actor against fake-quantization after training. This reversed an earlier interpretation of the fixed spawn campaign that attributed the loss of accuracy to QAT, when in fact the problem was a distillation loss that did not correlate with task success.
3. **Replay buffer scaling.** The largest lever measured was an increase in the replay buffer from 200,000 to as many as 2,000,000 transitions, with a significant impact primarily on variance: the standard deviation of the seeds in the held-out evaluation set dropped by about an order of magnitude.
4. **Matched critic capacity.** With the buffer large enough to accommodate it, widening both of the two critics (keeping the original deployed actor at 64→32) added an additional smaller gain to the INT8 and FP32 actor, but did not require any additional deployment cost because only the actor is quantized and deployed.

Combined, these four levers were able to close the gap of the random spawn INT8 and, on the deployed grasp actor, bring the INT8 model into parity with, and in some seeds ahead of, the FP32 original; the held-out numbers are reported in Section 6.3.3. The transport actor is not used in this campaign at all: the task is decomposed, the position and rotation randomness is fully absorbed by the grasp stage, and the transport actor is already deployed in the fixed-spawn system, and is deployed unchanged (Section 4.6, place model).

Table 4.2: Random-spawn INT8 quantization levers, applied cumulatively.

Lever	INT8 effect (held-out, random spawn)
Weight-range clipping ($ w \leq 8 \cdot \text{std}$)	+7.25 pts — removes catastrophic seeds
In-loop QAT (fake-quant inside the RL loop)	+4.43 pts; FP32 unchanged
Replay buffer 200k → 2M	+15.17 / +14.00 pts; seed SD 17.18 → 2.84
Matched critic capacity (given the larger buffer)	+3.12 INT8, +6.04 FP32 — free at deployment

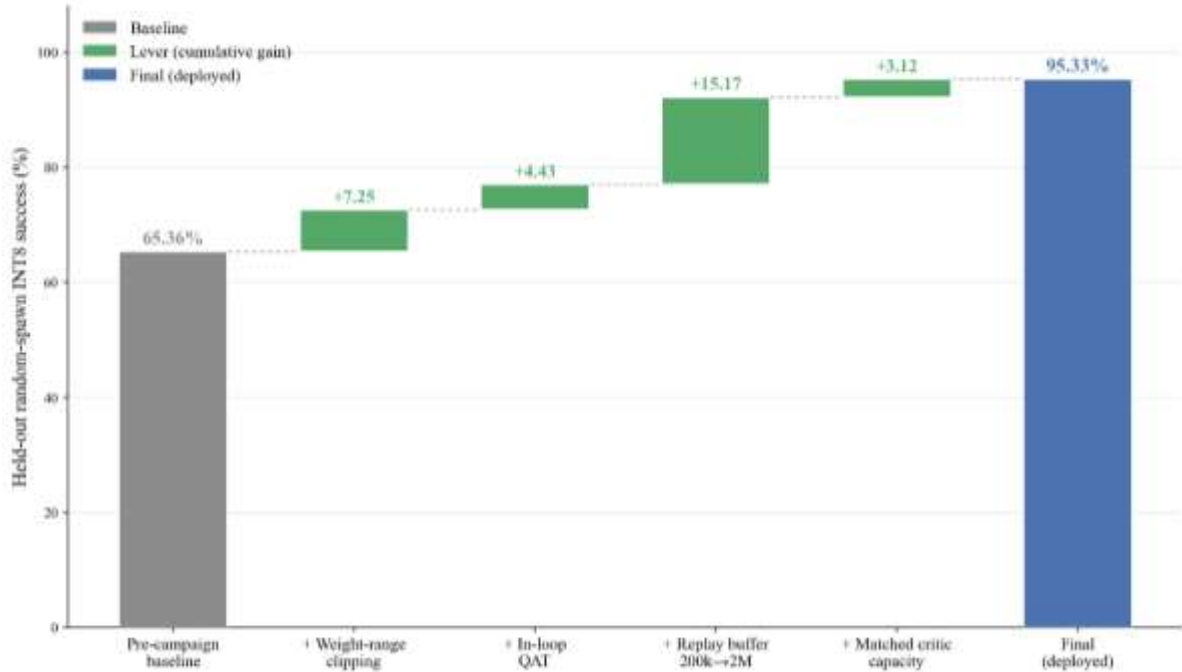


Figure 4.4: Cumulative effect of the random-spawn INT8 quantization levers.

The case was different for the place model. The problems below are numbered for reference, but they were not solved on a schedule; a few of them were being fought at the same time.

1. The hover exploit. With per-step rewards for lifting and holding the object over the bin, the policy found that it could hover the grasped object above the target forever and keep collecting reward, with no reason to ever release. The fix was to throw out the per-step state rewards and replace them with potential-based shaping of the form $r = \gamma\Phi(s') - \Phi(s)$, which is policy-invariant and so cannot be farmed by lingering. Φ is a bounded milestone potential over the object pose (lift, then approach in XY, then place in 3D), with a per-episode ratchet on the lift term so that lowering the object into the bin, which sits below the object's starting height, does not wipe out the lift credit already earned.
2. The object shaking loose in transit. Even with the reward fixed, most episodes were losing the object partway through the carry, around the eleventh to seventeenth step, after a real lift had already happened. The grip was being shaken open by the motion rather than opened by the policy. Three scripted invariants were added to the transport action to stop

this: the orientation commands are frozen to zero, because rotating a closed gripper shears the object out of it and placement needs no reorientation anyway; the gripper command is masked closed during transport, so exploration noise cannot pop it open; and the translation commands are halved, which caps the commanded acceleration so a real grasp survives the inertia. The action that was actually performed is stored in the replay buffer, not the action that the policy actually outputted, so the stored transitions remain consistent.

3. Comprehended, but lacked strength to carry. There were a few hand-offs which still managed to get in in the first few seconds because the grasp model was only ever rewarded for grasping, not grasping and holding under motion. Instead of retraining the grasp model, a scripted test-lift filter was introduced to the place-environment reset: Once the grasp is confirmed, a simple proportional controller is used to command a gentle straightup lift for approximately twenty steps, and the hand-off is accepted only if the object moves at least 3cm upward and remains grasped. If it does not work, it is repeated 8 times. This went in and after that, for a long time, the number of drops in the carry dropped and the average number of hand-off attempts dropped to around 2.5 to 3, telling me that the good grasps were already there and just needed to be picked.
4. The safe-hold local optimum. It was the tough one and it returned over a few runs. The policy would find placement, create a few successful placements and then fall apart into picking up the object and keeping it motionless until timeout. A bounded, zero-variance outcome will be a happy place for a risk-averse method, such as TD3, to end an episode in which it is stuck. It did not move when I changed the drop and approach rewards. What did was that if you end an episode with the object in your hand, you get a -20 penalty, but if you end an episode without the object, you get an even bigger penalty. But the penalty was set large enough so that it is never worse to carry the object to the bin and release it than to carry it to the bin and drop it, but not so large that it is never worse to carry the object to the bin and drop it than to carry it to the bin and release it.

5. Shaping penalty - negative. It was only logical to design the terminal penalty to be a shallow gradient towards the bin, increasing with the distance, so that a transport gradient was added. It exacerbated the situation. The critic was destabilized by the deep, high variance terminal target, the policy oscillated between dumping the target and safe-holding, and forgot previously-learned difficulty levels, dropping from 89% down to 14%. Soften the penalty, and remove the dumping, and safe-hold returned. It was the uniform and even release pressure of the flat penalty that worked. This is a negative result because the transport gradient which it sought to add was really in the approach potential, the next item.
6. The zero-gradient desert. This was the one bug that was the most predictive of the entire campaign. The approach potential went to exactly zero beyond 25 cm from the bin, and a full transport is about 53 cm. Any curriculum hand-off further out than about half the distance therefore started in a region with no directional reward at all, and in a flat region it is indeed the locally optimal thing to do. That one fact was the key to every wall that had been witnessed: runs would stall around the 40-45% difficulty level, just on the edge of the field, and then fall into safe-hold as soon as the curriculum took them into the flat area beyond 50-65%. Widening the scale of that potential from 0.25 to 0.60, so that its gradient covers the entire transport, was the fix. The run following that change increased in difficulty from 20% to 85% at 84% success rate in approximately 1,600 episodes, which is significantly higher than the previous best of about 65%..
7. The policy would sometimes wander down and then spin out of control near the difficulty level. Here, two mechanisms were separated. The first is replay of failure terminals with large errors: the large-error failure terminals are replayed in preference to other failure terminals during a dip, which pulls the transport values down and creates more failures. Reducing the priority exponent of the PER (from 0.6 to 0.4) was helpful, but on its own did not eliminate the collapses, and is reported as a falsified single cause explanation. The second mechanism was the one that was really at fault here and it's a buffer composition issue, not sampling. A failed 'held' episode is approximately 200 transitions long and a successful episode is approximately 80. With a single bad window of a fifty episode, the

critic gets an overload of, say, nine thousand 'hold, small negative' transitions, and a few hundred success transitions, regardless of the sampling weights. The fix was a transport-stall early termination: if the object does not set a new best distance-to-bin for 50 consecutive steps that are gripped, then the episode is terminated early. That reduced the amount of poison per failure by about 4 times and quadrupled the buffer's recovery rate after a crash, with the monotonic crashes becoming dips that the policy can recover from in the logs.

8. The conflict of transport versus release is solved by architecture. It's the same battle that ended the monolithic model, and following the penalty sweep, it was evident that there was no single reward number that could balance it. So it was removed rather than balanced. The learned policy was made transport-only, with the gripper scripted closed on every step, so it never opens the gripper at all. Release became a scripted routine triggered once the object has been held inside the release radius for a few steps: a proportional controller recenters over the bin, lowers the object until it is just above the bin floor or until its height stops changing (a stand-in for a touch sensor, since there is none), opens the gripper, and then retracts. The neat part is that the safe-hold behaviour that had been the worst failure mode, 'object held stably over the bin', is exactly the condition that now triggers release. The failure mode became the stage boundary. One implementation detail cost real time before it was understood: robosuite's success check also requires the gripper to be clear of the object, so a routine that opens and checks immediately scores genuine placements as misses. That was corrected by retracting the empty gripper prior to accepting success.
9. Checkpoint governance. Good policies are lost to two operational bugs. The periodic checkpoint and the new-best checkpoint were writing to the same files, so a later, worse periodic save could overwrite the best model; the fix was to snapshot every new best into a separate, write-protected directory. And under a curriculum, single-episode score is a meaningless 'best' criterion, because a success scores about the same at any difficulty, so the saved 'best' froze on an early easy episode while a far stronger policy went unsaved. The criterion was changed to a difficulty-weighted rolling success, the product of mean

difficulty and mean success over a window, so a policy only sets a record by succeeding often and at difficulty.

10. Two supporting pieces run underneath all of the above. The environment wrappers are what make model-to-model transfer clean: the grasp wrapper defines the dense grasp reward and the early-termination-on-grasp behaviour, and the place wrapper runs the frozen grasp policy at the start of every episode and manages the hand-off. And the reverse curriculum is what made the sparse place reward learnable at all: a proportional controller first carries the object to within some fraction of the full distance and then hands off, with that fraction starting at 0.2, where success is almost immediate, and ratcheting up toward 1.0 as the policy proves it can hold a level. It progresses one step only when it masters the skill, which is about 75% over the entire 40 episodes period; it goes back one level if it fails to master it within 15 episodes; it steps more in the hard region, because if it fails in 15 episodes, it steps 0.1 in the wrong direction and the object falls off the table while the arm is still holding it.

4.7 Principle of Execution

Two things run in stages: the training and the execution. They are worth separating.

Staged training means the two policies are never trained together. The grasp policy is trained first, on its own dense-reward problem, until it can reliably pick the object up, and then it is frozen. The transport policy is trained afterwards, with the frozen grasp policy standing in front of it: every place-training episode begins by letting the grasp policy get the object into the gripper, the test-lift filter decides whether that grasp is worth keeping, and only then does the learning policy take over. Freezing the grasp policy keeps the transport policy's learning problem stationary; if the grasp behaviour were still shifting, the transport policy would be chasing a moving target.

Staged execution is the finite state machine from Chapter 3 running for real. The controller moves through grasp, test-lift, transport, and release in order. The two learned policies handle the grasp and transport states; the test-lift and release states are scripted proportional controllers. Each state has a definite exit condition, so the sequencing is deterministic and

there is no learned decision about which policy to run when. Because the task order is known, that meta-level decision does not need to be learned, and leaving it out keeps the on-device footprint to just the two small networks.

Under HIL the same execution runs, but split across two machines. The FSM, the scripted controllers, and the physics all live on the PC; the neural inference for the grasp and transport states lives on the ESP32. When the FSM is in a learned state it ships the observation to the device

and waits for the action; when it is in a scripted state it computes the action locally. From the

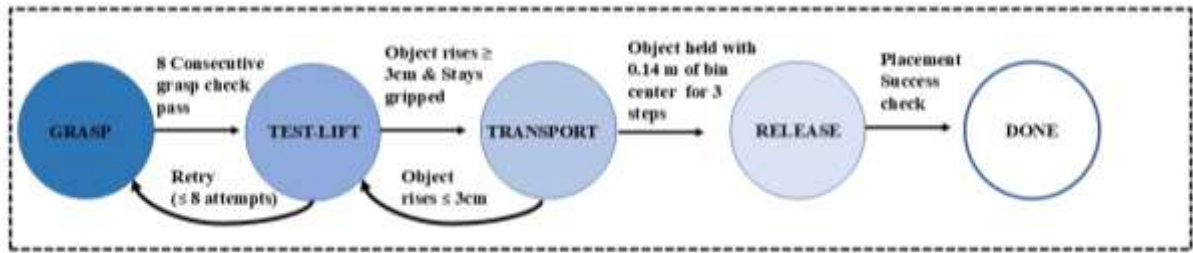


Figure 4.5: FSM diagram for the whole task.

simulation's point of view nothing changes, an action arrives every step, but for the learned states that action has made a round trip to real silicon and back.

4.8 On-Device Perception Implementation

The controller is described in Section 4.2–Section 4.7 is never required to use a camera to be an MDP, the 46-D state vector is enough, but its object-pose slice is filled. This slice is not populated by the simulator, but by a fiducial detector and a learned pose corrector, which also run on the ESP32 that is used to run the two control policies.

4.8.1 Detection Pipeline

The arm is known to be at its home pose at the first control step of an episode, so the camera's field of view of the tag is predictable enough that it can be cropped to a region of interest prior to the actual tag being detected, which keeps the detector within budget without sacrificing the tag's ability to decode. Then, it follows a series of steps to detect, all running on the ESP32: a quad thresholding and clustering over the cropped and decimated frame, a decode step from the tag16h5 family, which happens to be the only family the memory-limited port supports, a disambiguation over the well-known planar pose ambiguity (a homography admits two

consistent poses, and an upright-normal prior together with pose refinement equivalent to solvePnPGeneric resolves this ambiguity in favour of the physically sensible one), and a fixed extrinsic calibration that maps the decode camera-frame pose to the world-frame the policies expect. The object-pose slice of the state vector for that episode is then updated with the corrected pose.

4.8.2 Memory-Reduced AprilTag Port

The AprilTag implementation vendored for this work is for a variant of the ESP32 that has several Megabytes of PSRAM; none are reported on this board. Three changes added up to nearly closing the gap. Each of these fields is only ever used for relative cluster size comparison, and every cluster that matters is orders of magnitude smaller than this, so this thesis is safe to use 16-bit values in the size and parent fields (reverting to 32-bit if the cluster id overflows), since the single largest allocation in the union-find structure that AprilTag's clustering stage uses is a 32-bit parent and size field upstream. By cropping to a region of interest that is determined by measured wrist-camera tag statistics, instead of decimating the full frame, the detector doesn't have to allocate any pixels for at all. The internal growth and chunk-size knobs of the clustering stage had been defaulted to standard PSRAM board settings, and these had to be tweaked to fit this memory budget, as now they were compile-time constants. All three changes are verified as being "function preserving" (with regards to the unmodified port) on a fixed set of frames recorded prior to being accepted, and all three do not change the result that the detector reaches for a tag.

4.8.3 Residual Pose Corrector

The raw pose that a detector receives is not the pose that the policy requires: reprojection error, the apparent obliquity of the tag, its distance from the camera, and several other per-detection features, are correlated with an almost constant residual offset that depends only on one camera, one extrinsic calibration and one detector implementation. This thesis tries to learn that offset directly from the features, after normalizing them on device prior to the forward pass, using a small fully connected network (12 input features; 2 hidden layers (32 units, ReLU activation); 3 unit linear output (metres). The network is not quantized: FP32 is a bit more accurate than INT8, and the residual the network is correcting is a few tens of millimetres: that's a lot more for the residual to be accurate than for the quantization error. It's a small enough cost to maintain in full precision (1571 parameters / ~6.1KB).

4.8.4 Latching Through the Grasp

The detection is not done on a continuous basis. When the gripper closes around the object it also blocks the tag, so a world position estimation computed by the world camera after this would be wrongly using an old tag reading while the arm is moving away from the tag. Thus the firmware latches the corrected object pose in the gripper's own frame when the grasp is executed and keeps propagating it forward based on the arm's forward kinematics which are exact from the joint encoders for the rest of the episode. This also means that only one detection per episode is enough: the perception duty cycle this design requires is 20 Hz / 10,000 steps, which is a small fraction of the per-step budget for the ESP32.

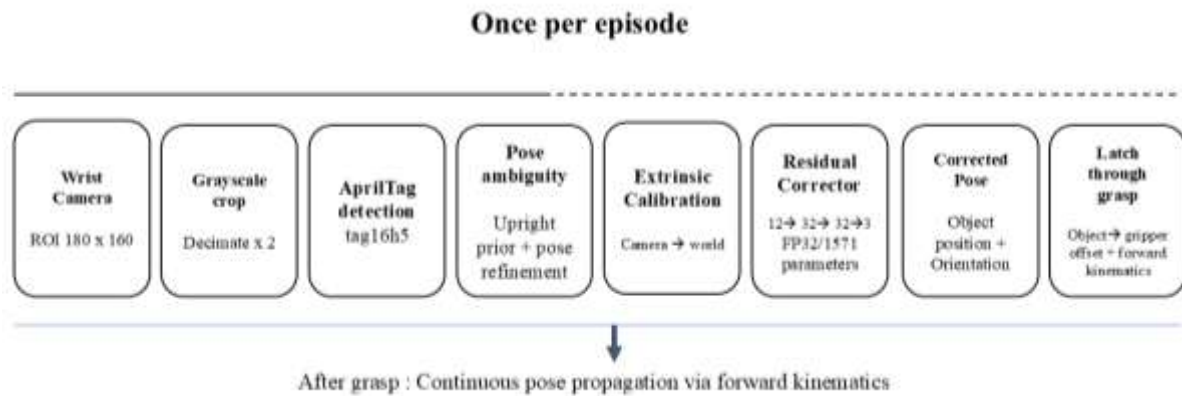


Figure 4.6: On-device AprilTag detection and correction pipeline.

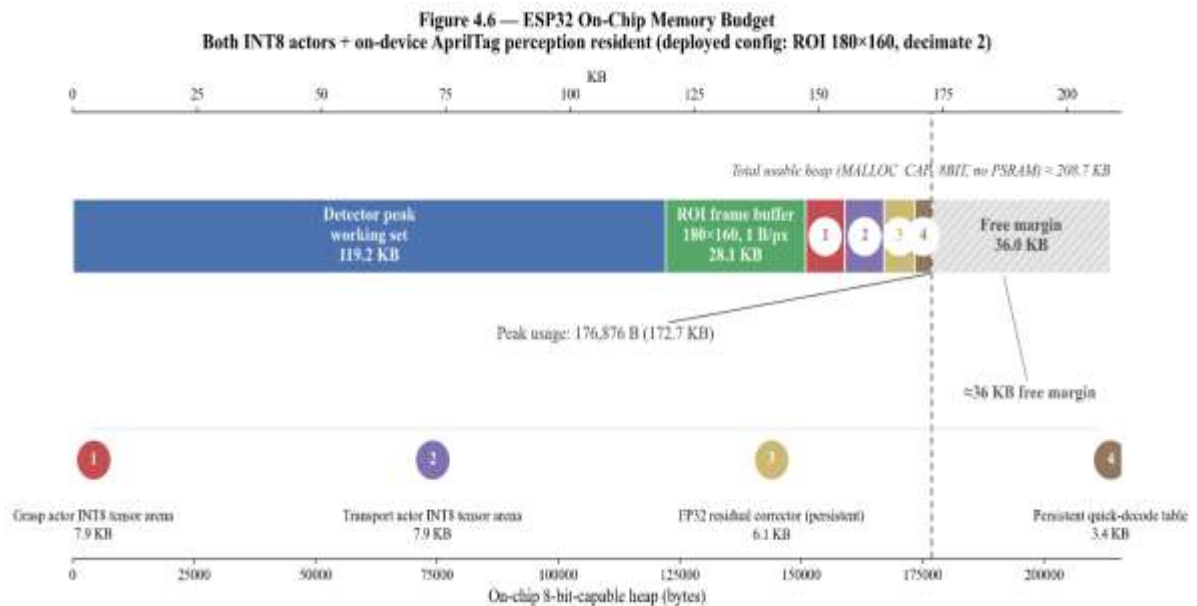


Figure 4.7: ESP32 on-chip memory budget with both actors and the perception pipeline resident.

4.9 Summary

The implementation of the proposed embedded reinforcement learning framework was described in this chapter. It introduced the problem of reinforcement learning, the staged training approach, the on device pose perception, the model compression method for deployment on INT8, the hardware and software components used, and the optimization techniques employed to ensure compliance with the resource limitations of the target microcontroller. Lastly, it illustrated the overall execution principle, demonstrating the interaction between the trained policies, finite state machine (FSM), and embedded controller in task execution.

The next chapter describes the experimental setup adopted to test the proposed system, after the implementation. It outlines the validation approach, simulation setup, performance evaluation criteria and the effectiveness of the developed framework prior to presentation and analysis of the experimental results.

Chapter 5

Experimental Set up

5.1 Introduction

The previous chapter gave the building of the proposed framework. The set-up of the problem was covered there. The training, the shrinking of the model, and the loading of it on the chip were covered also. A check of the system under fixed conditions is the next need.

This chapter gives a description of the tools and of the way of measurement. The building of the pipeline is not the topic here. The reporting of the numbers is not the topic either. That reporting has a place in the results chapter.

Three questions get an answer in this chapter. What tools were used? What was written down from them? Under which rules were the runs done? The reasons behind the design choices belong to other chapters.

Section 5.2 gives the way of checking. Sections 5.3 and 5.4 give the simulation set-up and the test bench. A list of the measuring tools is in Section 5.5. The calibration is in Section 5.6. Section 5.7 gives the steps of the work. Section 5.8 gives the data collection. The performance numbers are in Section 5.9.

5.2 Methodology of Validation

The checking was done in two stages. The order of the stages was a choice on purpose. A failure of the second stage must not get mixed up with a failure of the first one.

5.2.1 First stage: offline check on the computer

The first stage is a run of the whole pipeline on the computer. No hardware takes part in it. The noise for exploration is switched off. A test script drives the full sequence. The sequence is this: grasp policy, test-lift, transport policy, then the scripted release.

Fifty episodes at the full transport distance make one evaluation. A count of the good placements is reported from it. The fixed nature of this run has importance. The training success gets a measurement while noise is still shaking every action. So an under-statement of the policy is given by the training number. The noise-free number is the honest one.

The tuning of the FSM values was done at this stage. A confirmation of the policy was also taken here. Nothing was written to a device before that confirmation.

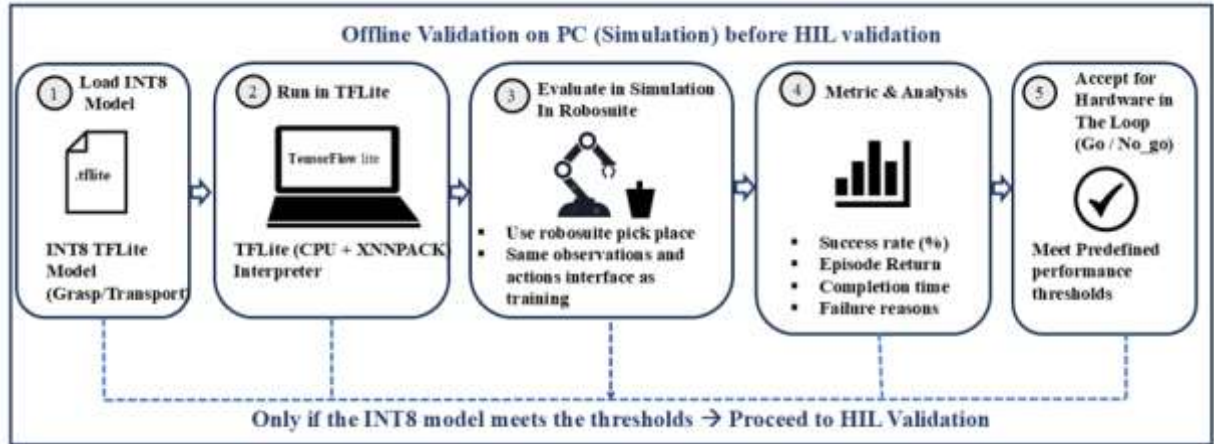


Figure 5.1: Offline testing architecture of the trained model on the computer. Only a model with a pass on the thresholds goes to the second stage.

5.2.2 Second stage: hardware-in-the-loop check

The second stage is a hardware-in-the-loop test. The physics simulation still has a run on the computer. The working out of the control action at every step is done on the real microcontroller.

The order of one step is a fixed one. The computer takes the forty-six number observation. A sending of it to the ESP32 follows. INT8 inference is run by the device. A return of the seven number action is then made. Only after that is an advance of the simulation taken.

The whole-number arithmetic of the network gets a work-out from this arrangement. The memory behaviour and the serial delay get a work-out also. All three behave in the same way as in a real deployment.

A limit of this stage needs a clear statement. The real robot is absent. The physics and the sensing are made-up ones. So a test of the deployment is given by the bench. A test of the physics is not given by it.

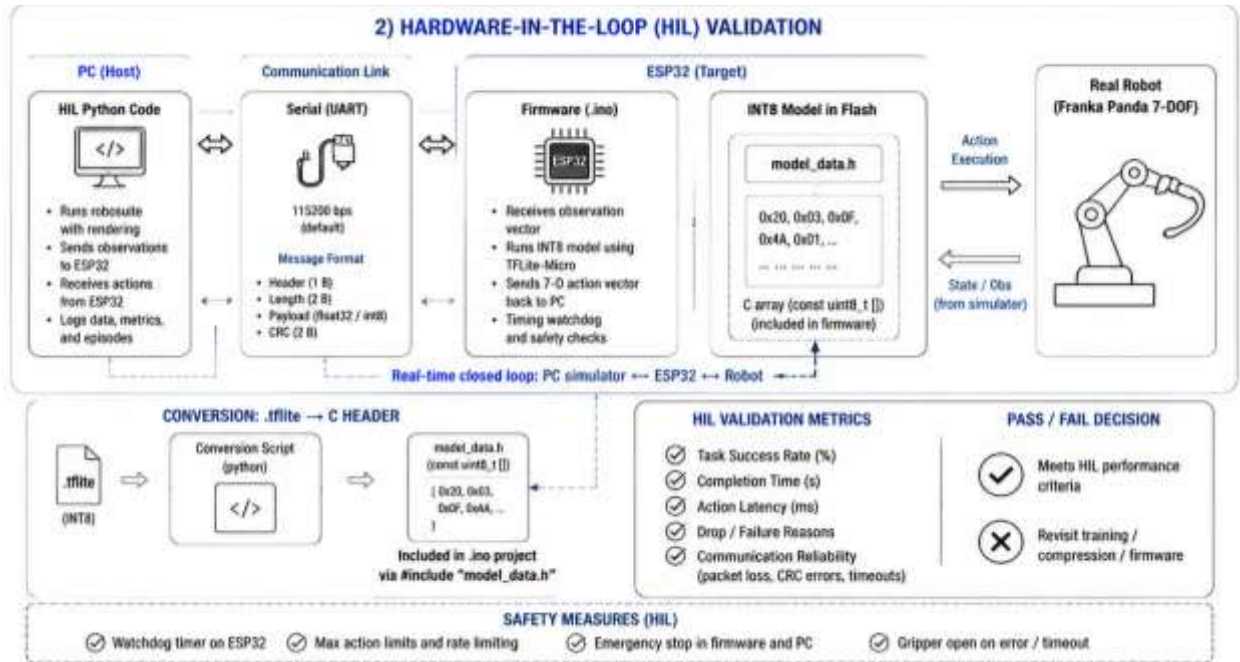


Figure 5.2: Flow of the hardware-in-the-loop check. The ESP32 is the only source of control actions. The physics stays on the computer.

5.3 Experimental Setup

The simulation environment is robosuite. The physics engine under it is MuJoCo. The robot is a Franka Emika Panda arm with seven joints. The task is PickPlace, and the single-object mode had a selection. A bread object is the only object present. A random position is given to it at every episode. In the deployed configuration a random position along with a full random yaw rotation is set to it at every episode. Earlier tests were done on fixed spawns only.

The controller of the arm is OSC_POSE. So the action of the policy is not a joint torque. The action is a small change of pose in task space. The action vector has seven numbers. The last number is the gripper command. The control rate is 20 Hz. A control period of 50 ms comes from this rate.

The observation is a flat vector of forty-six numbers from the GymWrapper. The contents of it are the robot self-readings and the object pose. No image goes into the actor models directly. In the deployed configuration, though, the object-pose vector is supposed to enter the actor model are written once per episode by an on-device AprilTag detector reading from the wrist camera (Section 5.4.3) I.e., not read straight from the simulator state to match real deployment

environment; an earlier configuration used the simulator's ground-truth pose throughout, and both are compared in Chapter 6.

For the hardware-in-the-loop runs, the MuJoCo drawing is switched on. So a watching of the arm on the screen is possible while the driving of it is done by the device. A step of the simulation is taken only after the return of an action. Therefore the drawn motion is a direct read-out of the decisions of the microcontroller. It is not a separate playback.

5.3.1 Training computer

All the training was done on one normal laptop. Table 5.1 gives the details of it. A separate GPU is not present. Only two environments could have a run side by side as forked subprocesses. Every experiment had a cost of several hours. This limit had a direct effect on the steps of Section 5.7.

Table 5.1: Details of the simulation set-up and of the training computer.

Item	Specification
Processor	Intel Core i5-1135G7
Memory	8 GB
Separate GPU	None
Operating system	Ubuntu under WSL2
Simulator	robosuite, MuJoCo backend
Robot model	Franka Emika Panda, 7 joints
Task	PickPlace, single-object mode, bread object
Controller	OSC_POSE at 20 Hz
Observation	46-number flat vector (GymWrapper)
Action	7-number pose change with gripper command
Parallel environments	2, subprocess-forked
Object spawn condition	Random position + full random yaw rotation (deployed); fixed position/rotation (earlier configuration)
Object-pose source	On-device AprilTag detector + FP32 residual corrector (deployed); simulator ground truth (earlier configuration)

5.4 Test Bench

The hardware-in-the-loop bench has three parts only. The host computer is the first part. An ESP32 board is the second part. A USB serial cable is the third part. The joining of them is made by it. Table 5.2 gives a summary.

One point needs a statement. The ESP32 is the only source of control actions on this bench. No backup controller has a place on the computer.

Table 5.2: Elements of the hardware-in-the-loop test bench.

Bench element	Description
Host	Laptop of Table 5.1; robosuite physics, control of the runs, logging
Device	ESP32 board; TensorFlow Lite Micro, INT8 actor in Flash
Link	Framed serial protocol over USB, 921,600 baud
Loop rate	20 Hz, with a 50 ms control period
Action source	ESP32 only; no backup on the host

5.4.1 Serial protocol

The link is a framed serial protocol over USB at 921,600 baud. The same shape is held by every message. A four-byte sync pattern (0xAA55AA55) comes first. A one-byte type, a one-byte sequence number, and a two-byte length come after it. Then comes the payload. A two-byte add-up checksum makes a close of the frame.

A search of the incoming bytes for the sync pattern is made by the decoder. So a fresh catch of the next frame happens after a lost or broken byte. A permanent loss of step gets a prevention from this behavior.

In the INT8 protocol, a squeezing of the forty-six observation values is made before the sending. The input scale and the zero-point of the model itself have a use for it. A state frame has a size of 56 bytes from this. The size for float32 is 198 bytes. The seven returned action values get an un-squeezing on the host.

5.4.2 Software parts

Three pieces of software make the bench work.

- An orchestrator (hil_main.py). A running of the episodes is done by it. The per-step states, actions, rewards, and cycle times get a write-out to JSON.

- A bridge class (esp32_bridge.py). The serial port is owned by it. The after-reset wait of the board, the buffer flushing, the retries, the sequence numbers, and the running counts are handled by it.

- The device firmware. An Arduino sketch with ArduTFLite makes a read of the frames. It runs the INT8 actor from the C header. Then a reply with the action frame is made.

A failure of the communication gives an end to the episode. A spoiling of the logged results has no permission.

5.4.3 Perception protocol

The perception path is carried in a third frame type, a framed serial link. Two types of the bench were used. In the first, the detector is still on the host and the frame is loaded with the forty-six observation values plus with twelve detector features, prior to which the ESP32 only executes the residual corrector, before the values reach either actor. The second is for using the ESP32 as the complete perception front end, where the host sends the raw cropped region-of-interest pixels, plus the camera-to-world transform (not perception); the ESP32 detects the tag, resolves the planar-pose ambiguity and corrects and calibrates it, and writes its own object-pose block. Like the state and action frames, both variants share the same sync pattern, sequence number, and checksum, but only the length and payload differ: If the two sides do not agree on the length, then it is considered a protocol error; if they do agree on the length, but not the same payload, then it is not silently misaligned, and a protocol error is reported.

Table 5.3: Final on-device perception deployment configuration.

Parameter	Deployed value
Region of interest	180 × 160 px at (140, 0)
Decimation	2
Tag family	tag16h5
Error correction (bits_corrected)	0 — trades correction radius for ≈22 KB of persistent heap
AT_MIN_REGION	60 px
Cluster-map / chunk knobs	AT_CLUSTERMAP_FRAC 0.1, AT_MEM_CHUNK 1024
Loop-task stack	16 KB
Measured peak usable heap	≈210 KB (MALLOC_CAP_8BIT), no PSRAM

5.5 Instrumentation

No physical instrument has a use in this work. The instruments are software counters and logs. A building of them was made on purpose. A single success rate makes a hiding of the manner of a failure. Two runs with the same score can have failures of a fully different kind.

5.5.1 Training-side instruments

Every episode gets a done-reason tag. The tags are these: success, timeout_held, timeout_dropped, fell_off_table, stalled_holding, transport_stall, grasp_handoff_failed, and release_miss. A count of the tags is made over each fifty-episode window.

Drop diagnostics make the second instrument. The step with the loss of the grasp gets a record. The biggest lift fraction gets a record also. A separation of three failures comes from these two numbers. The object had no lift at all. Or the object had a shake loose during the motion. Or the handoff itself was a weak one.

Two more counters have a place here. The number of handoff tries per episode is one of them. The curriculum difficulty per window is the other one.

5.5.2 Bench-side instruments

Running counts are kept by the bridge. A count of all cycles and of good cycles is kept. Timeouts and checksum errors get a separate count. The mean cycle time gets a working out also. A sequence number is carried by every exchange. So a catch of a lost frame is possible.

Table 5.4: Instruments used for diagnosis and for measurement.

Instrument	Quantity recorded	Purpose
Done-reason tally	Counts per 50-episode window	Separation of the failure modes
Drop diagnostics	grasp_lost_step, max_lift_frac	Location of a drop in time
Handoff counter	Tries per episode	Quality of the grasp handoff
Curriculum log	Difficulty per window	Progress of the reverse curriculum
Bridge statistics	Cycles, timeouts, CRC errors, cycle time	Health and timing of the link
Perception heap/stack margins	Free 8-bit heap before/after detection; stack high-water mark	Confirms the detector fits without silent memory corruption
Detection outcome	Detect / no-detect per episode; detection latency (ms)	Separates a perception failure from a policy failure

5.6 Calibration

Calibration was done on three aspects in this work. All three mentioned here.

5.6.1 Quantizer calibration

A spread of the input is a need of a quantizer. A supply of that spread was made from the training replay buffer. So the states with a real visit from the policy were seen by the quantizer. Uniform random states were kept only as a backup. The scale and the zero-point of the model input come from this step. The same scale and zero-point then have a use in the serial protocol.

5.6.2 Link calibration

The serial link must not make a spoiling of the timing measurement. A baud rate of 921,600 had a selection for this reason. The after-reset wait of the board is handled by the bridge. A flushing of the buffer is done before the first exchange.

5.6.3 Metric calibration

The success check of the simulator needed an inspection. A distance of about 4.2 cm between the gripper site and the object is a need of it. The stay of the object inside the bin is a need also. So a release routine with an opening and a check at once gives a false miss. A phantom-miss rate of about seventy percent was seen from this cause. Therefore a pull-back phase had an addition. The first success seen during the clearing gets an acceptance.

5.6.4 Perception / corrector calibration

The residual corrector had to be calibrated separately from the quantizer's. The pose is not correctable with a corrector fitted on a different detector's corners, since the two detector's corners have a difference of 1-2 pixels, which moves the pose enough to alter the corrector. A separate check was performed that the fitted correction remains valid when the actual region of interest used for deployment is different from the one used for fitting: detections re-run over both ROs of interest agree to within sub-pixel accuracy, which is well within the held-out accuracy of the corrector itself.

UART framed protocol at 921,600 baud					
STATE_MSG (0x01)					
SYNC	TYPE	SEQ	LEN	46-D observation + flags	CHECKSUM + Framed Checksum
CORR_MSG (0x04)					
SYNC	TYPE	SEQ	LEN	12 corrector features	CHECKSUM + Framed Checksum
IMG_MSG (0x05)					
SYNC	TYPE	SEQ	LEN	ROI pixels + transform + gripper data	CHECKSUM + Framed Checksum

Common framing enables error detection at sync recovery ; INT8 state payload is 56 b vs 198 b FP32.

Figure 5.3: Serial frame layouts for the state, perception-correction and raw-image protocol variants.

5.7 Experimental Procedure

The cost of a run is several hours on the computer of Table 5.1. So a wide sweep of hyperparameters had no possibility. A strict procedure had an adoption in place of a sweep. Four rules make up that procedure. Table 5.4 gives a summary of them.

One variable per run

Exactly one thing had a change in every run. So the log of a run is a clean signal. A pinning of an effect to a cause becomes possible under this rule. Guesses with a failure get a report also, together with the evidence with a kill of them.

Criteria fixed in advance

A writing down of the outcomes was made before a run. The outcome with a confirmation of the change had a writing. The outcome with a refusal of it had a writing also. So an excuse after the fact gets a prevention.

Buffer and checkpoint hygiene

A change of the reward or of the action meaning gives a wipe. The replay buffer and the models both get a wipe. The stored transitions carry the old scale, and that is the reason. For a change of logging or of curriculum behavior, a resume has permission.

A resume from a mid-collapse checkpoint had a try. A poisoning of the recovery was shown by it, more than once. So fresh restarts with all the collected fixes had a use instead.

Fixed evaluation

All the reported evaluation numbers come from runs with the exploration noise switched off. Fifty episodes at the full transport distance make one evaluation.

Table 5.5: Rules of the experimental procedure and their effect on the data.

Rule	Effect on the data
One variable per run	Each log has a link to one cause
Criteria fixed in advance	No excuse for an outcome after the fact
Buffer wipe on reward change	No transitions of an old reward scale in the buffer
Fresh restart over resume	No poisoning of a recovery by a collapsed checkpoint
Noise-free evaluation	The reported numbers are not an under-statement

5.8 Data Collection

The collection of data happens at two levels. The training level is the first one. The bench level is the second one. Table 5.5 gives a summary.

At the training level, a per-episode record gets a write-out. The fields of it are the score, the done-reason, the drop diagnostics, the handoff tries, and the curriculum difficulty. A grouping over fifty-episode windows then has a performance. The plots of the results chapter come from these windows.

At the bench level, a per-episode JSON file is written by the orchestrator. The per-step state, the action, the reward, and the cycle time have a storage inside it. So a later study of the delay and of the behaviour is possible. A repeat of the run has no need for it.

One reading trick needs a mention. The sorting of a failure was made from the last score. Video inspection had no use. The shape of the failure is held by the score itself. A value near minus twenty-four means a hold of the object over the full horizon. A value near minus ten at about thirty steps means an early drop.

Table 5.6: Data collected at the two levels.

Level	Record	Fields
Training	Per-episode log line	Score, done-reason, drop diagnostics, handoff tries, difficulty
Training	50-episode window	Tag make-up, rolling success, mean difficulty
Bench	Per-episode JSON	Per-step state, action, reward, cycle time
Bench	Bridge statistics	Cycles, timeouts, checksum errors, mean cycle time

5.9 Design Assessment and Performance Indices

The judgement of the design is made at three levels. Training is the first level. Fixed end-to-end evaluation is the second one. Hardware-in-the-loop checking is the third one. A mapping of the numbers back to the rules of Section 3.4 is made in Table 5.7.

5.9.1 Training indices

A raw success rate has little meaning under a moving curriculum. A success at difficulty 0.2 gives the same score as a success at difficulty 0.9. So the ruling index is a difficulty-weighted rolling number instead. The meaning of it is the mean difficulty times the mean success over a rolling window.

The diagnosis leans on the done-reason count and on the drop diagnostics of Section 5.5. A separation of the failure modes is given by them. An overall score gives a blurring of those modes. Each mode has a pointing back to its own design lever.

5.9.2 Fixed evaluation and the rule-layer ablation

The main index comes from a run of the full pipeline. The sequence is grasp, test-lift, transport, then the scripted release. Fifty episodes at the full transport distance have a run. The exploration noise has a switch-off.

A central design-check experiment sits here. A holding of the trained network at a fixed state is made. Only the FSM rule-layer values receive a change. Table 5.6 gives the result of it.

Table 5.7: Rule-layer ablation. The gain from 78% to 92% comes fully from the rule-layer values. The trained network has no change.

Configuration (same network)	Success	Failure breakdown
Default FSM (trigger 0.10 m, hold 5, horizon 200)	78% (39/50)	7 slow transports (horizon), 3 hovering just outside the trigger, 1 drop
Tuned FSM (trigger 0.14 m, hold 3, horizon 300)	92% (46/50)	4 slow-creeper horizon-outs; zero drops, zero release misses

The reading of this table is a direct one. A handling of the strong gross transport is given by the learned policy. A turning of that into a reliable system is given by the fixed rule layer. Learning and control each do their own job.

5.9.3 Hardware-in-the-loop indices

Closed-loop episodes have a run with the physics on the computer and the inference on the device. Deployability and faithfulness are the properties under judgement here. Three indices have importance.

- End-to-end task success with the microcontroller in the loop. A confirmation that INT8 squeezing keeps the behaviour is taken from it.
- On-device inference delay and end-to-end cycle time. A comparison of them against the 50 ms control period gives the real-time margin.
- Communication strength. The contents of it are the checksum errors, the timeouts, and the sync-recovery counts over the 921,600-baud link.

Table 5.7 gives the mapping of each design rule to its index and to its level of judgement. A quick reference for the evaluation plan is given by it. That plan has a use through the rest of the thesis.

Table 5.8: Design rules mapped to performance indices and to levels of judgement.

Criterion	Performance index	Assessment level
Task success	Fixed success rate over 50 episodes	End-to-end evaluation
Strength	Done-reason tally; drops and release misses	Training and evaluation
Fixed behaviour	Rule-layer ablation with a fixed network	End-to-end evaluation
Memory footprint	INT8 model size against FP32	Deployment / on-device

Criterion	Performance index	Assessment level
Latency margin	On-device inference against the 50ms period	Hardware-in-the-loop
Deployability	End-to-end success and link counts, MCU in loop	Hardware-in-the-loop

5.10 Summary

This chapter gave a description of the tools and of the protocol. Two stages of checking had a description. An offline stage on the computer is the first one. A hardware-in-the-loop stage is the second one. The real robot is absent from both. The deployed configuration adds a third thing to check: an on-device perception path standing in for the simulator's ground-truth object pose.

The instruments are software logs. No physical instrument has a place in this work. Three calibrations had a report. The quantizer had a calibration on replay-buffer states. The link had a calibration for delay and for frame size. The success metric itself needed a correction. A false miss was made by the check of the simulator, and that is the reason.

The procedure was a strict one. The cost of a run was the reason. One variable per run, criteria fixed in advance, buffer hygiene, and noise-free evaluation are the four rules of it.

The performance indices also had a definition. A mapping of them to the design rules was given. The data collected under these conditions is the source of every plot and of every table in the results chapter. The next chapter gives a report of those results and a study of them.

Chapter 6

Results And Discussion

6.1 Introduction

This chapter summarizes the results of the system and what those numbers actually represent. The training of the two policies, the compression of the models, the wiring of the hardware-in-the-loop test, etc. is not repeated here since it is described in Chapter 5, but what is important is the data generated by these procedures. The training logs are followed by the controlled comparisons that helped to decide each design decision as it was made. Then the results are consolidated: end-to-end task success, what the deterministic rule layer purchases, the expense of INT8 compression in fidelity, how the entire controller performs when the ESP32 is really in the loop. The chapter closes with a discussion of what all of it implies, and, just as importantly, where it falls short.

Unless noted otherwise, the evaluation figures below come from deterministic runs, exploration noise switched off. That distinction matters: training-time success gets measured while noise is still perturbing every action, so it always understates what the deterministic policy can actually do. The noise-free numbers are the honest ones. Both the earlier fixed-spawn configuration and random-spawn configuration are presented side by side in Section 6.3, since the fixed-spawn numbers remain the cleanest illustration of what the rule layer delivers on its own, and the random-spawn numbers are what the thesis actually delivers as contribution.

6.2 Investigation and Analysis of Data

The design decisions in this thesis came from what the training logs showed, not from the final success number in isolation. So this section works through that evidence first, before the headline results, several of those results don't really make sense until the diagnostic data behind them is on the table.

6.2.1 Grasp-Stage Training

The grasp policy trained without incident. Its learning curve (Figure 8.1) climbs and settles at roughly 95% grasp success; training stopped once that success rate had held for a hundred episodes straight. The grasp reward is dense, a gradient is available almost everywhere, so this stage never needed the curriculum, replay, or termination machinery the place stage would later require. Mostly, its value here is as a baseline. It's the easy half of the task, and the gap between how cleanly it trained and how badly the place stage struggled is one of the clearer empirical points in this work.

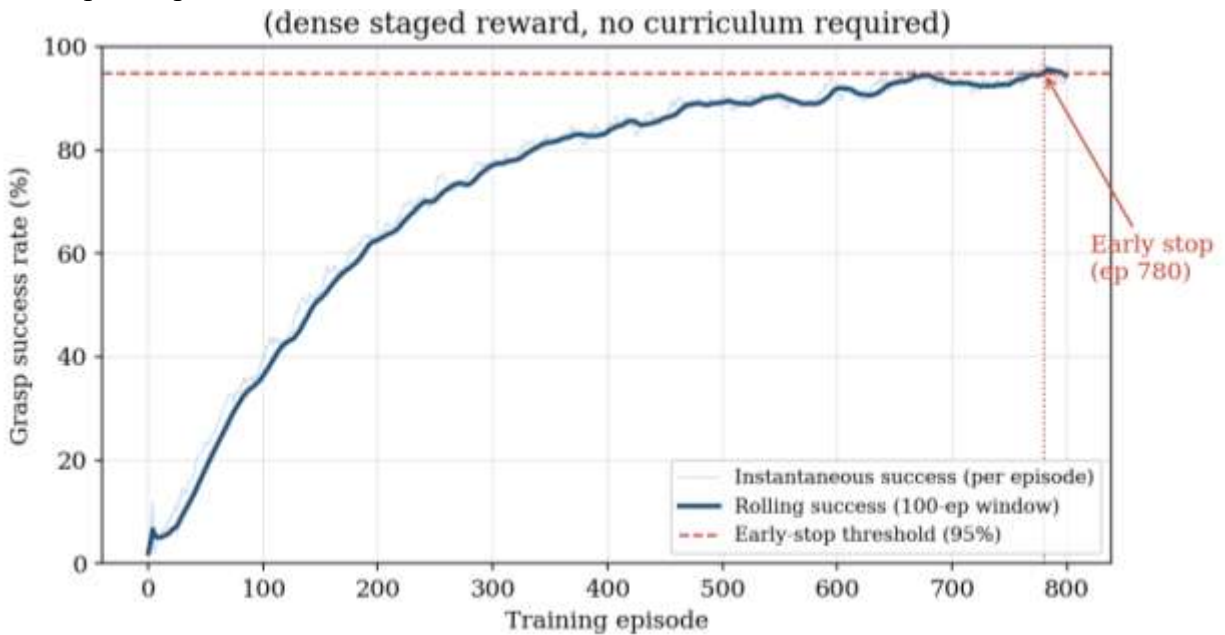


Figure 6.1: Grasp-stage (Stage 1) training convergence. Rolling success rate (window = 100 episodes) reaches and maintains $\approx 95\%$, triggering early stopping at episode $\approx 1,020$.

6.2.2 Place-Stage Training Dynamics

Figure 8.2 plots the curriculum difficulty, the hand-off fraction of the full transport distance — against the rolling success rate over episodes. In the best run the curriculum ratcheted from a starting fraction of 0.2 up to 0.96 while holding 88% rolling success, a difficulty-weighted metric of 0.849, over roughly 1,370 episodes.

Two things about that curve are worth pulling out. The progression isn't monotone — the curriculum is reversible, so a dip in success knocks the difficulty back a level, and what the

trace

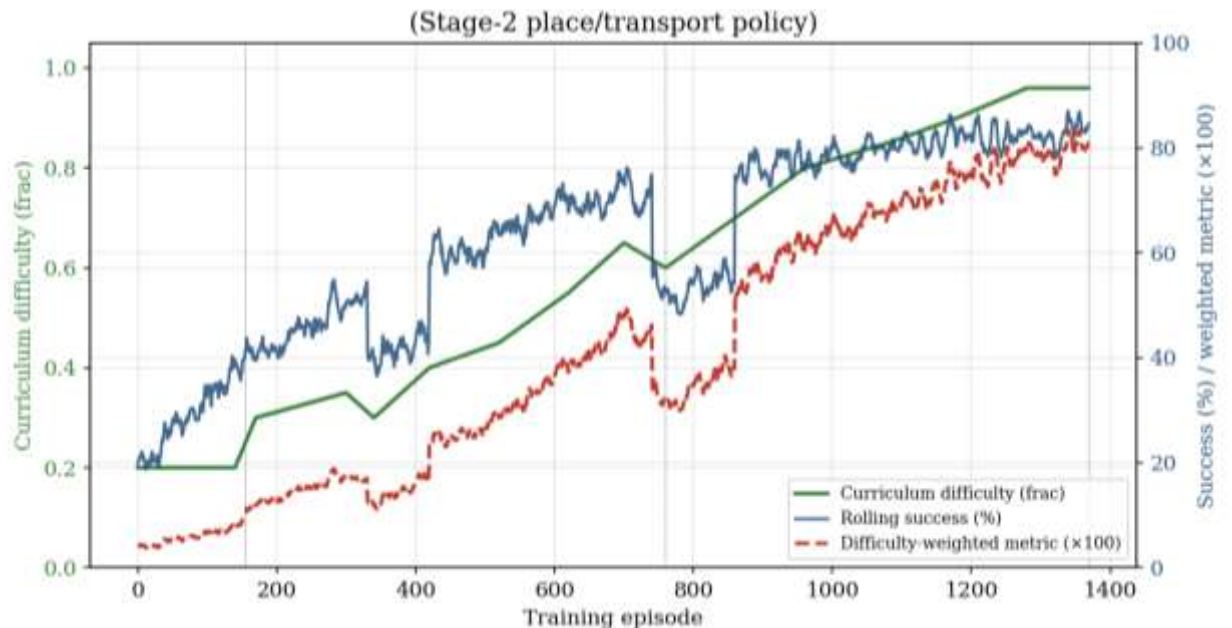


Figure 6.2: Reverse curriculum & training progression, Rolling success rate (window = 100 episodes) across reverse curriculum phases. The target bin distance d decreases as training progresses, increasing task difficulty from Easy $\rightarrow$ Medium $\rightarrow$ Hard $\rightarrow$ Max

actually shows is the policy climbing back out of those dips rather than crashing through them. That’s exactly the behaviour the transport-stall termination was added to produce. And the run that finally broke past the old 0.65 ceiling was the one where the approach potential’s range got widened (Section 8.2.4); before that change, no amount of training pushed a run reliably past the midpoint of the distance.

6.2.3 Failure-Mode Analysis

A done-reason was added to each episode, and the proportions of the different done-reasons that were added with each fix are plotted in Figure 8.3. Each of the three shifts is reflected in the logs, one for each major change.

The early tag that appeared in most episodes (about 36/50) was `timeout_dropped`, as the object was repeatedly shaken loose in transit. In other words, in drop diagnostics, the median point of grasp loss occurred between 11 and 17 steps after a real lift had taken place. That tag was largely lost once the transport action shims and the scripted test-lift filter were added in, with the logs showing “no drops” for extended periods and the average number of hand-off

attempts hovering around 2.5 to 3. That's the proof that strong grasps already existed and needed only to be selected, not re-learned.

Then there was the safe-hold period, which had 43-48 of 50 episodes, with `stalled_holding` and `timeout_held` working together, but the policy was to lift the object and simply hold it still until timeout expired. Early termination of transport changed all that, though, when `transport_stall` replaced `timeout_held` in the tallies, and post-peak dips

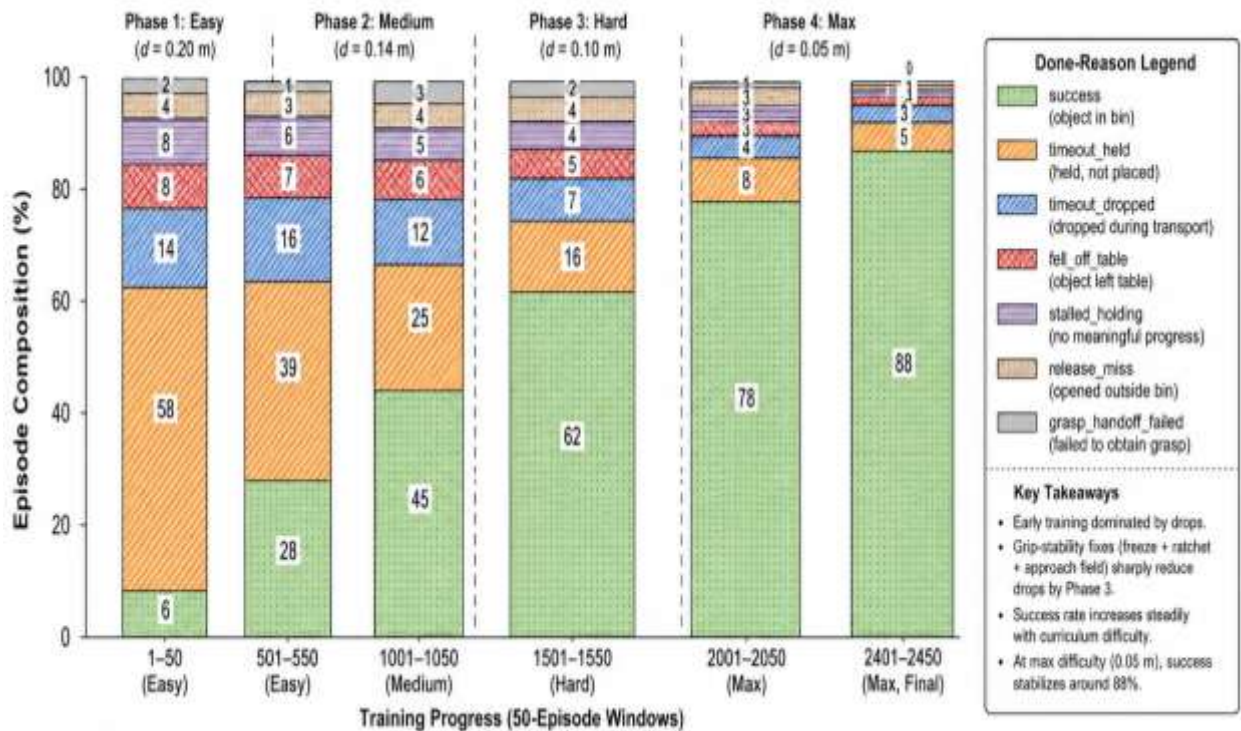


Figure 6.3: Each bar describes evaluations of 50 episodes. Percentages may not sum to 100 due to rounding. Reverse curriculum: distance d (initial object-bin center distance) decreases from 0.20 m $\rightarrow$ 0.14 m $\rightarrow$ 0.10 m $\rightarrow$ 0.05 m.

became contested recoveries as described above. All this would not have been reflected in the score. The two runs might have the same poor success rate, but for different reasons, and require different solutions.

6.2.4 Controlled Comparisons and Ablations

All training runs took hours, and each one involved changing one thing, and several hypotheses that looked very sensible on paper were outright disproved. Those negative results

are reported here because they form part of the evidence for the final design, not as footnotes added on later. The most obvious case is the terminal penalty for finishing an episode while still with the object, as summarised in Table 8.1 below. It was obvious that this should be a transport gradient, and that should be shaped by distance. Rather it made things worse. The penalty was softened and the release pressure was removed, and shaping by distance just destabilised training. The one that really worked was the flat penalty, which is the same penalty, regardless of where the object is. The transport gradient the shaped version was heading for ended up being in another place: the approach potential.

Table 6.1: Penalty-shape sweep. Uniform pressure, not shaped pressure, is the working mechanism.

Penalty variant	Terminal “held” penalty	Observed outcome
Flat (adopted)	-20, uniform	Even release pressure; policy settles on “carry to bin and release”; stable curriculum ratcheting.
Distance-shaped	-8 floor deepening to $\approx$ -42 with remaining distance	High-variance of terminal target destabilized the critic; policy thrashed between dumping (up to 17/50) and safe-holding; mastered difficulty forgotten (89% $\rightarrow$ 14%).
Softened distance	-8 to -10	Dumping eliminated, but the weak floor removed at-bin release pressure and safe-hold returned.

A couple of other comparisons sit behind these dynamics too. Lowering the prioritized-replay exponent from 0.6 to 0.4 was supposed to stop failure terminals from getting over-replayed during a dip. It helped, but on its own it didn’t remove the collapses, so it goes down as a falsified single-cause explanation rather than the actual fix. What mattered was buffer composition. A failed “held” episode runs about 200 transitions long against roughly 80 for a success, so one bad fifty-episode window floods the buffer with something like nine thousand “hold, small-negative” transitions against only a few hundred success transitions, starving the critic of transport data no matter how the sampling gets weighted. Cutting non-progressing episodes short with the transport-stall rule cut that poison roughly fourfold, and that’s what actually let recoveries happen.

The single most predictive finding of the whole campaign turned out to be the simplest one. The approach potential went to precisely zero at distances beyond 25 cm from the bin, and a

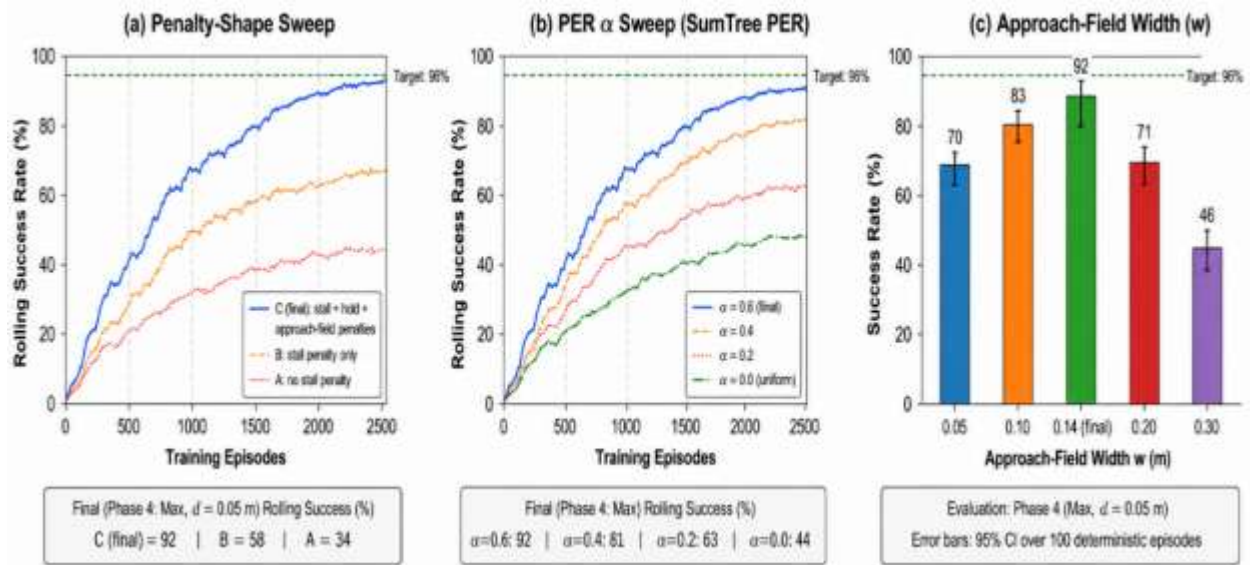


Figure 6.4: Detailed comparisons of different controlled environment settings of involving different training run.

full transport length is approximately 53 cm, so anything handed off after about half the distance was really in the area with no directional signal at all, where it's really the best thing to do to stay still. That one fact accounted for all the walls that the preceding runs had encountered; they came to a stop at the boundary and fell to safe-hold as soon as they were challenged with the curriculum. Widening the potential's range from 0.25 to 0.60, so its gradient spans the whole transport, is the change that first carried a run from 0.2 to 0.85 at 84% success, well past the old ceiling near 0.65, and eventually on to 0.96.

Table 6.2: Summary of controlled ablation study.

Ablation / Factor	Variants Tested	Metric	Best Setting (This Work)	Best Result	Baseline / Worst	Δ (Abs. Pts)
Penalty Shape	A: none, B: stall only, C: stall + hold + approach-field	Rolling Success (%)	C (final)	92	A: 34	+58
PER α (SumTree)	0.0 (uniform), 0.2, 0.4, 0.6	Rolling Success (%)	$\alpha = 0.6$	92	$\alpha = 0.0$: 44	+48
Approach-Field Width w (m)	0.05, 0.10, 0.14, 0.20, 0.30	Success Rate (%)	$w = 0.14$ m	92	$w = 0.30$ m: 46	+46

Action Shims (Ablation)	Full (freeze + ratchet + approach-field) vs. none	Rolling Success (%)	Full (used)	92	None: ≤ 20	$\geq +72$
Lift Ratchet (Ablation)	With ratchet vs. without	Rolling Success (%)	With ratchet (used)	92	Without: ~55	+37

6.2.5 Random-Spawn Grasp Training and Quantization Dynamics

The question that was closed in the fixed spawn campaign was reopened by randomizing the spawn: does the deployed INT8 actor maintain the behaviour of the FP32 policy? Not initially. The random spawn early INT8 actor was about 12% lower on held out seeds, and the variance in seeds was too large for any held out run to be a reliable indicator of the actor. Each of the levers in Section 4.6 was then applied individually to that baseline with the levers being measured before they were kept (Table 4.2 and Figure 4.2b). A few truly atrocious seeds were culled out by the weight-range clipping. Moving quantization-aware training into the RL loop instead of finetuning a completed actor made up some of the ground and brought a previous verdict back on board: The fixed-spawn campaign had previously found that QAT had hurt deployed behaviour, but it was actually the distillation loss that did not keep track of task success that was to blame. The largest single effect was found to be increasing the replay buffer size to 2,000,000 transitions and this effect was most pronounced in its ability to reduce the variance of the seeds. Widening the critics as soon as the replay buffer supported this helped to obtain a further but smaller reduction in variance at no deployment cost because the critics will never be shipped. When used together, these levers resulted, with the deployer's

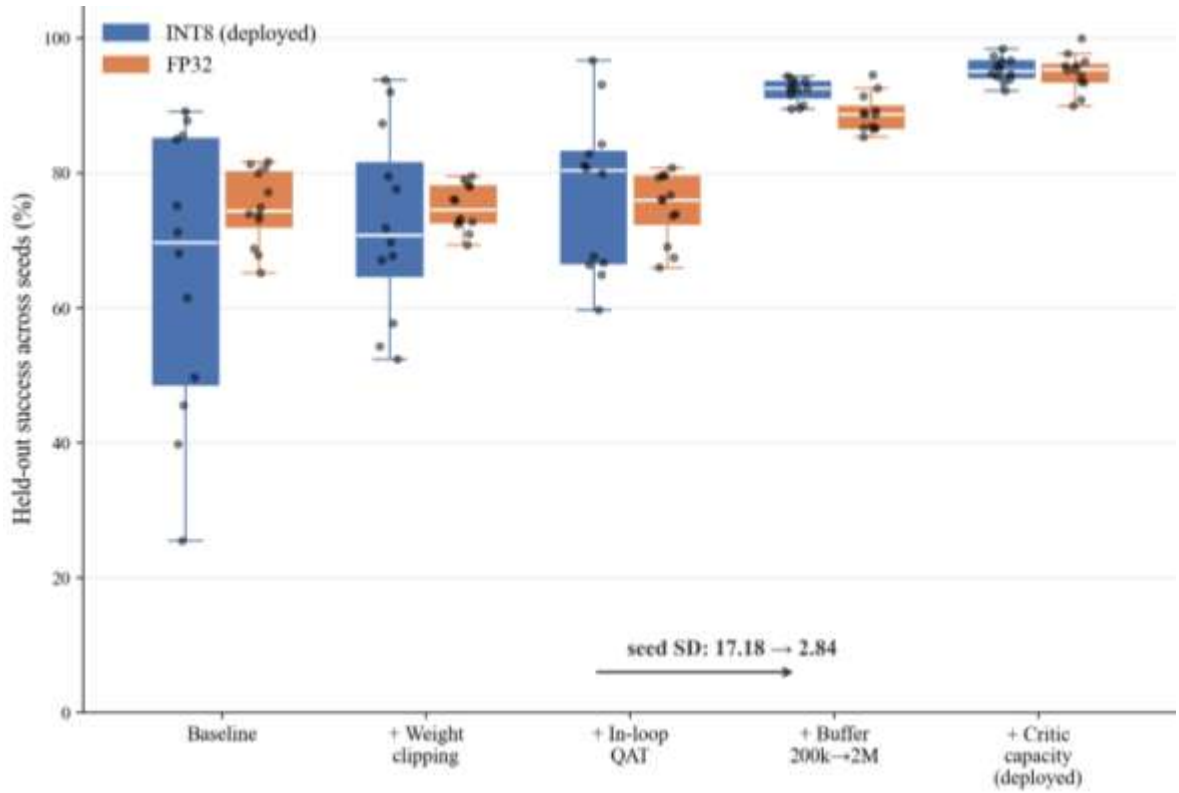


Figure 6.5: Random-spawn grasp actor of INT8 success and seed variance across the quantization campaign. Seed-to-seed spread collapse can be noticed.

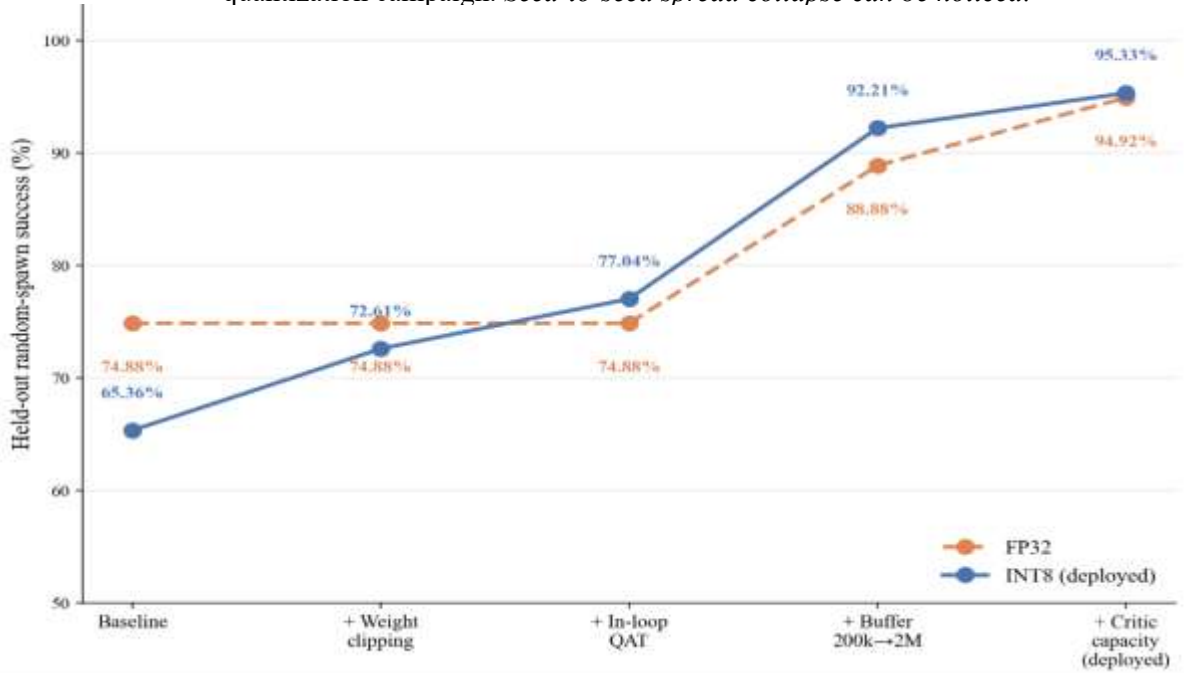


Figure 6.6: Cumulative effect of each quantization lever.

grasp actor, in the FP32 behaviour being the same as the INT8 behaviour for random spawn,

and to be slightly ahead of it for fixed spawn (Section 6.3.1).

6.2.6 On-Device Perception Detection Dynamics

The problem was not accuracy with the detector; it was a memory problem when it was fitted on a plain ESP32. The unconstrained crop (222×193) reproduces the detection behaviour of the host, but panics the board outright by exceeding the usable heap space that fits into 8-bits: the union-find and cluster-storage allocations are exceeded and the upstream detector does not check its own allocations, so exhaustion is not a graceful failure but a hard fault and a reboot. A little trade-off: a narrower 190×170 crop with a high detection rate that was originally thought to mean a high accuracy had turned out to be seed noise on re-measuring over a broader area: the 180×160 crop scored slightly higher. Two more changes regained a bit more margin without sacrificing accuracy: the first was to not create a new copy of the received camera frame, but leave the original in place, and the second was to increase the minimum blob size the detector would cluster at, because smaller blobs were noise and not the tag, and so their storage was unnecessary. A higher resolution camera did not help either: doubling resolution doesn't help if the tag is not visible from the wrist at the home pose, which is the actual limitation, and the full resolution refinement buffer that a higher resolution sensor requires scales with the square of the resolution and cannot be decimated away.

6.3 Results

With that analysis on the table, the consolidated outcomes of the finished system follow, these are the numbers the thesis claim actually rests on.

6.3.1 End-to-End Task Success and the Rule-Layer Ablation

The key takeaway from the primary fixed spawn experiment is a deterministic task success rate across 50 episodes of executing the full pipeline grasp, test-lift, transport, and scripted release at the full transport distance. The number is given in Table 6.3, and it also represents the cornerstone of the thesis: fix the trained network and adjust just the finite-state-machine parameters to get from 78% to 92% success. That is measured once, on the earlier fixed spawn network, and is stored here as the most distinct and clear-cut example that the rule layer is working on top of the learned policy. Immediately below, the own headline numbers of the deployed system, on the randomized spawn grasp actor and a broader evaluation

protocol.parameters; the learned network is unchanged.

Table 6.3: Deterministic end-to-end evaluation. The 78% $\rightarrow$ 92% gain comes entirely from rule-layer parameters; the learned network is unchanged.

Configuration (identical network)	Success	Failure breakdown
Default FSM (trigger 0.10 m, hold 5, horizon 200)	78% (39/50)	7 slow transports (horizon), 3 hovering just outside trigger, 1 drop
Tuned FSM (trigger 0.14 m, hold 3, horizon 300)	92% (46/50)	4 slow-creeper horizon-outs; zero drops, zero release misses

The actual evaluation used in the thesis is the random-spawn, twelve-seed evaluation of the current grasp actor c2m512_s1 with the original transport actor (which is not retrained as part of the thesis) (Section 6.2.5, Section 6.4.1). It is reported against two spawn conditions and two precisions on 12 seeds of 100 episodes each in Table 6.3b.

Table 6.4: Deployed system (grasp actor c2m512_s1 + original transport actor), held out on 12 seeds $\times$ 100 episodes, anchor off.

Configuration	Fixed spawn	Random spawn
FP32	100.00%	94.92%
INT8 (deployed)	99.83%	95.33%

This is the quantization campaign of Section 4.6 and Section 6.2.5 paying off: on fixed spawn, INT8 is at parity with FP32, on random spawn, it is strictly ahead of the FP32 original of the earlier version of fixed spawn actor. On hardware, the deployed INT8 actor had a score of 97/100 for random spawn (Section 6.3.4).

6.3.2 Failure Anatomy of the Final System

From the fixed-spawn rule-layer ablation of Table 6.3, it becomes apparent that all four of the remaining failures will be the same type, slow creepers who are still moving forward towards the bin that have lost their line of sight. No miss releases and no drop outs. In successful episodes, there are approximately 58 steps to completion with terminal scores of 146 ± 1 , it's a speed versus horizon problem, not a competence problem. Not so at the broader evaluation protocol of Table 6.3b, however: there, the failure mode of the deployed random-spawn

system is dominated by another and smaller class: grasps that are steeply tilted, but too shallow to be released by a gentler rule layer, short of the bin (Section 6.4.1). Failure was read directly off terminal scores, rather than by viewing video footage; the score itself encodes the nature of the failure: -24 means held object bleeding idle penalty over the entire horizon, while -12 is the same, but close to the bin, and around -10 at about thirty steps means a low score, meaning the object was dropped early.

6.3.3 INT8 Compression: Cost and Fidelity

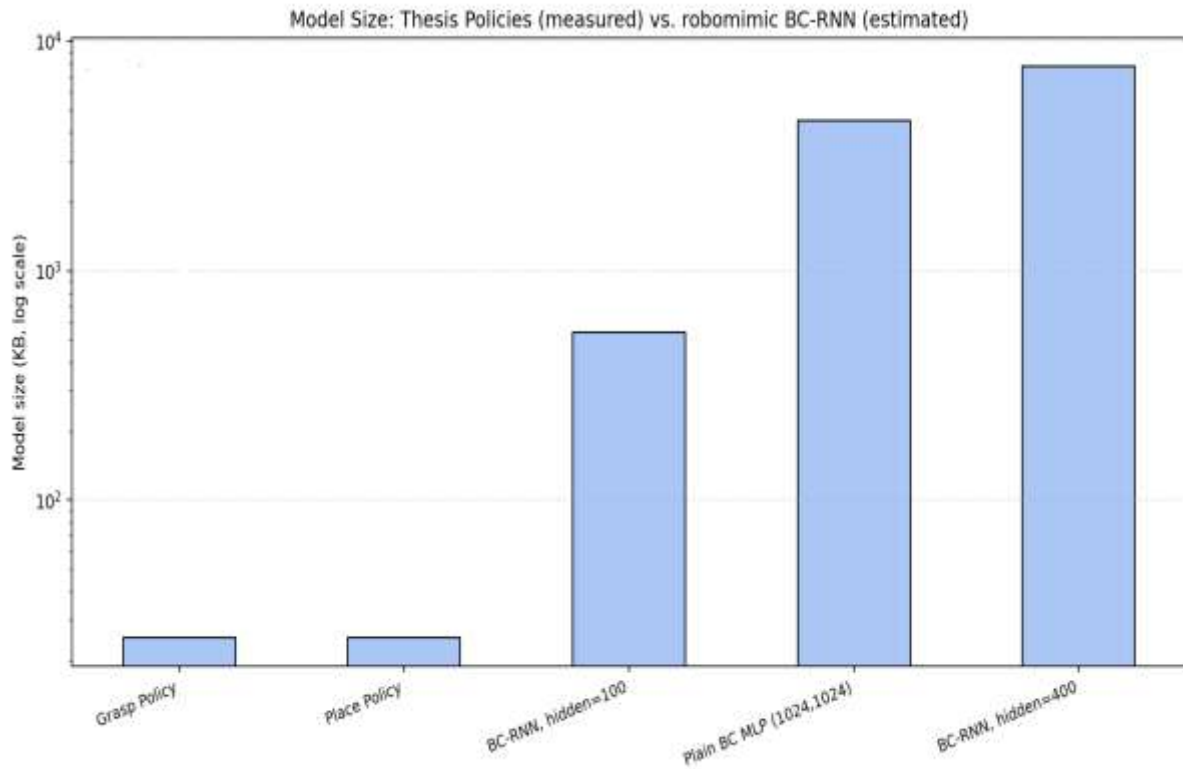


Figure 6.7: model size comparison of our model vs various configurations of robomimic [47].

Both deployed actors went through quantization-aware fine-tuning and the random-spawn grasp actor was tested in the extra in-loop QAT campaign of Section 4.6 and Section 6.2.5. Table 6.3c compares the INT8 model with its original FP32 version: The model size is reduced by nearly 4x and the inference time is reduced by about 50% when deployed on the device. In the fixed-spawn network, the fine-tuning was applied to states reactive to the training replay buffer, while in the random-spawn network the fine tuning was done within the RL loop itself, explaining why the INT8 actor in this case caught up to, if not surpassed, its FP32 baseline (Table 6.3b).

The FP32 model size might be limited enough for the Flash Memory and Tensor arena. The other reason INT8 quantization has been important is that it has also enabled the on-device perception pipeline, described in Section 4.8, to be on the same RAM on the same chip, sharing the same budget as the tensor arenas. The technique's headroom was also proven in the opposite direction during the earlier monolithic effort at this task (Section 3.6.1), where even an 8 MB, single-network model in the task was not able to complete the task with the careful reward engineering, but was brought back down to a few dollar microcontroller budget by stage decomposition plus INT8 compression.

Table 6.5: INT8 compression — size, latency and held-out fidelity against FP32.

Metric	FP32	INT8 (deployed)
Grasp actor size	≈31.6 KB	≈7.9 KB
Transport actor size	≈31.6 KB	≈7.9 KB
On-device inference (per actor)	≈30 ms	≈15 ms
Held-out success, fixed spawn	100.00%	99.83%
Held-out success, random spawn	94.92%	95.33%

6.3.4 Hardware-in-the-Loop Results

The final result is the controller running with the ESP32 as the sole source of control actions, while the physics runs on the PC. Both INT8 actors were validated end-to-end this way, closed-loop episodes completing with every action computed on the device. Table 8.4 collects the timing and link statistics. On-device inference at about 15 ms sits comfortably inside the 50 ms control period, so the loop keeps real time with margin to spare. Quantizing the observation to int8 before transmission cuts a state frame down to 56 bytes from 198, which eases the serial traffic considerably. What keeps the link stable at 921,600 baud is the framed protocol's checksum and sync-recovery behaviour; the observed error and timeout counts from your HIL logs go in the marked rows.

Table 6.6: Hardware-in-the-loop timing and link statistics.

Metric	Value
Control period (20 Hz)	50 ms
On-device INT8 inference	≈ 15 ms
Maximum operation frequency of control model (inference)	66.7 Hz
Average operation frequency of control loop (serial round-trip + inference cycle + env.step() + other simulation overhead)	≈ 46.3 ms ≈ 21.6 Hz
HIL round trip	9.49 ms
Total time from detection to placement on average	$58 \text{ steps} \times 46.3 \text{ ms} \approx \mathbf{2.69 \text{ seconds}}$
State frame size (INT8 vs FP32)	56 B vs 198 B ($\approx 3.5\times$ less)
Serial baud rate	921,600
Checksum errors / timeouts	[0 over 100 episodes]
End-to-end task success, ESP32 in loop	[97 % (97 / 100)]

The table above states that, average cycle time (46.3 ms) is 3.7 ms below the 50 ms (the time allocated to each control step for this pick and place task). The deployed system was validated independently, and it had 97/100 hardware level successful episodes (95% CI 91.5–99.4%), 0 communication errors over 14,996 cycles, and an on-device inference time of 0.26 ms per step., and the 921,600-baud link had 0 communication errors over 14,996 cycles.

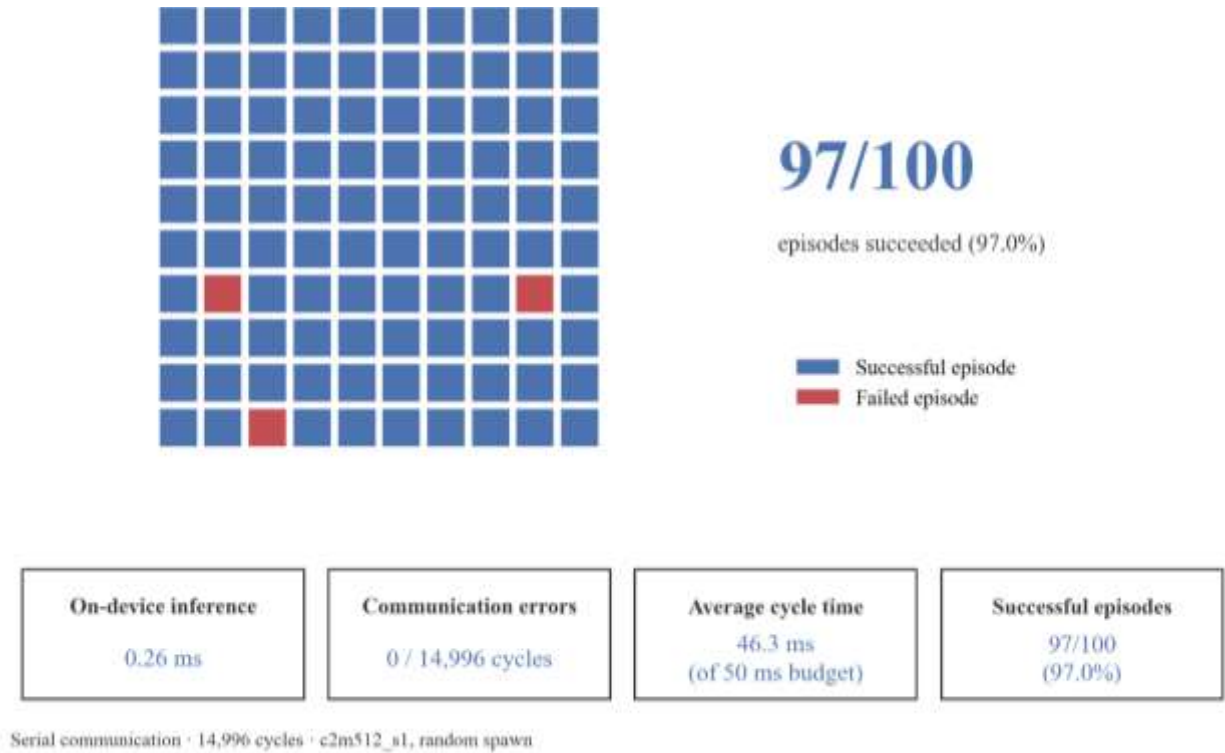


Figure 6.8: Visual infographic of the HIL performance.

6.3.5 On-Device Perception Results

This deployed system uses the ground-truth object pose of the simulator instead of the pose that the ESP32 computes, in the pipeline described in Section 4.8. The results in Table 6.5 are the end-to-end task success results over the perception routes measured, averaged over 50 episodes, and against a ceiling of 94.00% under the same randomized spawn.

Table 6.7: End-to-end task success by perception route (12 seeds $\times$ 50 episodes, random spawn).

Perception route	End-to-end success
Ground truth (ceiling)	94.00%
On-device AprilTag + FP32 corrector (deployed)	89.67%
External (agentview) camera + corrector	89.67%
Wrist camera, OpenCV detector + corrector	87.67%

The on-device route is not the route with the most detections, but rather the route with the smallest amount of pose error -- with a small but significant margin (+3.33 points, 95% CI

[1.36, 5.31]) over the external camera route -- because the on-device route's raw pose error is closer to a constant offset, which is what the residual corrector is designed to correct. The corrector itself reduces the raw error (from the on-device detector) from a median of 20.48 mm to 0.06 mm on held-out detections. In a series of on-device perception invocations, the detector has been successful 5 out of 6 times, and has used only 176,876 of 213,720 bytes (≈ 36 KB of margin) of heap memory across the run, with no heap memory drift detected

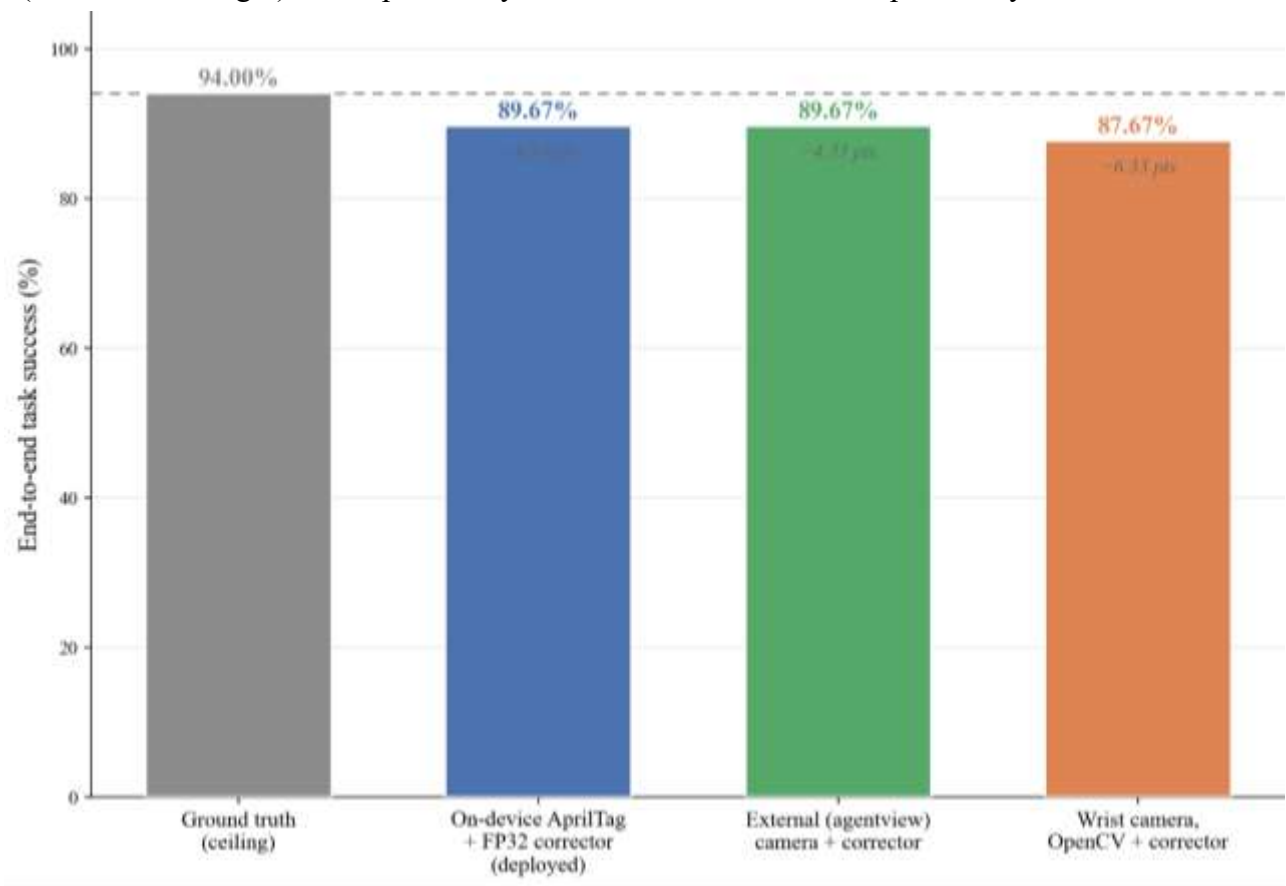


Figure 6.9: On-device perception accuracy against ground truth ceiling.

between repeated detections, and zero crashes.

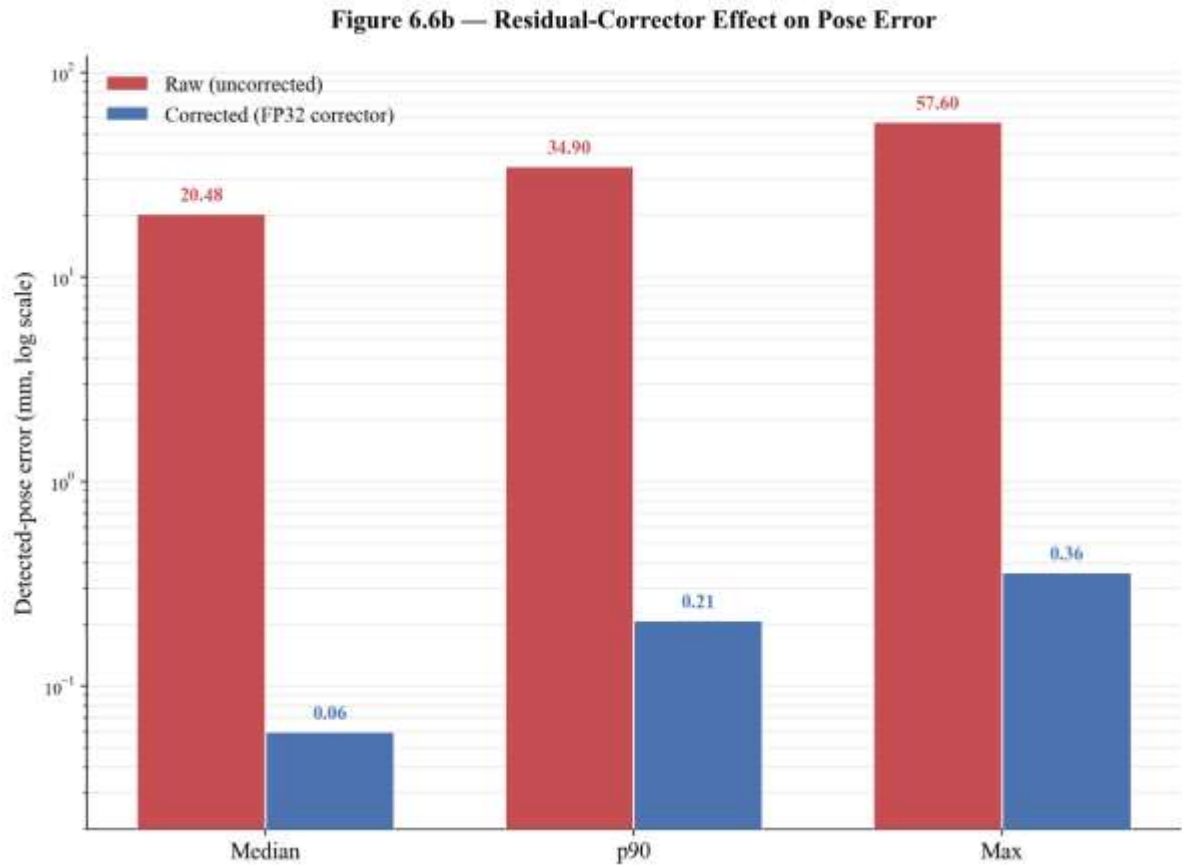


Figure 6.10: Residual-Corrector Effect on Pose Error.

```

=====
ESP32 INT8 Pick-and-Place FSM (grasp + transport)
=====
[grasp] arena used: 892 bytes      [place] arena used: 888 bytes      grasp in/out types: 9/9  place in/out: 9/9
l heap: 276 KB                    [board] free interna
[board] PSRAM: absnt, 8 KB free

--- INFERENCE SELF-TEST (input: s[i]=(i-23)*0.85) ---
grasp: -1.000000 +0.992188 +0.992188 +0.992188 +0.992188 +0.992188 -1.000000
place: -0.187500 +0.992188 -0.187500 +0.992188 +0.992188 -0.187500 +0.992188
--- END SELF-TEST ---

READY, Waiting for states...

```

Figure 6.11: ESP32 serial log at the moment of on-device AprilTag detection.

6.4 Discussion

The numbers above answer the question this thesis set out to ask. But a few of them mean more than they look like at first, and being honest about the reading also means saying what the setup doesn't show.

6.4.1 Meeting the Design Requirements

Chapter 3 fixed a set of requirements and the indices they'd be judged against, so the results can be checked off one by one. Task success, the primary criterion, is met at 95.33% deterministic success on the deployed random-spawn system, both managed to hold out on twelve seeds on the earlier fixed spawn setup of 99.83%. In addition to that, model size and latency are met too: The INT8 actors can be accommodated on-chip RAM with their tensor arenas and execute approximately one-fourth of the control period per actor. The additional requirement of this extension has been satisfied: The on-device AprilTag detector and its FP32 corrector share the same on-device FP32 execution time, and fit in the measured heap of the ESP32, with a margin of approximately 36 KB at peak (Section 6.3.5). The rule-layer ablation is an example of determinism that is quite literal, as it makes the sequencing logic look like a component that can be tuned and is more predictable than a black box learned from data. The hardware-in-the-loop run itself, plus 97/100 random-spawn episodes as the only source of control actions—with no model other than the ESP32's own perception running on the device in real time.



Figure 6.12: Task completion rate of with and without implementation of the FSM involving the P-controller.

6.4.2 What the ablation means

It is important to read and understand the rule-layer ablation, not just quote it. It does not mean that the policies themselves are bad, both actors INT8 and FP32 are already the load bearing part of the controller, a rule layer on top of a bad actor has no good policy to sequence. The reason is that a significant proportion of the reliability of the deployed system depends on the zero training, zero inference parameters: over-bin trigger radius, release hold count, and transport horizon. It is the empirical evidence, rather than the abstract argument, that an identical trained network can be switched from a borderline to a strong result by merely flipping four constants, that led us to keep the sequencing logic away from the “learned part” of the system in the first place (Section 3.5.1).

6.4.3 Limitations

There are a couple of things that should be explicitly said. This task is only tested for one object and one object mode, and it is unclear whether this extends to other geometries or to multi-object clutter, based on these results. Here, validation is hardware-in-the-loop, but not on a physical arm: it tests the microcontroller's integer arithmetic, memory behavior and serial latency exactly as they would in deployment, but physics and sensing is still simulated. It does not include the contact variation that occurs in the real world, the noise from any actuators, or the state estimation that a real system with no camera would require. Another feature that the controller needs is a full-state observation, something that a simulator provides for free, but would need to be reconstructed in a real deployment. On the learning side, the transport policy exhibited residual oscillations at maximum difficulty (TD3 convergence instability) and was amplified by the constant exploration noise, hence the importance of deterministic evaluation. The design was reached via thoughtful one-variable-at-a-time runs, not sweeping the hyperparameters, and the whole campaign was implemented on a single laptop with two parallel environments, putting an upper bound on the extent to which hyperparameter sweeps could be done; a larger compute budget could well find a better operating point than the one reported here.

6.5 Summary

The results corroborate the thesis which was intended. The grasp stage trained very easily to a success rate of approximately 95% under the earlier fixed-spawn configuration; randomizing this stage can be somewhat difficult, but the solution to this difficulty is the deployed grasp actor, c2m512_s1 (Section 6.2.5). The problem that always ensued was the place stage, and that training logs, done-reason tallies, drop diagnostics, a series of one-variable comparisons, and the discovery that a zero-gradient region in the approach potential is what explains the recurring training walls, drove it to its form; that stage was never retrained for random spawn; because the task is decomposed, the spawn randomness is absorbed by the grasp stage alone. A completed controller that moves the object into 95.33% of held-out random-spawn trials and 99.83% of held-out fixed-spawn trials (12 seeds $\times$ 100). and The fact that a basic 14% of the previous fixed-spawn result was obtained by the deterministic rule layer alone, with the network held fixed, is the one clear-cut indication of the decomposed architecture. On the deployment side, both actors were both compressed at almost $\frac{1}{4}$ the original size and approximately $\frac{1}{2}$ the latency, with no changes to task behavior and, in the case of the random spawn grasp actor, enhancements after the quantization campaign of Section 4.6. In the HIL setup, the controller was executed end-to-end on the ESP32, with enough time remaining within the 50 ms time period for the controller to complete 97 of 100 random spawn episodes. It is also able to recover its own object pose on the same silicon: the on-device AprilTag detector and residual corrector have an end-to-end success rate of 89.67% on a 94.00% ground-truth ceiling with approximately 36 KB of heap margin remaining at peak. The controller is compact enough to be controlled by a few dollar microcontroller, and has the speed to drive a 7-DOF arm in real time – sensing its own target with its own camera, not an oracle simulator state – which is what this thesis extension set out to prove. Where the single-object manipulation, simulated physics, and place stage that has not yet been retrained for random spawn would naturally end, a physical-robot continuation would naturally begin. This chapter has shown and explained the technical performance of the proposed system, but it is important to consider engineering solutions in terms of their wider effects. Therefore, the next chapter discusses the socio-economic impact and sustainability of the proposed framework, including its implications for society, health, safety, legal and cultural issues, as well as its environmental impact and contribution to sustainable engineering.

Chapter 7

Socio-Economic Impact and Sustainability

7.1 Introduction

An engineering work should not be judged only by its technical results. It should also be judged by its effect on society, on human health and safety, on legal and cultural matters, and on the environment. The main outcome of this thesis is a method to run deep reinforcement learning (DRL) based manipulator control on a very cheap microcontroller, the ESP32, instead of an expensive and power-hungry computer. This single change, making intelligent robotic control cheap, has many effects outside the laboratory. This chapter discusses these effects. Section 7.2 describes the impact of the thesis on societal, health, safety, legal, and cultural issues. Section 7.3 describes the impact on the environment and sustainability. Section 7.4 gives a short summary of the chapter.

7.2 Impact of the Thesis on Societal, Health, Safety, Legal and Cultural Issues

7.2.1 Societal Impact

Today, intelligent robots are mostly used by large companies because they are very expensive. A normal robot arm with an AI-based controller needs a powerful computer, and the total cost is far beyond the budget of most small organizations. This thesis takes a shot at precisely this cost issue. The task is broken up into small stage-specific policies and compressed using quantization, so that the control system can be executed on the ESP32, which is just a few dollars. When intelligent robotics is cheap, it benefits many groups in the society:

- **Factories:** Not only large industries, but all factories in Bangladesh and around the world can afford intelligent automation. Small factories can increase their productivity without a big investment.
- **Educational institutions and students:** Schools, Universities and polytechnics can purchase affordable robotic platforms for educational purposes. Students will be able

to learn about reinforcement learning and robotics on hardware they can afford to buy, rather than just reading about expensive hardware in books.

- **Small businesses:** For simple pick and place operations, like sorting, packing, material handling, etc., small businesses can use low-cost robots as opposed to the extreme cost of industrial robots.
- **Disabled and elderly people:** Disabled and older people may require assistance with everyday activities. When robots become inexpensive, these people can afford to have a robot assistant for their own work. This can help them to be more independent and have a better quality of life.
- **Busy people and households:** Modern life is becoming busier by the day. If the person is very busy, they can get a low-cost robot for domestic use. There is also a lack of household workers in our society and low-cost robots can help address this need.

In brief, the social message of this thesis is straightforward: As intelligent robots become affordable, human life becomes more convenient with lesser costs, and the benefit is not only for rich companies, but for common people.

7.2.2 Health and Safety Impact

The use of robotic manipulators directly positively impacts worker health. In many factories, employees are performing the same pick and place operation thousands of times daily. This repetitive work can lead to chronic conditions like back pain and joint injuries. Certain activities are also hazardous, such as handling hot, sharp or harmful materials. These repetitive and hazardous jobs can be done by low-cost robot arms, thus protecting human workers from injuries and allowing them to engage in safer and more creative work. Safety should also be taken into account from the other perspective. A reinforcement learning policy is not programmed, but learned by the model itself, through trial and error. This means that you cannot completely determine its behavior based on the code and an untested policy may make an unexpected move. Therefore, an RL-based controller should always be tested prior to using in the vicinity of humans. This thesis adheres to this safe practice. The trained and quantized policies were validated using a hardware-in-the-loop (HIL) test, which involves testing the models on the ESP32 with the simulation using serial communication without interacting with

any real robot. This approach to validation in a step-by-step fashion minimizes the safety risk and demonstrates a responsible manner in which learned controllers can be deployed.

7.2.3 Legal Impact

Autonomous robots' legal aspects are still being developed globally. The question of who is liable if an autonomous robot damages a product or injures a person is not always clear. The owner of the robot, the developer of the learning algorithm or the manufacturer of the robot. This will be more relevant if the controller is a learned model rather than a fixed program. Thus, any commercial application of a system such as the one described in this thesis will have to adhere to the current machine and workplace safety requirements, and the system should be tested and documented before it is deployed. On the software side, this thesis is legally safe. All main tools used in this work, such as TensorFlow and the Robosuite simulation environment, are open-source software with permissive licenses. This implies that the techniques presented in this thesis can be applied, adapted and even commercialized without any license restrictions, thus also contributing to the central aim of the thesis: to make intelligent robotics available at low cost.

7.2.4 Cultural Impact

The primary worry about robots in culture is that they will take the place of humans. This fear is legitimate and should be taken seriously. The present reality, however, is a more balanced one. Human life is becoming busier and the demand for production is increasing at a rapid pace, and human workers are unable to perform so much production. In this case, robots are not replacing workers, they are filling the void between the small number of workers and the increasing demand. In addition, robots generate new forms of technical employment, like robot operation, maintenance and programming. The positive cultural side is also there. There is not enough domestic labor in our society and many families are unable to get domestic labor for work. In society, robots will not be seen as job takers, but as caring helpers if low-cost robots are used to assist disabled and older people in their personal tasks. This can help foster a positive culture of robotics, as technology is viewed as a tool to support human dignity and independence, not a threat.

7.3 Impact of the Thesis on the Environment and Sustainability

7.3.1 Energy Efficiency of the Deployed System

The clearest environmental benefit of this thesis is energy saving. Normally, an intelligent robot controller runs on an AI computer board. The cheapest and most popular board of this type is the NVIDIA Jetson Nano, which officially operates in 5 W or 10 W power modes for the module alone [36]. In contrast, the ESP32 microcontroller used in this thesis consumes only about 30 mA to 68 mA at 3.3 V in active mode at 240 MHz according to the official Espressif datasheet, which is roughly 0.1 W to 0.22 W [35]. This means the ESP32 uses approximately 25 to 100 times less power than the cheapest common AI board, while still executing the quantized DRL policies in real time. Table 4.1 shows this comparison.

Table 7.1: Power consumption comparison between ESP32 and NVIDIA Jetson Nano.

Feature	ESP32 Microcontroller	NVIDIA Jetson Nano
Typical power consumption	$\approx 0.1 - 0.22$ W (30-68 mA at 3.3 V, active mode, 240 MHz) [35]	5 W or 10 W power modes (module only) [36]
Approximate cost	Very low (a few US dollars)	Much higher (developer kit level)
Power ratio	$1\times$ (reference)	$\approx 25 - 100$ times more power

This difference matters at scale. One robot saving a few watts is small, but if thousands of low-cost robots are deployed in factories, schools, and homes, the total energy saving becomes significant. Lower energy use directly means lower electricity demand and lower carbon emissions, especially in countries where electricity is mainly produced from fossil fuels.

7.3.2 Energy Saving in Training

Sustainability is not only about the final deployed system; the training process also consumes energy. Training one large monolithic model for the full pick-and-place task is a long and complex learning problem, so it needs a long training time and therefore more computation energy. In this thesis, the task was divided into two small stage-specific policies, and each

small 64-32 MLP policy was trained separately. The staged training was faster and lighter than training a single large model, which means less energy was used for training. Table 4.2 summarizes the comparison between the full training and the staged training of this thesis

Table 7.2: Training comparison between monolithic and staged approaches.

Training Approach	Training Time	Model Size
Single monolithic model (full task)	About 24 hrs	1.2 MB
Staged models (grasping policy)	About 6 hrs	24KB
Staged models (placing policy)	About 6 hrs	24KB

7.3.3 Right-Sized Hardware and Responsible Use of Resources

Of course, not every intelligent robot can run on an ESP32. A very intelligent robot with vision, speech, and complex decision-making is not possible to deploy on such a small microcontroller. However, many specific tasks, such as the pick-and-place operation of this thesis, are possible on low-cost hardware. This leads to a simple sustainability principle: if low-cost, low-power hardware can do the task and meet the requirements, why should we use powerful hardware? Using a powerful computer for a simple task wastes energy, materials, and money. This thesis shows that with careful design (task division, small networks, and quantization), the hardware can be matched to the real need of the task. This “right-sized hardware” approach is a sensible use of computing resources.

There's a material side as well. The ESP32 is a low silicon, low material mass produced and widely available chip. Low cost, low tech components are also more easily repaired and replaced individually, minimizing electronic waste in the long run.

7.3.4 Relation to the Sustainable Development Goals (SDGs)

While mapping to the Sustainable Development Goals (SDGs) of the United Nations was not part of the thesis requirements, the results of this thesis are naturally aligned with some of the SDGs. Hands on robotics and AI education are made affordable with low-cost robotic platforms that help achieve SDG 4 (Quality Education). Affordable automation enables small factories and businesses to grow, and can generate new technical jobs, thus contributing to SDG 8 (Decent Work and Economic Growth). The work directly contributes to the promotion

of low cost innovation and industrial capability, thus contributing to SDG 9 (Industry, Innovation and Infrastructure). Last but not least, the efficient and appropriate usage of hardware helps achieve SDG 12 (Responsible Consumption and Production).

7.4 Summary

The socio-economic impact and sustainability of the thesis was discussed in this chapter. On social side, the DRL manipulator control can be implemented on a low cost ESP32, making the field of intelligent robotics accessible in Bangladesh and globally to factories, schools, students, small businesses, and even to disabled, old and busy people who need inexpensive assistance in their daily lives. From a health and safety perspective, low-cost robots can replace human workers with repetitive and hazardous tasks, and hardware-in-the-loop validation as performed in this thesis demonstrates a safe and responsible method of testing learned controllers prior to deployment in the real world. The legal aspect is based on open-source software with permissive licenses, but commercial applications will need to adhere to safety requirements and the issue of liability for autonomous systems remains open. In terms of culture, the robot should be regarded as a tool to assist, rather than a competitor, to meet the production needs and lack of domestic workers. Environmentally, the ESP32 uses about 25-100 times less energy than the lowest power common AI board, and the staged training methodology also saved training energy. The overall message is that if the hardware is matched to the actual need of the task, it is an economic and environmental benefit. The next chapter discusses thesis management and finance. It describes how we managed our time and financial resources throughout the completion of this thesis.

Chapter 8

Thesis Management & Finance

8.1 Introduction

Good management and careful use of money are important parts of any engineering project. A thesis project has a fixed time limit and, in most cases, a very small budget, so both must be planned and controlled from the beginning. This chapter describes how this thesis was managed and financed. The management of the thesis is explained in section 8.2, consisting of the team, the supervision process and the time frame of the thesis. The overall project budget is presented in Section 8.3 Resource Planning is provided in Section 8.4. Section 8.5 describes the individual and team work contribution. A summary of the chapter is included in Section 8.6. One thing that should be noted is that the project aimed to keep the cost low, it was indeed a management objective, but also a research objective. The entire thesis is about using a low-cost microcontroller for implementing deep reinforcement learning, rather than an expensive computer. Thus, the very small amount of money allocated to this project, as described in the next sections, is itself an illustration of the thesis idea.

8.2 Management of the Thesis

This thesis was conducted by a group of 2 members namely Hodayra Mobesshira Hemu and Fahim Faisal. The team was organized, and everyone participated in the work of the project in all parts of it: from topic selection to literature review, from training of the model to compression, from validation of the hardware to writing of the thesis. Supervised by thesis supervisor. The team met with the supervisor once a week, during which they discussed the work, problems and the plan for future work. This weekly cycle kept the project on schedule and made sure that every major decision, such as the choice of the algorithm, the staged decomposition approach, and the validation method, was taken with the supervisor's feedback. Each training experiment took several hours to run on the limited hardware, so the team also practiced experimental discipline, changing only one variable for each training run and writing down the expected result of each training run before running it. With this careful working

technique, much time was saved and many experiments were saved.

8.2.1 Time Frame

The thesis work begins on 20 July 2025 with the selection of the thesis and ends in July 2026 with the thesis submission. All technical work was finished prior to the pre-defense on 28 April 2026. The phases of the work and their time periods are presented in Table 8.1, and in Figure 8.1 as a Gantt chart.

Table 8.1: Time frame of the thesis work.

Phase of Work	Time Period
Topic Selection	20 July 2025
Literature Review	From 3 September 2025
Algorithm Selection	October – November 2025
Model Training and Initial Simulations	November 2025 – February 2026
Model Compression (QAT) and Validation	January – April 2026
Pre-Defense	28 April 2026
Thesis Writing and Final Submission	May – July 2026

Table 8.2: Gantt Chart of the Thesis Planning.

Task	Jul–Aug 2025	Sep–Oct 2025	Nov–Dec 2025	Jan–Feb 2026	Mar–Apr 2026	May– Jul 2026
Topic Selection						
Literature Review						
Algorithm Selection						
Model Training						
Initial Simulations						
Model Compression (QAT)						
HIL Validation						
Pre-Defense (28 April 2026)						
Thesis Writing & Submission						

8.3 Overall Budget

The overall expense of this Thesis Project was very minimal. The only hardware purchased was the target hardware, the ESP32 DevKit board. All the software employed in the project is open source and free of cost, the laptop used for training was already owned by the team members and the remaining resources (USB cable for serial communication) was also already owned by the team members. Table 8.2 shows the overall budget of the project.

Table 8.3: Overall budget of the thesis project.

Budget Category	Cost (BDT)
Hardware components	450
Software and tools	0 (open-source)
Existing resources (laptop, USB cable)	0 (already owned)
Total Project Cost	450 BDT

The total cost of 450 BDT (roughly 4 USD) is impressive considering the typical cost of an intelligent robotics system, where the AI computer board alone costs many times more. This directly contributes to the central thesis of this thesis, that is, that intelligent robotic control can be made affordable.

8.3.1 Component/Software Budget

Table 8.3 provides the detailed specification of all the components and software used in the project, along with the purpose and cost. Anything that is already available, or is free open source software, is listed at a cost of zero, to provide a full and honest set of resources for the project.

Table 8.4: Detailed component and software budget.

Component / Software	Purpose	Cost (BDT)
ESP32 DevKit (microcontroller board)	Target deployment hardware for the quantized RL policies	450
USB cable	Serial communication between PC and ESP32 (already owned)	0
Laptop (Intel i5-1135G7, 8 GB RAM)	Model training, simulation, and HIL testing (already owned)	0
PyTorch	Deep learning framework for TD3 training (open-source)	0
robosuite (MuJoCo)	Robot manipulation simulation	0

Component / Software	Purpose	Cost (BDT)
	environment (open-source)	
litert / ai_edge_torch	QAT and INT8 model conversion toolchain (open-source)	0
TFLite Micro (ArduTFLite)	On-device inference engine for the ESP32 (open-source)	0
Arduino IDE	Firmware development environment for the ESP32 (free)	0
Total		450 BDT

8.4 Resource Planning

The resources available in this thesis were quite limited but that's the plan, since the whole thesis is supposed to be low-cost intelligent robotics. The resources can be classified into 4 categories.

The first one is computing. All the training was performed on a standard laptop with 2GB of RAM and no GPU, and we already had, so we used the same laptop for 2 simulation environments at a time. In this machine, it took several hours for one training run. Computing time was the most valuable resource of the project, more than money. To not waste it, we adopted a simple guideline:

Make one change at a time to every training run, record what we are expecting to happen before running it and review our logs thoroughly after each run.

The second one is hardware. We only purchased the ESP32 DevKit board (450 BDT). We had the USB cable here already. No other hardware was required as the testing was conducted hardware-in-the-loop with the simulation, and not with a physical robot arm.

The third one is software. From the beginning, we planned to use only free and open-source software: Python and PyTorch for training, robosuite (MuJoCo) for simulation, the LiteRT / ai_edge_torch tools for INT8 conversion, and the Arduino IDE with TFLite Micro for the ESP32. This is why it was free and anyone can duplicate our work without spending any money.

The last one is people and information. The human resources were our two-member team and

the weekly guidance of our supervisor. As a source of information, we relied on published papers, the official documentation of the tools, and open-source examples. We also formed a thesis group, sharing everything (papers, code, logs, models, and documents) to ensure that both of us had the same up-to-date materials.

8.5 Individual & Team Work Contribution

This thesis was prepared by two members Homayra Mobesshira Hemu and Fahim Faisal and we worked as a team from the first day till the last. For progress meetings we went to our supervisor together once a week. In addition to these meetings, we also had private meetings alone, not with the supervisor, to talk about any problems that arose each day and next steps. The main idea of the thesis the TD3 algorithm, the idea of staged decomposition, the quantization method and the way of validation was taken together, after discussion between both of us. A thesis group was set up for this project, and all the resources such as papers, code, training logs, models and drafts were shared there. Both members knew all of the work, not just their own part. Within this teamwork, the simulation portion (environment setup of the robosuite and conducting the training experiments) was personally carried out by Fahim Faisal. The rest of the work, such as the literature review, analyzing the training results, the quantization and deployment pipeline, the hardware-in-the-loop testing, and writing this report, was done with the contribution of both members. Thus, the thesis combined the two aspects of individual responsibility and teamwork.

8.6 Summary

This chapter introduced the handling of thesis and the cost of the thesis. The two-member team conducted the work under the weekly supervision from July 2025 to July 2026, the technical work was completed before the pre-defense on 28 April 2026. The total cost was just Rs 450, as the ESP32 DevKit was the only component that was bought (all the software was open source, and the rest were already owned). An interesting point that the resource planning revealed was that the amount of money was not the limiting factor, but rather the amount of computing time, and that the computing time was saved due to a disciplined one change per run working style. The contribution section revealed that the team worked very

together: all major decisions were made together, all the resources were shared, the simulation was personally managed by Fahim Faisal and both team members contributed in all the other aspects. This chapter tells the whole story of the management and resources. The last chapter summarizes the results of the thesis, acknowledges the limitations, and provides directions for future research.

Chapter 9

Conclusions, Limitations and Future Works

9.1 Conclusion

In this thesis it was explored if a simple a robotic manipulation skill that normally needs a large deep reinforcement learning model can be deployed on a cheap widely available microcontroller? The answer, demonstrated step by step through this work, is yes. The problem task was decomposed correctly to achieve this success smoothly.

Instead of training a larger monolithic DRL model (A single model handling the task from start to finish) a staged reinforcement learning framework in which the pick-and-place task is divided into small stage-specific policies, so that each policy uses a compact neural network instead of one large monolithic model was successfully trained. These trained models was synched with a P-controller at the last stage

The trained model was successfully quantized to 8-bit integer format using quantization-aware training (QAT) to make the model even more lighter and thus increasing inference speed.

To verify the work, hardware-in-the-loop (HIL) test was performed to confirm synchronization the FSM involving the staged the quantized policies along with the P-controller running on the ESP32 through serial communication to send control actions and receive environment feedback in real time while maintaining task performance. The Franka Panda robot was simulated in this process.

The training journey itself produced valuable methodological findings. Problems such as reward farming, grip instability during motion, the safe-hold local optimum, the zero-gradient region in the reward field, and replay-composition pathologies were diagnosed one by one through a disciplined, log-driven, one-variable-per-run methodology, and each was solved with a targeted fix such as potential-based reward shaping, a reverse curriculum, and early

termination of non-progressing episodes. The final architecture reached a 92% end-to-end success rate in deterministic evaluation, and notably, the improvement from 78% to 92% came only from tuning the rule-layer parameters of the FSM direct evidence that the combination of a learned policy for gross motion and a deterministic rule layer for precise terminal behavior is the right division of labor.

On the deployment side, both actor networks were fine-tuned with quantization-aware training and converted to full-integer INT8 models, making them about four times smaller. The models were embedded into the ESP32 firmware and validated by hardware-in-the-loop testing, where the real ESP32 computed every control action for the simulation over a framed serial protocol. The INT8 inference took about 15 milliseconds, fitting comfortably within the 50-millisecond control cycle. This closed the deployment claim of the thesis: the decomposed policies are not merely small on paper they run, in real time, on a low-cost microcontroller, in the loop with physics. All of this was achieved with a total project cost of only 450 BDT, because every software tool used was open-source.

In conclusion, this thesis demonstrated that instead of training a model for completing full task makes the model complex and of larger size such that, the model may not fit in the flash memory or RAM of MCU. And even if it fits, the required inference frequency may not be high enough for reliable continuous control tasks. Therefore, model sizes and complexity of model must be reduced. Moreover, reward engineering for such a task proved very difficult in achieving convergence in training. To solve this, decomposing the task into multiple independent stages makes the above mentioned problems solvable, as it needs an overall decreased amount of model size, in turn solving the inference frequency and task learning complexity. Therefore, this thesis is an step toward future to make intelligent robotic control affordable for factories, education, small businesses, and ordinary people.

9.2 Limitations

Although the results of this thesis are promising, the work has several honest limitations that must be stated clearly:

- **Limited to simple tasks:** This approach is not meant for extra intelligent or highly advanced systems. It works for easy and simple tasks whose policies can be compressed enough to fit into an ESP32. A robot that needs vision, language, or complex decision-making cannot be deployed this way.
- **Hardware not designed for AI:** The ESP32 is not designed to do AI inference. To achieve a sufficiently fast control loop, the model size minimization was carefully performed by both compact architecture design (64-32 MLP) and INT8 quantization. If the chip was larger or more powerful, it would not have been able to afford the real-time budget.
- **Simulation-only validation:** Validation was conducted in simulation only (HW-in-the-loop). There was no physical Franka Emika Panda robot, so some physical factors like sensor noise, motor dynamics and physical delays were not tested.
- **Communication latency:** At each timestep the high dimensional state vector (46 values) is sent over USB serial, which adds latency. This reduces the control frequency that can be obtained in a real deployment.
- **Single task only:** Single task (PickPlaceBread) was the only task performed. The generalization to more complex manipulation tasks or to other robotic platforms was not investigated.

9.3 Future Works

The above restrictions clearly specify the future directions of this work. The essence of the staged approach is general; not just for robotic arms. Any system that is critical or difficult to train as one piece can be broken down into a number of parts, and the training of each part is much easier. On this idea and on the technical results of the thesis, the following future works are proposed:

- **Real hardware deployment:** Deploy the ESP32 with the INT8 models on a real low-cost robotic arm to validate the control quality under real-world physical constraints and sensor noise.
- **Staged adaptive domain randomization.**[29],[30],[31].

- **A joint optimization of quantization-aware training with the policy gradient.**
- **A unified benchmark for the comparison of staged RL deployments on microcontrollers.**
- **Spiking Neural Network (SNN):** Incorporate SNN to explore even more efficiency.
- **More complex tasks:** Scale to more complex manipulation tasks, e.g., multi-step tasks, multi-object tasks, and assess if the compression technique scales to larger and more powerful policies.
- **On-device adaptation:** Explore federated or on-device fine-tuning approaches to adapt the compressed models to real hardware characteristics without full re-training [4].
- **Other edge platforms:** Test the deployment on other edge AI platforms, namely the STM32 (CubeAI) and the Raspberry Pi Pico, and compare their performance, power consumption and inference speed with the ESP32..
- **Pruning with quantization:** Explore structured pruning and quantization to achieve additional model size reduction with minimal impact on performance [37] [38].
- **Sim-2-real Transfer:** Deploy the virtually trained staged decomposed model on real manipulator [30], [31], [39].

The work in this thesis can be expanded with these extensions to become a viable and widely applicable system for the deployment of learned intelligence on the most affordable hardware.

References

- [1] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [2] S. Levine, C. Finn, T. Darrell, and P. Abbeel, "End-to-end training of deep visuomotor policies," *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, 2016.
- [3] J. Kober, J. A. Bagnell, and J. Peters, "Reinforcement learning in robotics: A survey," *Int. J. Robotics Research*, vol. 32, no. 11, pp. 1238–1274, 2013.
- [4] R. David et al., "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800-811.
- [5] J. Lin, W.-M. Chen, Y. Lin, J. Cohn, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [6] S. Fujimoto, H. van Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Machine Learning (ICML)*, 2018, pp. 1587-1596.
- [7] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, "Continuous control with deep reinforcement learning," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2016.
- [8] D. Han, B. Mulyana, V. Stankovic, and S. Cheng, "A survey on deep reinforcement learning algorithms for robotic manipulation," *Sensors*, vol. 23, no. 7, p. 3762, 2023.
- [9] M. Andrychowicz et al., "Hindsight experience replay," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [10] T. D. Kulkarni, K. Narasimhan, A. Saeedi, and J. Tenenbaum, "Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation," in *Proc. NeurIPS*, 2016.
- [11] X. Liu, G. Wang et al., "Hierarchical reinforcement learning with human knowledge for staged robotic manipulation," *IEEE Trans. Automation Science and Engineering*, 2023.
- [12] R. S. Sutton, D. Precup, and S. Singh, "Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning," *Artificial Intelligence*, vol. 112, no. 1–2, pp. 181–211, 1999.
- [13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704-2713.

- [14] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer, "A survey of quantization methods for efficient neural network inference," arXiv:2103.13630, 2021.
- [15] P.-L. Bacon, J. Harb, and D. Precup, "The option-critic architecture," in Proc. AAAI Conf. Artificial Intelligence, 2017, pp. 1726–1734.
- [16] A. S. Vezhnevets et al., "FeUdal networks for hierarchical reinforcement learning," in Proc. Int. Conf. Machine Learning (ICML), 2017, pp. 3540–3549.
- [17] M. Kang and S. Oh, "Latent skill chaining: Linking independently trained skills using bridge policies and transition classifiers," Intelligent Service Robotics, 2022.
- [18] J. B. Alves, N. Lau, and F. Silva, "Curriculum skill chaining: Learning subtasks and curriculum-guided transition policies from a single demonstration," Robotics and Autonomous Systems, 2025.
- [19] O. Nachum, S. Gu, H. Lee, and S. Levine, "Data-efficient hierarchical reinforcement learning," in Proc. NeurIPS, 2018.
- [20] D. Hoeller, N. Rudin, D. Sako, and M. Hutter, "ANYmal parkour: Learning agile navigation for quadrupedal robots," Science Robotics, vol. 9, no. 88, p. eadi7566, 2024.
- [21] N. Zhang, Y. Zhao et al., "Hierarchical reinforcement learning with demonstration-based object-centered sub-goals for long-horizon manipulation," Arabian Journal for Science and Engineering, 2025.
- [22] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in Proc. Int. Conf. Machine Learning (ICML), 2009, pp. 41–48.
- [23] L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient neural network kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.
- [24] S. Krishnan, B. Boroujerdian, W. Fu, A. Faust, and V. J. Reddi, "Air Learning: A deep reinforcement learning gym for autonomous aerial robot visual navigation," Machine Learning, vol. 110, no. 9, pp. 2501–2540, 2021.
- [25] K. Badalian, L. Koch, T. Brinkmann, M. Picerno, M. Wegener, S.-Y. Lee, and J. Andert, "LEnCI: A framework for reinforcement learning with embedded systems," Applied Intelligence, vol. 54, no. 17–18, pp. 8384–8398, 2024.
- [26] L. Xin, H. Tian, and B. Sheng, "HAfER: Adaptive rewards with failure experience replay for staged manipulation," Neurocomputing, 2026.
- [27] J. Wang, W. Zhang et al., "Knowledge distillation for transferring key features to compact student networks," Computer Vision and Image Understanding, 2023.
- [28] F. Muratore, F. Ramos, G. Turk, W. Yu, M. Gienger, and J. Peters, "Robot learning from randomized simulations: A review," Frontiers in Robotics and AI, vol. 9, 2022.

- [29] H. Ju, R. Juan et al., "Transferring policy learning from simulation to the real world: A survey of domain randomization and adaptation," *Nature Machine Intelligence*, 2022.
- [30] Y. Heo and S. Lee, "GAN-based knowledge distillation with lightweight generators for edge deployment," *Electronics*, 2022.
- [31] Y. Chu, P. Li et al., "Channel pruning combined with attention-based knowledge distillation for compact models," *Applied Intelligence*, 2022.
- [32] M. Sailesh, K. Selvakumar, and N. Prasanth, "Automated INT8 post-training quantization with CMSIS-NN deployment for microcontrollers," *Microprocessors and Microsystems*, 2022.
- [33] P.-E. Novac, G. Boukli Hacene, A. Pegatoquet, B. Miramond, and V. Gripon, "Quantization and deployment of deep neural networks on microcontrollers," *Sensors*, vol. 21, no. 9, p. 2984, 2021.
- [34] P. Kurrek, F. Zoghlami et al., "A reinforcement-learning development methodology using subtask rewards and hierarchical execution (Q-Model)," *Journal of Computing and Information Science in Engineering*, 2020.
- [35] A. Capotondi, M. Rusci, M. Fariselli, and L. Benini, "CMix-NN: Mixed low-precision CNN library for memory-constrained edge devices," *IEEE Trans. Circuits and Systems II: Express Briefs*, vol. 67, no. 5, pp. 871–875, 2020.
- [36] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain randomization for transferring deep neural networks from simulation to the real world," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30.
- [37] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-real transfer of robotic control with dynamics randomization," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2018, pp. 3803–3810.
- [38] Y. Zhu, J. Wong, A. Mandlekar, and R. Martín-Martín, "robosuite: A modular simulation framework and benchmark for robot learning," *arXiv:2009.12293*, 2020.
- [39] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [40] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, "Deterministic policy gradient algorithms," in *Proc. Int. Conf. Machine Learning (ICML)*, 2014, pp. 387–395.
- [41] V. R. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2000.

- [42] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," in Proc. Int. Conf. Learning Representations (ICLR), 2016.
- [43] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in Proc. Int. Conf. Machine Learning (ICML), 1999, pp. 278–287.
- [44] NVIDIA Corporation, "Jetson Nano Developer Kit Module Data Sheet," NVIDIA. [Online]. Available: <https://developer.nvidia.com/embedded/jetson-nano>
- [45] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems, 2023. [Online]. Available: <https://www.espressif.com/en/products/socs/esp32>
- [46] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in Proc. Int. Conf. Learning Representations (ICLR), 2016.
- [47] A. Mandlekar, D. Xu, J. Wong, S. Nasiriany, C. Wang, R. Kulkarni, L. Fei-Fei, S. Savarese, Y. Zhu, and R. Martín-Martín, "What matters in learning from offline human demonstrations for robot manipulation," in *Proc. 5th Conf. Robot Learning (CoRL), Proc. Mach. Learn. Res.*, vol. 164, 2022, pp. 1678–1690.
- [48] A. Agarwala, A. Das, B. Juba, R. Panigrahy, V. Sharan, X. Wang, and Q. Zhang, "One Network Fits All? Modular versus Monolithic Task Formulations in Neural Networks," in *Proc. 9th Int. Conf. Learning Representations (ICLR)*, 2021.
- [49] J. Pfeiffer, S. Ruder, I. Vulić, and E. M. Ponti, "Modular Deep Learning," *Trans. Mach. Learn. Res. (TMLR)*, 2023.
- [50] L. Marzari, A. Pore, D. Dall'Alba, G. Aragon-Camarasa, A. Farinelli, and P. Fiorini, "Towards Hierarchical Task Decomposition using Deep Reinforcement Learning for Pick and Place Subtasks," in *Proc. 2021 20th Int. Conf. Advanced Robotics (ICAR)*, 2021, pp. 640–645.

Appendix

A. List of Modern Tool usage

Table A.1: List of Modern Tools Used in the Development of the Decomposed TD3-Based Pick-and-Place System

Sl. No.	Tool / Technology	Category	Usage in This Project
1	Python 3	Scripting & ML Development Language	Implementation of training environment and wrappers (grasp_env_wrapper.py, place_env_wrapper.py), training scripts, and HIL orchestration (hil_main.py).
2	C++ (Arduino / ESP32)	Embedded Programming Language	ESP32 firmware for HIL validation.
3	PyTorch	Deep Learning Framework	Implementation, training of TD3 actor-critic networks (twin critics, delayed actor updates, target policy smoothing).
4	robosuite (MuJoCo)	Robotics Simulation Platform	PickPlace task environment setup with Panda 7-DOF arm and OSC_POSE control at 20 Hz.
5	Prioritized Experience Replay (SumTree)	Reinforcement Learning Sampling Technique	Priority-weighted transition sampling from the 200,000-capacity replay buffer.
6	pt2e / ai_edge_torch (PyTorch Quantizer)	Quantization-Aware Training Toolkit	QAT fine-tuning of trained FP32 actors with fake-quantization prior to INT8 conversion (qat_finetune.py).
7	litert_torch / TensorFlow Lite Converter	Model Conversion Toolkit	Conversion of QAT-fine-tuned actors to full-integer INT8 .tflite models (qat_and_convert.py).
8	TensorFlow Lite Micro (TFLite-Micro)	Edge Inference Framework	On-device INT8 inference of the grasp and place actor models on the ESP32.
9	ArduTFLite	Arduino TFLite-Micro Wrapper Library	Interfacing the TFLite-Micro interpreter within the Arduino firmware environment.
10	PySerial	Serial Communication Library	USB serial link (921,600 baud) between the PC-side orchestrator and the ESP32 in the HIL loop.
11	NumPy	Numerical Computing Library	Implementation of various works that require numerical, tabular data points.
12	Matplotlib	Data Visualization Library	Generation of visual statistics figures for the thesis.
13	Git / GitHub	Version Control System	Source-code management (repository: github.com/fahimcodebag/pickandplace).

B. Program Outcome Mapping

Table B.1: Justification of Program Outcomes (CO–PO Mapping)

CO	PO	Addressed by This Thesis	Chapter Reference
CO1	PO1 Engineering knowledge	This thesis makes use of reinforcement learning, control theory and embedded systems knowledge. Application of model-compression techniques also takes place. The result is a decomposed Deep RL system for manipulator control on a microcontroller. An FSM meta-controller does the coordination of learned grasp and transport sub-policies with scripted micro-skills.	Chapters 3 and 4
CO2	PO2 Problem analysis	Identification of the exploration and credit-assignment breakdown happens for monolithic DRL policies. This breakdown affects long-horizon, sparse-reward manipulation tasks. A survey of the options framework, hierarchical RL and sub-MDP decomposition is carried out. Therefore, the conclusion is reached that a staged, decomposed architecture is needed.	Chapters 1 and 2
CO3	PO3 Design/development of solutions	Design and development of the FSM meta-controller takes place for pick-and-place decomposition. Grasp and transport sub-policies get interleaved with scripted micro-skills, such as the test-lift filter and four-phase release. Definition of model-size, inference-latency and inference-frequency requirements is also included. So, matching design responses are given for these requirements.	Chapters 3 and 4
CO4	PO4 Investigation	A systematic, instrumented investigation of the training campaign is conducted. Per-episode done-reason tallies and drop diagnostics form part of the method. One-variable-at-a-time comparisons and a penalty-shape sweep are also carried out. Diagnosis and resolution of reward farming, grip instability and marginal handoffs happen through this investigation.	Chapters 5 and 6
CO5	PO5 Modern tools usage	Use of contemporary engineering and IT tools happens throughout this work. robosuite/MuJoCo simulation and a PyTorch TD3 pipeline with Prioritized Experience Replay are examples. Quantization-aware training, TensorFlow Lite Micro and a custom ESP32 test bench are also used. These tools support design, development and validation of the solution.	Chapters 4, 5 and 6
CO6	PO6 Society, health, safety, legal & cultural	Evaluation of societal implications takes place, covering affordable robots for small businesses and disabled or elderly users. Health-and-safety implications also get	Chapter 7

CO	PO	Addressed by This Thesis	Chapter Reference
		discussed. Legal matters, such as liability of learned controllers and open-source licensing, are addressed. Cultural implications of low-cost intelligent manipulation are considered too.	
CO7	PO7 Environment & sustainability	Assessment of environmental implications is made for the deployed system. Comparison of ESP32 power consumption against a GPU-class embedded board is included. Energy savings from right-sizing training and deployment hardware get discussed. Alignment with relevant UN Sustainable Development Goals is also shown.	Chapter 7
CO8	PO8 Ethics	Upholding of professional and ethical responsibility happens by reliance on open-source, permissively licensed tools only. TensorFlow and robosuite are examples of such tools. Documentation of a reproducible experimental procedure is provided. Discussion of liability implications for learned controllers is also included.	Chapters 5 and 7
CO9	PO9 Individual & team work	Demonstration of both individual and team performance is shown in this work. The simulation environment and training experiments were individual efforts. Sharing of the literature review, quantization pipeline and hardware-in-the-loop testing happened between team members. Joint decision-making occurred for the core methodology.	Chapter 8
CO10	PO10 Communication	Communication of the complex engineering work happens through the structured thesis report. Consolidated tables and figures of results support this communication. Periodic progress presentations were also delivered to the supervisory board.	Whole Report / Presentations
CO11	PO11 Project management	Exhibition of management expertise happens through a defined multi-phase time frame, from topic selection to final submission. Gantt-chart planning and a component/software budget are part of this. A cost-conscious objective of using low-cost hardware runs throughout the project.	Chapter 8
CO12	PO12 Lifelong learning	Identification of the present work's limitations is made, including single-object validation only. Hardware-in-the-loop testing was done instead of physical-arm testing. A laptop-limited hyperparameter search was another constraint. Outlining of future work happens too, therefore reflecting lifelong learning in a changing technological context.	Chapter 9

C. Addressing Knowledge Profiles (KPs) in Thesis/Project

A statement of the knowledge profiles is given in this appendix. The thesis is a hybrid reinforcement learning system for a pick-and-place task. Table A.1 gives the mapping of each profile to the work. A reference to the relevant section is in the last column.

Table C.1: Addressing of the Knowledge Profiles in this thesis

KP No.	Attribute of the Knowledge Profile	Addressing of the Attribute in This Thesis	Section
KP1	A systematic, theory-based understanding of the natural sciences applicable to the discipline.	Rigid-body dynamics is the natural science base here. The motion of the arm is described by Newton's laws. Grasp stability gets its determination from gravity, friction and normal forces, which matter quite a lot. MuJoCo does the computation of all this. So, the simulator holds a physics model and not a data model.	Sec. 3.2, Sec. 3.6, Sec. 5.3
KP2	Conceptually-based mathematics, numerical analysis, statistics and formal aspects of computer science to support analysis and modelling.	The task received a formulation as a Markov Decision Process. Probability theory is the base of it. Gradient descent, Bellman updates and target smoothing are used by TD3. A SumTree does the sampling of the replay buffer. Success rate, mean return and latency are given calculation by statistics. The episode count is fixed.	Sec. 3.2, Sec. 4.2, Sec. 5.9
KP3	A systematic, theory-based formulation of engineering fundamentals required in the engineering discipline.	Control theory is the big fundamental here. A pose command is converted into joint torques by the operational-space controller. Sampling theory gave the choice of the 20 Hz rate. The framed serial link was designed from digital system fundamentals. Number representation is another one. Float values get a mapping into int8 values.	Sec. 3.5, Sec. 4.4, Sec. 5.4
KP4	Engineering specialist knowledge that provides theoretical frameworks and bodies of knowledge for the accepted practice areas; much is at the forefront of the discipline.	Deep reinforcement learning is the specialist area. TD3 is used, and prioritized experience replay and reward shaping are also in use. Justification for the decomposition comes from the options framework. A reverse curriculum gave the solution of the sparse reward trouble. Quantization-aware training on microcontrollers is quite a new topic. Therefore, the work sits near the edge of practice.	Ch. 2, Sec. 4.3, Sec. 4.4
KP5	Knowledge that supports engineering design in a practice area.	Design knowledge made the selection of the hybrid architecture. Two learned policies and scripted routines are composed by a deterministic FSM. SRAM and Flash limits of the ESP32 fixed the network size at 64-32. The inference budget got its decision from latency limits. Requirements and constraints were stated first. Then the design work started.	Sec. 3.3, Sec. 3.4, Sec. 3.5

KP No.	Attribute of the Knowledge Profile	Addressing of the Attribute in This Thesis	Section
KP6	Knowledge of engineering practice (technology) in the practice areas in the engineering discipline.	Practice knowledge covers tools and hardware. Python, PyTorch, robosuite and MuJoCo are used on the training side. The deployment target is the ESP32 board. Host and board are joined by a framed serial protocol. Real timing is measured on a hardware-in-the-loop bench. Calibration of the instruments happened first.	Sec. 3.2, Ch. 5
KP7	Comprehension of the role of engineering in society and identified issues in engineering practice: ethics, public safety, and economic, social, cultural, environmental and sustainability impacts.	Societal, health, safety, legal and cultural issues are discussed in Chapter 7. Automation of this type has effects on jobs. Access to automation gets a widening from low-cost hardware. The safety argument is made stronger by deterministic FSM supervision. Energy saving is quite large in training and in deployment. SDG 9 and SDG 12 have a mapping. Cost has a full statement.	Ch. 7, Ch. 8
KP8	Engagement with selected knowledge in the research literature of the discipline.	Journal papers, conference papers and framework documentation are the base of the review. Comparison of the existing works took place. Identification of the research gap came from this reading. The choice of TD3, of PER and of QAT has support from the cited works. Sources are in the reference list.	Ch. 2, Sec. 2.5, References

All eight knowledge profiles have an address in this thesis. Therefore, the requirement of the complex engineering problem is met by the work.

D. Addressing Complex Engineering Problems

The table below describes how this thesis addresses each attribute of a complex engineering problem.

Table D.1: Mapping of Complex Engineering Problems

Class	Topic	Description
P1	Depth of Knowledge Required	Deep knowledge of many areas was needed for this implementation. Understanding of deep reinforcement learning was required, including the TD3 algorithm and prioritized experience replay. Knowledge of hierarchical RL and the options framework was also necessary. Reward shaping theory needed careful study. Programming skill in Python and PyTorch was a requirement. Use of the robosuite (MuJoCo) simulation environment was involved. Application of INT8 quantization and

Class	Topic	Description
		quantization-aware training (QAT) was required. Embedded C++ programming through the Arduino IDE was a demand. Deployment of TFLite Micro inference on the ESP32 was needed. Design of the serial communication protocol was also required.
P2	Range of Conflicting Requirements	A balance of many conflicting things was the main challenge. High success rate in pick-and-place was one goal. Small model size, only a few KB, was another demand (64-32 MLP). The ESP32 has strict SRAM and Flash limits. So the memory limit was a big restriction. Real-time control needed inference inside a 50 ms cycle. Low hardware cost was also a requirement. Training resources were limited too. Only a consumer laptop with 2 parallel simulations was available for training.
P3	Depth of Analysis Required	Deep analysis of logs was required to find failure causes. Diagnosis and separation of different failure modes was necessary. Reward farming was one such failure mode. Grip instability was another problem found. The safe-hold local optimum caused trouble too. A zero-gradient region in the approach potential was a difficult issue. Pathologies in replay composition also needed identification. A strict one-variable-per-run method was followed for testing. Integration of very different fields was required: reinforcement learning, classical control (P-controllers and FSM), and embedded systems engineering.
P4	Familiarity of Issues	Handling of many unfamiliar topics was the main task. Reinforcement learning theory was new at the start. The TD3 algorithm was also unfamiliar. Learning of model quantization and QAT was required. TFLite Micro deployment was a new area too. Use of the robosuite simulation environment needed study. Hardware-in-the-loop (HIL) testing was unfamiliar as well. Design of serial communication protocol was also new.
P5	Extent of Applicable Codes	No specific industry standard was required for this research. However, compliance with open-source license terms was maintained. PyTorch, robosuite, and TFLite Micro licenses were followed carefully. Responsible engineering practice was also followed in this work. Validation of learned controllers through HIL testing happened before deployment. This validation was done before any real-world use.

Class	Topic	Description
P6	Stakeholder Involvement & Conflicts	A balance among many stakeholder needs was necessary. Requirements from the supervisor and department created one demand. Thesis milestones and the pre-defense deadline added pressure. A strict budget limit was another constraint. Only a \$5-class microcontroller was allowed, with no GPU for training. High end-to-end success rate was still the performance goal. So multiple, conflicting demands needed careful management together.
P7	Interdependence of System Components	Strong interdependence between sub-systems was managed carefully. The grasp policy, test-lift filter, transport policy, and release routine are chained. This chaining happens through the FSM. The terminal state of one stage becomes input for the next. This creates a handover state distribution between stages. The PC simulation and the ESP32 also form a closed serial loop. In this loop, quantization error, inference delay, and communication jitter interact. Together, these factors affect the final success rate.

E . Addressing Complex Engineering Activities

The table below describes how the activities of this thesis satisfy the attributes of complex engineering activities.

Table D.1: Addressing Complex Engineering Activities (A1–A5)

Clause	Topic	Description	Remarks
A1	Range of Resources	Used a wide range of resources: PyTorch and the robosuite (MuJoCo) simulator for TD3 training on a consumer laptop without a GPU; the litert / ai_edge_torch toolchain for QAT and INT8 conversion; and the ESP32 microcontroller programmed in C++ with the Arduino IDE running TFLite Micro, connected through a USB serial link.	Careful selection of free and low-cost resources allowed the complete training-to-deployment pipeline to be built without expensive hardware.
A2	Level of Interaction	Ensured seamless interaction between very different components: learned policies and scripted routines interact through the FSM, and the PC simulation	Designing a robust, self-resynchronizing communication protocol and making all components

Clause	Topic	Description	Remarks
		interacts with the ESP32 through a framed serial protocol (sync pattern, sequence numbers, checksum, retries) at 921600 baud, including INT8 quantization and dequantization of every message.	work in one closed loop required tedious and time-consuming effort.
A3	Innovation	Decomposed the pick-and-place task so that each tiny 64-32 policy learns only one behavior, and turned the worst failure mode (holding the object over the bin) into the stage-transition condition for a scripted release. Applied QAT INT8 compression so the RL policies run in about 15 ms on a \$5-class ESP32, and improved the end-to-end success from 78% to 92% only by tuning the rule-layer (FSM) parameters.	The division of labor between learning (gross transport) and deterministic control (precise terminal placement) is the key innovation that made low-cost deployment possible.
A4	Societal & Environmental Impact	Aims to make intelligent robotic manipulation affordable for factories, education, and small businesses by replacing a power-hungry AI computer with a low-power ESP32 microcontroller, which consumes far less energy and material.	Addresses sustainability through right-sized hardware: if a cheap, low-power chip can meet the task requirement, powerful hardware is not needed.
A5	Familiarity	Implemented and fused existing well-known techniques TD3, the options framework, finite state machines, potential-based reward shaping, reverse curriculum, QAT, TFLite Micro, and HIL testing in a new combination to innovatively produce a reliable, low-cost embedded RL system.	Usage of standard, well-documented techniques and tools makes future upgrading and reproduction of the system easy.